TecPro

TECNOLOGIAS

Desenvolvimento, Apostilas e Cursos Online



Fluxograma

se...entao

Portugol

C++

Linguagem



Repetição

Algoritmos e Programação

Lógica, Portugol e C++

Algoritmos e Programação

Lógica, Portugol e C++

TecPro Tecnologias

Desenvolvimento, Apostilas e Cursos Online

2026

Todos os direitos reservados. Nenhuma parte desta publicação pode ser reproduzida, armazenada ou transmitida de qualquer forma, ou por qualquer meio, sem autorização prévia da TecPro Tecnologias.

Sumário

Capítulo 1 – Lógica, algoritmos e o computador **6**
1.1 Lógica . . . 6
1.2 Algoritmos . . . 6
1.3 Como o computador executa um algoritmo . . . 7
1.4 Do problema ao algoritmo . . . 9
1.5 Fluxogramas . . . 9
1.6 Portugol . . . 10
Teste de mesa . . . 11
1.7 Do Portugol ao C++ . . . 12
Erros Comuns . . . 13

Capítulo 2 – Linguagens de programação **14**
2.1 O que é uma linguagem de programação . . . 14
2.2 Sintaxe e semântica . . . 14
As peças de um programa . . . 15
2.3 Compilação e interpretação . . . 15
As etapas da compilação em C++ . . . 16
2.4 O ambiente de programação . . . 17
2.5 A estrutura de um programa em C++ . . . 17
2.6 Saída de dados com cout . . . 18
2.7 Entrada de dados com cin . . . 19
2.8 Um programa completo . . . 19
2.9 Lendo as mensagens de erro . . . 20
Erros Comuns . . . 21

Capítulo 3 – Variáveis, tipos e operadores **22**
3.1 Variáveis . . . 22
3.2 Tipos de dados . . . 23
3.3 Operadores aritméticos . . . 24
3.4 Conversões de tipo . . . 25
3.5 Operadores relacionais . . . 26
3.6 Operadores lógicos . . . 26
3.7 Precedência geral . . . 27
3.8 Funções matemáticas . . . 28
3.9 Um programa completo . . . 28
Erros Comuns . . . 29

Capítulo 4 – Estruturas de decisão **30**
4.1 Decisão simples: if . . . 30
4.2 Decisão composta: if e else . . . 31
4.3 Decisões encadeadas: else if . . . 32
4.4 Decisões aninhadas . . . 33
4.5 Escolha múltipla: switch . . . 34
4.6 O operador ternário . . . 36
4.7 Armadilhas das decisões . . . 36
Erros Comuns . . . 37

Capítulo 5 – Estruturas de repetição **39**
5.1 Repetição com teste no início: while . . . 39
Laço controlado por sentinela . . . 40
5.2 Repetição com teste no fim: do-while . . . 41

5.3 Repetição com contador: for . . . 42
5.4 Contadores, acumuladores, maior e menor . . . 43
5.5 Interrompendo um laço: break e continue . . . 44
5.6 Laços aninhados . . . 45
5.7 O laço que não termina . . . 46
5.8 Um programa completo: números primos . . . 46
Erros Comuns . . . 47

Capítulo 6 – Funções e procedimentos **49**
6.1 Dividir para conquistar . . . 49
6.2 Funções: recebem dados e devolvem um resultado . . . 49
Mais de um return . . . 51
6.3 Procedimentos: executam uma tarefa . . . 52
6.4 O que acontece em uma chamada . . . 53
6.5 Escopo: onde cada variável existe . . . 53
6.6 Passagem por valor e por referência . . . 54
Devolvendo mais de um resultado . . . 56
Referência constante . . . 56
6.7 Protótipos, valores padrão e sobrecarga . . . 57
Protótipos . . . 57
Valores padrão e sobrecarga . . . 57
6.8 Um programa completo: notas da turma . . . 58
Erros Comuns . . . 59

Capítulo 7 – Vetores e matrizes **60**
7.1 O que é um vetor . . . 60
7.2 Percorrendo um vetor com laços . . . 61
7.3 Cuidado com os limites . . . 62
7.4 Algoritmos clássicos com vetores . . . 63
Contagem por categoria . . . 63
Maior elemento e sua posição . . . 63
Invertendo um vetor . . . 64
7.5 Vetores e funções . . . 64
7.6 Matrizes . . . 65
7.7 Um programa completo: jogo da velha . . . 67
7.8 Vetores com tamanho variável: vector . . . 68
Erros Comuns . . . 69

Capítulo 8 – Caracteres e strings **70**
8.1 Caracteres são números . . . 70
8.2 O tipo string . . . 71
8.3 Lendo textos . . . 72
8.4 Percorrendo uma string . . . 72
8.5 Procurando, recortando e convertendo . . . 73
8.6 Algoritmos com strings: palíndromos . . . 74
8.7 Acentos e a codificação UTF-8 . . . 75
8.8 Strings no estilo C . . . 76
8.9 Um programa completo: a cifra de César . . . 76
Erros Comuns . . . 78

Capítulo 9 – Recursão e backtracking **79**
9.1 Uma função que chama a si mesma . . . 79
9.2 O que acontece na pilha de chamadas . . . 80

9.3 Pensando recursivamente . . . 81
9.4 Quando a recursão desperdiça: Fibonacci . . . 82
9.5 Recursão ou laço? . . . 83
9.6 As Torres de Hanói . . . 84
9.7 Backtracking: tentar, avançar e voltar atrás . . . 85
9.8 Um programa completo: as oito rainhas . . . 85
Erros Comuns . . . 87

Capítulo 10 – Busca e ordenação **89**
10.1 Busca sequencial . . . 89
10.2 Busca binária . . . 90
10.3 Quanto custa um algoritmo . . . 91
10.4 Ordenação por seleção . . . 92
10.5 Ordenação por bolha . . . 93
10.6 Ordenação por inserção . . . 94
10.7 Comparando os algoritmos e usando a biblioteca . . . 95
O sort da biblioteca . . . 96
10.8 Um programa completo: classificação da turma . . . 97
Erros Comuns . . . 98

Capítulo 11 – Introdução às estruturas de dados **100**
11.1 Registros . . . 100
11.2 Vetores de registros . . . 101
11.3 Estruturas de dados e suas operações . . . 102
11.4 Pilha . . . 102
11.5 Fila . . . 103
11.6 Ponteiros e memória dinâmica . . . 105
11.7 Lista encadeada . . . 105
11.8 As estruturas da biblioteca . . . 107
11.9 Um programa completo: parênteses balanceados . . . 108
Erros Comuns . . . 109

Capítulo 1 – Lógica, algoritmos e o computador

Programar é, antes de tudo, organizar o pensamento. Antes de escrever uma única linha em uma linguagem de programação, é preciso entender o problema, decidir quais passos o resolvem e em que ordem executá-los. Neste capítulo você vai conhecer as ideias que sustentam todo o resto da disciplina: o que é lógica, o que é um algoritmo, como um computador executa instruções e duas formas de escrever algoritmos antes de programar, o **fluxograma** e o **Portugol**. No final, vai ver o mesmo algoritmo escrito em C++, a linguagem que usaremos ao longo da apostila.

1.1 Lógica

Lógica é o estudo das formas corretas de raciocinar: como chegar a conclusões verdadeiras a partir de informações verdadeiras. Você a usa o tempo todo sem perceber. Se o aplicativo diz que vai chover e você vai sair, então leva o guarda-chuva. Se o saldo da conta é menor que o preço, então a compra não passa.

O ponto de partida da lógica é a **proposição**: uma frase que pode ser classificada como verdadeira ou falsa, mas não as duas coisas ao mesmo tempo. "A nota é maior ou igual a 6" é uma proposição; "Estude para a prova!" não é, porque não faz sentido perguntar se é verdadeira.

Proposições podem ser combinadas com três conectivos, que vão aparecer em todos os programas que você escrever:

- **E**: "tem nota maior ou igual a 6 **e** frequência maior ou igual a 75%" só é verdadeira quando as duas partes são verdadeiras.
- **OU**: "paga meia entrada se é estudante **ou** tem mais de 60 anos" é verdadeira quando pelo menos uma das partes é verdadeira.
- **NÃO**: inverte o valor. Se "a porta está aberta" é verdadeira, "a porta **não** está aberta" é falsa.

A **tabela-verdade** mostra o resultado para todas as combinações possíveis. Nela, V significa verdadeiro e F, falso:

A	B	A E B	A OU B	NÃO A
V	V	V	V	F
V	F	F	V	F
F	V	F	V	V
F	F	F	F	V

Na programação, a lógica é indispensável porque o computador faz **exatamente** o que mandamos, nem mais nem menos. Ele não adivinha intenções nem corrige um raciocínio mal construído. Se a condição de aprovação for escrita com OU no lugar de E, um aluno com nota 9 e frequência de 30% será aprovado, e o computador não verá problema nenhum nisso.

1.2 Algoritmos

Um **algoritmo** é uma sequência finita de passos bem definidos que resolve um problema. A palavra vem do nome do matemático persa Muhammad ibn Musa al-

Khwarizmi, que viveu em Bagdá no século IX e escreveu obras que levaram à Europa o sistema de numeração com algarismos indo-arábicos e os métodos de cálculo com ele. O título de um de seus livros também deu origem à palavra "álgebra".

Algoritmos existem muito antes dos computadores. Uma receita de bolo, as instruções para montar um móvel e o caminho que um aplicativo de mapas indica são algoritmos. Veja um, escrito em português comum:

```
algoritmo "Trocar uma lâmpada queimada"
   1. Desligar o interruptor
   2. Pegar uma escada e posicioná-la embaixo da lâmpada
   3. Subir na escada
   4. Retirar a lâmpada queimada
   5. Colocar a lâmpada nova
   6. Descer da escada
   7. Ligar o interruptor
   8. Se a lâmpada não acender, voltar ao passo 1 e verificar
      se a lâmpada nova está bem encaixada
```

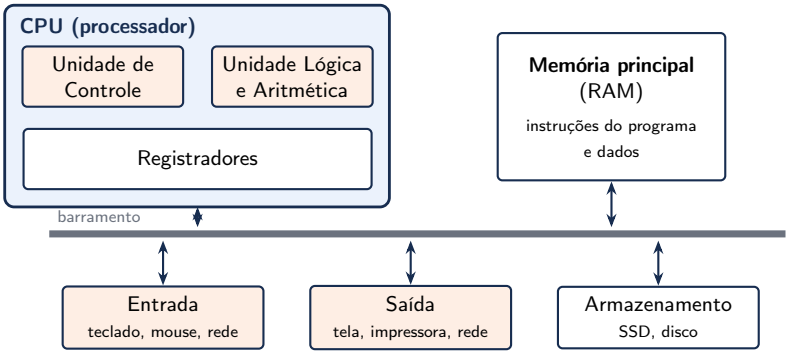
Para que uma sequência de passos seja de fato um algoritmo, ela precisa ter cinco características:

Característica	Significado
finitude	termina depois de um número finito de passos
definição	cada passo é preciso e não deixa dúvida sobre o que fazer
entrada	recebe zero ou mais dados para trabalhar
saída	produz pelo menos um resultado
efetividade	cada passo é simples o bastante para ser executado de fato

A **definição** é a mais difícil de garantir. "Coloque sal a gosto" funciona em uma receita, porque quem cozinha decide, mas um computador não tem gosto. "Aguarde a água ferver" é mais preciso, e "aguarde 5 minutos" é ainda mais. Escrever algoritmos é, em boa parte, aprender a eliminar esse tipo de ambiguidade.

1.3 Como o computador executa um algoritmo

Antes de programar, vale entender o que está sendo programado. Praticamente todos os computadores atuais, do celular ao servidor, seguem a **arquitetura de Von Neumann**, descrita pelo matemático John von Neumann em 1945, no relatório sobre o projeto do computador EDVAC. A grande ideia foi guardar o **programa na mesma memória que os dados**: para trocar o que o computador faz, basta carregar outro programa, sem mexer nos circuitos.



o programa e os dados ficam juntos na memória; a CPU busca uma instrução por vez, decodifica e executa, e repete esse ciclo bilhões de vezes por segundo

- A **CPU** (unidade central de processamento, ou processador) executa as instruções. A **unidade de controle** busca cada instrução e coordena a execução; a **unidade lógica e aritmética** (ULA) faz as contas e as comparações; os **registradores** são pequenas memórias dentro da CPU, extremamente rápidas, que guardam os valores em uso naquele instante.
- A **memória principal** (RAM) guarda o programa em execução e seus dados. É rápida, mas perde tudo quando o computador é desligado.
- O **armazenamento** (SSD ou disco) guarda arquivos e programas de forma permanente, mas é bem mais lento que a RAM. Para executar um programa, o sistema o copia do armazenamento para a memória.
- Os dispositivos de **entrada** e de **saída** trocam informações com o mundo: teclado, tela, rede.
- O **barramento** é o conjunto de ligações por onde os dados circulam entre essas partes.

A CPU trabalha em um ciclo que se repete sem parar: **busca** a próxima instrução na memória, **decodifica** o que ela pede e a **executa**. Um processador atual repete esse ciclo bilhões de vezes por segundo. Cada instrução, porém, é muito simples, como somar dois números, comparar dois valores ou copiar um dado de um lugar para outro. Tudo o que um programa faz, de uma planilha a um jogo, é uma sequência enorme dessas operações simples.

Dentro do computador, tudo é representado em **binário**, com apenas dois símbolos, 0 e 1. Cada um desses dígitos é um **bit**, e um grupo de 8 bits forma um **byte**, a unidade usada para medir memória e arquivos. Um byte pode representar $2^8 = 256$ valores diferentes, o suficiente para um número de 0 a 255 ou para uma letra.

Essa visão explica a sequência que todo programa segue, e que vai aparecer em todos os algoritmos desta apostila:

entrada → processamento → saída

Os dados chegam pela entrada, são guardados na memória, transformados pela CPU e o resultado é enviado para a saída.

1.4 Do problema ao algoritmo

O erro mais comum de quem começa a programar é sair escrevendo código antes de entender o problema. Um roteiro simples evita isso:

1. **Entender o problema**: o que precisa ser feito? Qual resultado se espera? Se possível, resolva um exemplo à mão.
2. **Identificar as entradas**: quais dados o algoritmo precisa receber?
3. **Identificar as saídas**: quais resultados ele deve produzir?
4. **Definir o processamento**: quais cálculos e decisões transformam as entradas nas saídas?
5. **Escrever o algoritmo**, em fluxograma ou em Portugol.
6. **Testar à mão**, com alguns valores, antes de passar para o computador.

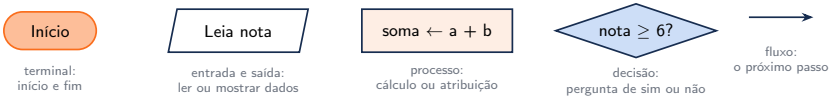
Veja o roteiro aplicado a um problema do dia a dia: um grupo de amigos quer dividir a conta de um restaurante, que cobra 10% de taxa de serviço.

Etapa	Resposta
entender	somar 10% ao valor da conta e dividir o total entre as pessoas; com uma conta de R$ 200 e 4 pessoas, cada uma paga R$ 55
entradas	o valor da conta e o número de pessoas
saída	o valor que cada pessoa paga
processamento	total ← conta × 1,10; porPessoa ← total / pessoas

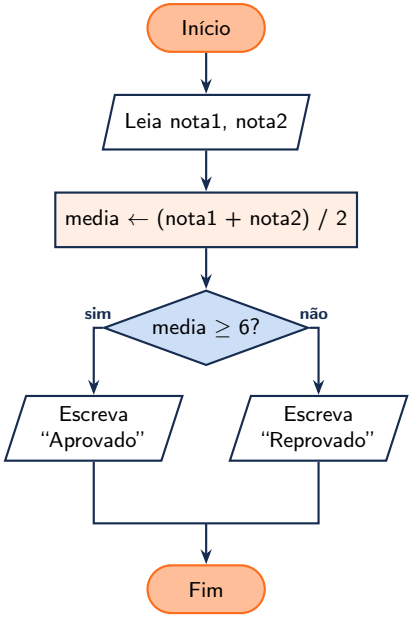
Repare que o exemplo resolvido à mão, na primeira linha, já serve de teste: quando o algoritmo estiver pronto, ele precisa dar R$ 55 para essa entrada.

1.5 Fluxogramas

O **fluxograma** representa um algoritmo com desenhos: cada tipo de passo tem uma forma geométrica, e setas mostram a ordem. Ele é útil para enxergar o caminho do algoritmo, principalmente quando há decisões, antes de se preocupar com a escrita.



O fluxograma abaixo lê duas notas, calcula a média e informa se o aluno foi aprovado. A seta ← indica **atribuição**: "media ← (nota1 + nota2) / 2" significa "calcule (nota1 + nota2) / 2 e guarde o resultado em media". O losango tem duas saídas, e o caminho seguido depende da resposta à pergunta.



Alguns cuidados deixam o fluxograma fácil de ler: ele tem um único início e um único fim; o fluxo corre de cima para baixo; toda decisão tem as duas saídas identificadas; e as setas não se cruzam. Para algoritmos grandes, porém, o desenho fica difícil de manter, e é aí que o pseudocódigo se torna mais prático.

1.6 Portugol

O **Portugol** é um **pseudocódigo** em português: uma forma estruturada de escrever algoritmos, com poucas palavras e regras fixas, mas sem os detalhes de uma linguagem de programação real. Ele permite pensar na lógica sem se preocupar com ponto e vírgula, bibliotecas ou mensagens do compilador.

Não existe um Portugol único: cada livro e cada ferramenta usa uma variação. Nesta apostila usamos a do **VisuAlg**, um programa gratuito, muito usado no ensino no Brasil, que executa algoritmos em Portugol. Assim, todo algoritmo das próximas páginas pode ser digitado e testado nele. Outra ferramenta conhecida é o Portugol Studio, que usa uma sintaxe um pouco diferente, com chaves.

Este é o algoritmo do fluxograma anterior em Portugol:

```
algoritmo "Media"
var
   nota1, nota2, media: real
inicio
   escreva("Digite a nota 1: ")
   leia(nota1)
```

```
   escreva("Digite a nota 2: ")
   leia(nota2)
   media <- (nota1 + nota2) / 2
   se media >= 6 entao
      escreval("Aprovado! Média: ", media)
   senao
      escreval("Reprovado. Média: ", media)
   fimse
fimalgoritmo
```

- `algoritmo "Media"` dá um nome ao algoritmo.
- Em `var` são declaradas as **variáveis**, os espaços da memória onde os dados ficam guardados, cada uma com seu **tipo**: `inteiro`, `real` (com casas decimais), `caractere` (texto) ou `logico` (verdadeiro ou falso). O Capítulo 3 trata disso em detalhe.
- Entre `inicio` e `fimalgoritmo` ficam os comandos, executados de cima para baixo.
- `leia` recebe um valor digitado; `escreva` mostra um texto ou valor na tela, e `escreval` faz o mesmo e pula para a linha seguinte.
- `<-` é a atribuição, a mesma seta do fluxograma.
- `se ... entao ... senao ... fimse` escolhe entre dois caminhos, como o losango.

O recuo das linhas internas não é obrigatório, mas mostra visualmente o que está dentro de quê. Acostume-se a usá-lo desde já: em C++ ele vai ser igualmente importante para a leitura.

O algoritmo da conta do restaurante, da Seção 1.4, fica assim:

```
algoritmo "DividirConta"
var
   conta, total, porPessoa: real
   pessoas: inteiro
inicio
   escreva("Valor da conta: ")
   leia(conta)
   escreva("Número de pessoas: ")
   leia(pessoas)
   total <- conta * 1.10            // 10% de serviço
   porPessoa <- total / pessoas
   escreval("Cada pessoa paga R$ ", porPessoa)
fimalgoritmo
```

O que vem depois de `//` é um **comentário**: uma anotação para quem lê, que não é executada. Note também que, no Portugol e nas linguagens de programação, o separador decimal é o **ponto**, e não a vírgula: `1.10`, e não `1,10`.

Teste de mesa

O **teste de mesa** é a forma de testar um algoritmo sem computador: você executa os passos no papel, anotando em uma tabela o valor de cada variável a cada linha. Ele

revela erros de lógica antes mesmo de o programa existir. Para o algoritmo da média, vale testar um caso de cada resultado e o valor exatamente na fronteira, 6:

nota1	nota2	media	Saída
7.0	4.0	5.5	Reprovado
8.0	9.0	8.5	Aprovado
6.0	6.0	6.0	Aprovado (a condição é `>=`)

Se o algoritmo tivesse sido escrito com `>` no lugar de `>=`, a terceira linha mostraria o erro: um aluno com média exatamente 6 seria reprovado.

1.7 Do Portugol ao C++

O Portugol ajuda a pensar, mas o computador não o entende diretamente. Para que o algoritmo vire um programa, ele precisa ser escrito em uma **linguagem de programação**. Nesta disciplina usamos o **C++**, uma das linguagens mais usadas no mundo, presente em sistemas operacionais, jogos, navegadores e bancos de dados. Veja o algoritmo da média em C++, lado a lado com o que você já conhece:

```
#include <iostream>
using namespace std;

int main() {
    double nota1, nota2, media;       // var ... : real
    cout << "Digite a nota 1: ";      // escreva
    cin >> nota1;                     // leia
    cout << "Digite a nota 2: ";
    cin >> nota2;
    media = (nota1 + nota2) / 2;      // media <- ...
    if (media >= 6) {                 // se ... entao
        cout << "Aprovado! Média: " << media << endl;
    } else {                          // senao
        cout << "Reprovado. Média: " << media << endl;
    }                                 // fimse
    return 0;
}
```

A estrutura é a mesma; mudam as palavras e alguns sinais. As principais correspondências:

Portugol	C++
`inicio ... fimalgoritmo`	`int main() { ... }`
`real`, `inteiro`	`double`, `int`
`leia(x)`	`cin >> x;`
`escreva(x)`	`cout << x;`
`x <- 5`	`x = 5;`
`se ... entao ... fimse`	`if (...) { ... }`

Não se preocupe ainda com o `#include`, o `using namespace std` e o ponto e vírgula no fim das linhas: o Capítulo 2 explica cada parte de um programa em C++, como ele é

transformado em algo que o computador executa e como montar o seu ambiente para programar.

Erros Comuns

- **Começar pelo código.** Sem entender o problema e identificar entradas e saídas, o programa resolve o problema errado. Siga o roteiro da Seção 1.4.
- **Escrever passos ambíguos.** "Calcule o desconto" não diz qual desconto nem sobre qual valor. Cada passo precisa ser executável sem que ninguém precise adivinhar nada.
- **Criar um algoritmo que nunca termina.** "Repita até a água ferver" sem um fogão ligado não acaba. Toda repetição precisa de uma condição que um dia se torne falsa.
- **Confundir E com OU.** "Nota maior ou igual a 6 **ou** frequência maior ou igual a 75%" aprova quem só cumpriu uma das exigências.
- **Esquecer um dos caminhos da decisão.** Todo losango tem duas saídas. Pergunte sempre: e se a resposta for não?
- **Confundir atribuição com igualdade.** `media <- 6` guarda 6 em `media`; não é uma pergunta. A comparação "media é igual a 6?" é escrita de outra forma, como você verá no Capítulo 3.
- **Usar vírgula como separador decimal.** No Portugol e no C++, escreve-se `1.10`.
- **Testar só o caso fácil.** Inclua no teste de mesa os valores de fronteira, como a média exatamente 6.

Capítulo 2 – Linguagens de programação

No Capítulo 1, o algoritmo da média foi escrito em Portugol e depois em C++. O Portugol serve para pensar; o C++ é uma **linguagem de programação**, que um computador consegue de fato executar. Neste capítulo você vai entender o que é uma linguagem de programação, a diferença entre as regras de escrita de uma linguagem (sintaxe) e o significado do que se escreve (semântica), como um programa em C++ vira algo que o processador executa e como montar o ambiente para programar. No final, vai escrever programas completos com entrada e saída de dados e aprender a ler as mensagens de erro do compilador.

2.1 O que é uma linguagem de programação

O processador só entende **linguagem de máquina**: sequências de bits que representam instruções muito simples, como "some o conteúdo destes dois registradores". Escrever programas assim é possível, mas extremamente trabalhoso e cheio de erros. Por isso, ao longo da história foram criadas linguagens cada vez mais próximas da forma como as pessoas pensam:

Nível	Como é	Exemplo
linguagem de máquina	bits que o processador executa diretamente	`10110000 01100001`
montagem (*assembly*)	cada instrução de máquina ganha um nome curto	`mov al, 97`
alto nível	comandos próximos da linguagem humana e da matemática	`x = a + 97;`

Uma **linguagem de programação de alto nível** é um conjunto de palavras, símbolos e regras para escrever instruções de forma precisa, que depois são traduzidas para linguagem de máquina. Algumas das mais importantes:

- **FORTRAN** (1957): uma das primeiras linguagens de alto nível, criada para cálculos científicos.
- **C** (1972): criada por Dennis Ritchie para escrever o sistema operacional Unix. Combina comandos de alto nível com acesso direto à memória, e até hoje é a base de sistemas operacionais e de equipamentos eletrônicos.
- **C++** (1985): criada por Bjarne Stroustrup como uma extensão do C, acrescentando, entre outras coisas, a orientação a objetos (Capítulo 12). É usada em jogos, navegadores, bancos de dados e em qualquer sistema em que o desempenho importa.
- **Python** (1991) e **Java** (1995): linguagens de alto nível muito populares; o Java é estudado no Capítulo 12.

Todas as linguagens têm os mesmos ingredientes básicos: variáveis, operadores, decisões, repetições e funções, os assuntos dos próximos capítulos. Quem aprende bem a lógica e uma linguagem aprende as outras com muito mais facilidade.

2.2 Sintaxe e semântica

Assim como o português, toda linguagem de programação tem dois níveis de regras:

- A **sintaxe** define **como** escrever: quais palavras existem, em que ordem podem aparecer, onde vão os parênteses e o ponto e vírgula. "Rato o gato comeu o" tem erro de sintaxe em português: as palavras existem, mas a ordem não forma uma frase.
- A **semântica** define **o que** cada construção significa. "A pedra almoçou o professor" tem a sintaxe perfeita, mas o significado não faz sentido.

Em um programa, erros de sintaxe são encontrados pelo compilador, que se recusa a continuar. Erros de semântica são mais traiçoeiros: o programa compila e roda, mas faz algo diferente do que o programador queria. Veja um exemplo com a sintaxe correta e uma surpresa semântica:

```
#include <iostream>
using namespace std;

int main() {
    int a = 7, b = 2;
    cout << a / b << endl;      // 3, e não 3.5
    cout << a / 2.0 << endl;    // 3.5
    return 0;
}
```

Em C++, a divisão entre dois números inteiros descarta a parte decimal: `7 / 2` vale 3. Basta um dos números ter casas decimais, como `2.0`, para o resultado ser 3.5. Nada disso é um erro para o compilador: é o significado definido pela linguagem. Conhecer a semântica é o que evita esse tipo de surpresa, e o Capítulo 3 volta a esse assunto.

As peças de um programa

Para o compilador, o texto de um programa é formado por pequenas peças, chamadas **elementos léxicos** (ou *tokens*). Reconhecer cada uma ajuda a ler qualquer código:

Peça	O que é	Exemplos
palavras reservadas	têm significado fixo na linguagem	`int`, `if`, `return`
identificadores	nomes escolhidos pelo programador	`media`, `nota1`
literais	valores escritos diretamente	`7`, `2.0`, `"Olá"`
operadores	indicam operações	`+`, `=`, `>=`
pontuação	organiza a estrutura	`;`, `{ }`, `( )`

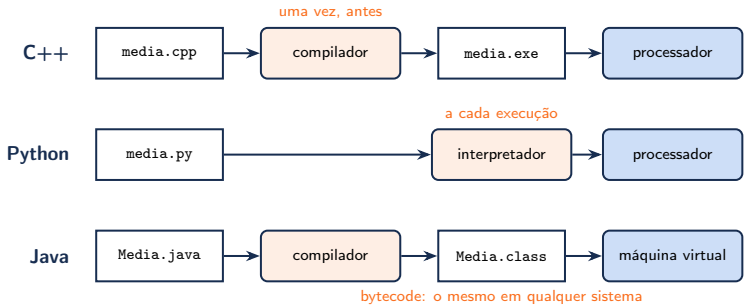
Um identificador pode ter letras, dígitos e o sublinhado (`_`), mas não pode começar com dígito nem ser uma palavra reservada. O C++ diferencia maiúsculas de minúsculas: `media`, `Media` e `MEDIA` são três nomes diferentes. Evite acentos nos identificadores e prefira nomes que digam o que a variável guarda: `precoTotal` é muito melhor que `x`.

2.3 Compilação e interpretação

Um programa escrito em uma linguagem de alto nível é chamado de **código-fonte**. Para ser executado, ele precisa ser traduzido para linguagem de máquina, e existem duas estratégias principais:

- Um **compilador** traduz o programa inteiro **uma vez**, antes de executá-lo, e gera um arquivo executável. Depois disso, o programa roda sem precisar do compilador. É o caso do C e do C++.
- Um **interpretador** lê o código-fonte e o executa aos poucos, **a cada execução**. É o caso do Python e do JavaScript.

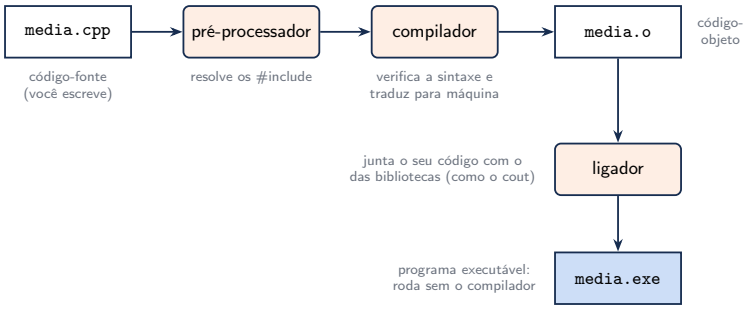
O Java usa um caminho intermediário: o compilador gera um código para uma máquina imaginária, o *bytecode*, que é executado por uma **máquina virtual** instalada no computador. Assim, o mesmo programa compilado roda em qualquer sistema que tenha a máquina virtual.



Programas compilados costumam ser mais rápidos, porque toda a tradução foi feita antes, e o compilador encontra muitos erros sem que o programa precise rodar. Programas interpretados são mais práticos para testar ideias rapidamente, mas um erro só aparece quando a linha com problema é executada.

As etapas da compilação em C++

O que chamamos de "compilar" um programa em C++ são, na verdade, três etapas, executadas em sequência por um único comando:



1. O **pré-processador** trata as linhas que começam com `#`, como o `#include`, que copia para o programa as declarações de uma biblioteca.

2. O **compilador** verifica a sintaxe e traduz o código para linguagem de máquina, gerando o **código-objeto**.
3. O **ligador** (*linker*) junta o código-objeto com o código das bibliotecas usadas, como a que implementa o `cout`, e gera o **executável**.

2.4 O ambiente de programação

Para programar em C++, você precisa de duas coisas: um **editor**, onde escreve o código, e um **compilador**. O compilador mais usado é o **g++**, do projeto GCC, gratuito e disponível para Windows, Linux e macOS. No Windows, ele é instalado por pacotes como o MinGW-w64.

Uma **IDE** (ambiente de desenvolvimento integrado) reúne editor, compilador e depurador em um só programa, e permite compilar e executar com um clique. Algumas opções gratuitas: VS Code (com a extensão de C++), Code::Blocks e Dev-C++. Também existem compiladores on-line, que funcionam no navegador sem instalar nada, úteis para os primeiros exercícios.

Mesmo usando uma IDE, vale saber compilar pelo terminal, porque é isso que ela faz por baixo:

```
$ g++ -Wall -Wextra media.cpp -o media
$ ./media
Digite a nota 1: 7
Digite a nota 2: 8
Aprovado! Média: 7.5
```

O `$` representa o terminal, e não deve ser digitado. A primeira linha compila `media.cpp` e gera o executável `media` (no Windows, `media.exe`). As opções `-Wall -Wextra` ligam os **avisos** do compilador, que apontam trechos suspeitos mesmo quando não são erros de sintaxe: use-as sempre. A segunda linha executa o programa.

2.5 A estrutura de um programa em C++

Este é o programa mais simples que faz alguma coisa visível, e cada linha dele vai aparecer em todos os programas da apostila:

```
// Meu primeiro programa em C++
#include <iostream>      // biblioteca de entrada e saída
using namespace std;     // cout em vez de std::cout

int main() {             // o programa começa aqui
    cout << "Olá, mundo!" << endl;
    return 0;            // 0 indica que terminou sem erro
}
```

- `//` inicia um **comentário**, que vai até o fim da linha e é ignorado pelo compilador. Comentários de várias linhas ficam entre `/*` e `*/`.
- `#include <iostream>` inclui a biblioteca de entrada e saída, que traz o `cout` e o `cin`.

- `using namespace std;` evita ter de escrever `std::` antes de `cout`, `cin` e `endl`, que pertencem ao "espaço de nomes" padrão, o `std`.
- `int main()` é a **função principal**: todo programa em C++ começa a ser executado por ela. O que está entre as chaves `{ }` é o **corpo** da função.
- Cada **instrução** termina com ponto e vírgula. Ele é para o C++ o que o ponto final é para uma frase.
- `return 0;` encerra o `main` e informa ao sistema que o programa terminou normalmente.

Espaços, linhas em branco e recuos não mudam o significado do programa, mas fazem enorme diferença para quem lê. O padrão desta apostila é recuar quatro espaços tudo o que está dentro de um par de chaves.

2.6 Saída de dados com cout

O `cout` (*console output*) mostra dados na tela. O operador `<<` envia cada valor para ele, e vários valores podem ser encadeados em uma mesma instrução. O `endl` termina a linha; o caractere especial `\n` faz o mesmo dentro de um texto.

```
#include <iostream>
#include <iomanip>     // fixed e setprecision
using namespace std;

int main() {
    cout << "Linha 1" << endl;
    cout << "Linha 2\n";                  // \n também quebra
    cout << "Nome:\tAna\n";               // \t: tabulação
    cout << "Ela disse \"oi\"" << endl;   // \" : aspas no texto
    cout << "C:\\Arquivos" << endl;       // \\ : a própria barra
    cout << 7 << " + " << 3 << " = " << 7 + 3 << endl;
    // 7 + 3 = 10

    double preco = 55;
    cout << preco << endl;                          // 55
    cout << fixed << setprecision(2) << preco << endl;   // 55.00
    cout << 10.0 / 3 << endl;                       // 3.33
    return 0;
}
```

As combinações que começam com a barra invertida são as **sequências de escape**: elas representam caracteres que não daria para escrever diretamente no meio do texto. Para valores com casas decimais, `fixed` junto com `setprecision(2)`, da biblioteca `<iomanip>`, mostra sempre duas casas, o formato certo para dinheiro. Essa configuração continua valendo para todos os números mostrados depois dela, como o `3.33` da última linha.

No Portugol do VisuAlg, a mesma formatação é feita com dois números depois do valor: `escreval(preco:6:2)` mostra o preço com 6 posições no total e 2 casas decimais.

2.7 Entrada de dados com cin

O `cin` (*console input*) lê dados digitados no teclado. O operador `>>` pega o próximo valor e o guarda na variável, convertendo-o para o tipo dela. Vários valores podem ser lidos de uma vez, separados por espaços ou por Enter.

O `>>` para de ler no primeiro espaço, então não serve para textos com espaços, como um nome completo. Para esses, use `getline`, que lê a linha inteira. Antes dele, `cin >> ws` descarta o Enter que o `>>` anterior deixou para trás; sem isso, o `getline` leria uma linha vazia.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int idade;
    double altura;
    cout << "Idade e altura: ";
    cin >> idade >> altura;        // ex.: 19 1.72
    string nome;
    cout << "Nome completo: ";
    cin >> ws;                     // descarta o Enter que sobrou
    getline(cin, nome);            // ex.: Ana Lima
    cout << nome << ", " << idade << " anos, " << altura << " m"
         << endl;                  // Ana Lima, 19 anos, 1.72 m
    return 0;
}
```

A variável `nome` é do tipo `string`, que guarda textos e precisa da biblioteca `<string>`. Os tipos de dados são o assunto do Capítulo 3, e os textos, do Capítulo 8.

2.8 Um programa completo

Com entrada, processamento e saída, já é possível transformar em programa o algoritmo da conta do restaurante, do Capítulo 1. Compare com o Portugol:

```
algoritmo "DividirConta"
var
   conta, total, porPessoa: real
   pessoas: inteiro
inicio
   escreva("Valor da conta: ")
   leia(conta)
   escreva("Número de pessoas: ")
   leia(pessoas)
   total <- conta * 1.10
   porPessoa <- total / pessoas
   escreval("Total com serviço: R$ ", total:8:2)
   escreval("Cada pessoa paga: R$ ", porPessoa:8:2)
fimalgoritmo
```

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double conta, total, porPessoa;
    int pessoas;
    cout << "Valor da conta: ";
    cin >> conta;
    cout << "Número de pessoas: ";
    cin >> pessoas;
    total = conta * 1.10;               // 10% de serviço
    porPessoa = total / pessoas;
    cout << fixed << setprecision(2);
    cout << "Total com serviço: R$ " << total << endl;
    cout << "Cada pessoa paga: R$ " << porPessoa << endl;
    return 0;
}
// Exemplo de execução:
// Valor da conta: 200
// Número de pessoas: 4
// Total com serviço: R$ 220.00
// Cada pessoa paga: R$ 55.00
```

O resultado confere com o exemplo resolvido à mão no Capítulo 1: R$ 55 por pessoa. Repare que `total / pessoas` divide um `double` por um `int`; como um dos lados tem casas decimais, o resultado também tem, e não há a surpresa da divisão inteira da Seção 2.2.

2.9 Lendo as mensagens de erro

Errar faz parte de programar, e o compilador é um aliado: ele aponta onde está o problema. Considere este programa, em que falta o ponto e vírgula no fim do `cout`:

```
// erro de compilação: falta o ; no fim do cout
#include <iostream>
using namespace std;

int main() {
    cout << "Ola" << endl
    return 0;
}
```

O g++ responde:

```
erro.cpp: In function 'int main()':
erro.cpp:6:26: error: expected ';' before 'return'
    6 |     cout << "Ola" << endl
      |                          ^
      |                          ;
```

A mensagem diz o arquivo (`erro.cpp`), a linha (6) e a coluna (26) do problema, o que ele esperava ("expected `';'`", um ponto e vírgula) e ainda desenha onde ele deveria estar. Três dicas valem para qualquer mensagem:

- **Comece pelo primeiro erro.** Um único erro pode causar uma cascata de mensagens seguintes; corrigido o primeiro, muitas delas desaparecem.
- **Olhe também a linha anterior.** O compilador percebe a falta do ponto e vírgula só quando encontra o `return`, na linha seguinte.
- **Leia a sugestão.** Ao usar uma variável `idad` que não existe, o g++ responde `'idad' was not declared in this scope; did you mean 'idade'?`: "`idad` não foi declarada; você quis dizer `idade`?".

Além dos **erros**, que impedem a compilação, o compilador mostra **avisos** (*warnings*), que não impedem, mas indicam algo suspeito, como uma variável declarada e nunca usada. Trate todo aviso como um erro a corrigir.

Os erros que o compilador encontra são os de **compilação**. Há ainda os erros de **execução**, quando o programa compila mas para no meio, por exemplo ao dividir um inteiro por zero, e os erros de **lógica**, quando ele roda até o fim mas dá a resposta errada. Esses dois últimos só aparecem testando, e é por isso que o teste de mesa do Capítulo 1 continua valendo depois que o programa existe.

Erros Comuns

- **Esquecer o ponto e vírgula.** Toda instrução termina com `;`. Repare que a mensagem de erro costuma apontar para a linha seguinte.
- **Esquecer o `#include`.** Sem `<iostream>`, o `cout` não existe; sem `<string>` ou `<iomanip>`, `string` e `setprecision` também não.
- **Trocar maiúsculas por minúsculas.** `Cout`, `Main` e `Int` não existem. O C++ diferencia as duas.
- **Inverter os operadores do cin e do cout.** Saída usa `<<` (os dados vão para o `cout`); entrada usa `>>` (os dados vêm do `cin` para a variável).
- **Ler um texto com espaços usando `>>`.** Só a primeira palavra é lida. Use `getline`, precedido de `cin >> ws` se houver uma leitura com `>>` antes.
- **Dividir inteiros esperando casas decimais.** `7 / 2` vale 3. Use pelo menos um valor com ponto decimal, como `7 / 2.0`.
- **Usar vírgula decimal.** Em C++, assim como no Portugol, `1,10` não é um número: escreva `1.10`.
- **Ignorar os avisos.** Compile sempre com `-Wall -Wextra` e leia o que o compilador diz.

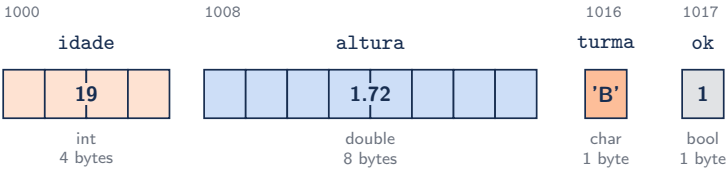
Capítulo 3 – Variáveis, tipos e operadores

Todo programa trabalha com dados: notas, preços, nomes, datas, respostas de sim ou não. Neste capítulo você vai aprender como esses dados ficam guardados na memória, em **variáveis**; por que cada variável tem um **tipo**, que define o que ela pode guardar e quanto espaço ocupa; e como combiná-los em cálculos e comparações com os **operadores**. Esses três conceitos são a matéria-prima de todos os capítulos seguintes, e boa parte dos erros de quem começa a programar vem de detalhes que aparecem aqui, como a divisão entre inteiros e a diferença entre `=` e `==`.

3.1 Variáveis

Uma **variável** é um espaço da memória, identificado por um nome, que guarda um valor. Ela tem quatro características:

- um **nome**, escolhido pelo programador, usado para se referir a ela no código;
- um **tipo**, que diz que espécie de valor ela guarda e quantos bytes ocupa;
- um **valor**, que pode mudar durante a execução (daí o nome "variável");
- um **endereço**, a posição da memória onde ela está, escolhida pelo compilador.



cada quadrado é um byte; o número acima de cada variável é o endereço do seu primeiro byte
(os endereços são ilustrativos: quem os escolhe é o compilador)

Em C++, toda variável precisa ser **declarada** antes de ser usada, indicando o tipo e o nome. O ideal é já dar um valor inicial na declaração, o que se chama **inicializar**. Uma variável declarada sem valor contém o que estiver naquele pedaço da memória, um valor imprevisível conhecido como "lixo". O g++ avisa quando isso acontece: `'x' is used uninitialized`.

Quando um valor não deve mudar nunca, como o número π ou a média mínima para aprovação, ele é declarado como **constante**, com a palavra `const`. Qualquer tentativa de alterá-lo vira um erro de compilação. Por convenção, nomes de constantes são escritos em maiúsculas.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int idade = 19;               // número inteiro
    double altura = 1.72;         // número real
    char turma = 'B';             // um caractere: aspas simples
    bool matriculado = true;      // verdadeiro ou falso
    string nome = "Ana Lima";     // texto: aspas duplas
```

```
    const double MEDIA_MINIMA = 6.0;   // constante

    idade = idade + 1;            // usa o antigo e o substitui
    cout << nome << ", " << idade << " anos, turma " << turma
         << endl;                  // Ana Lima, 20 anos, turma B
    cout << matriculado << endl;               // 1
    cout << boolalpha << matriculado << endl;   // true
    cout << "Média mínima: " << MEDIA_MINIMA << endl;   // 6
    return 0;
}
```

A linha `idade = idade + 1;` mostra como a atribuição funciona: primeiro o lado direito é calculado, usando o valor atual (19); depois o resultado (20) é guardado na variável da esquerda, substituindo o anterior. Para o `cout`, um `bool` verdadeiro aparece como 1; o `boolalpha` faz aparecer `true` e `false`.

3.2 Tipos de dados

O **tipo** de uma variável define três coisas: quais valores ela pode guardar, quanto espaço ocupa na memória e quais operações fazem sentido com ela. Os tipos básicos do C++ e seus equivalentes em Portugol são:

C++	Portugol	Guarda	Exemplos
`int`	`inteiro`	números inteiros	`19`, `-3`
`double`	`real`	números com casas decimais	`1.72`, `-0.5`
`char`	`caractere`	um único caractere	`'B'`, `'7'`
`bool`	`logico`	verdadeiro ou falso	`true`, `false`
`string`	`caractere`	um texto	`"Ana"`

Repare que o Portugol usa `caractere` tanto para uma letra quanto para um texto inteiro, enquanto o C++ separa os dois: `char` guarda exatamente um caractere, entre aspas simples, e `string` guarda um texto, entre aspas duplas. `'A'` e `"A"` são coisas diferentes em C++.

Cada tipo tem um tamanho, e o tamanho limita os valores possíveis. O tamanho exato depende do compilador e do sistema, mas nos computadores atuais costuma ser o que o programa abaixo mostra. A função `sizeof` informa quantos bytes um tipo ocupa, e a biblioteca `<climits>` traz os limites de cada tipo inteiro.

```
#include <iostream>
#include <climits>     // INT_MAX, LLONG_MAX
using namespace std;

int main() {
    cout << "int: " << sizeof(int) << " bytes, até "
         << INT_MAX << endl;       // 4 bytes, até 2147483647
    cout << "long long: " << sizeof(long long) << " bytes, até "
         << LLONG_MAX << endl;
    // 8 bytes, até 9223372036854775807
    cout << "double: " << sizeof(double) << endl;   // 8
```

```
    cout << "char: " << sizeof(char) << endl;       // 1
    cout << "bool: " << sizeof(bool) << endl;       // 1
    return 0;
}
```

Um `int` vai de cerca de −2,1 a 2,1 bilhões. É suficiente para idades, quantidades e anos, mas não para a população mundial ou para valores em centavos de uma grande empresa: para esses, use `long long`. Quando uma conta passa do limite, acontece um **estouro** (*overflow*), e o resultado fica errado sem nenhum aviso durante a execução.

O `double` guarda cerca de 15 dígitos significativos, o que é ótimo para medidas e cálculos científicos. Mas ele guarda os números em binário, e muitas frações que são exatas em decimal, como 0,1, não têm representação exata em binário. Por isso, `0.1 + 0.2` dá `0.30000000000000004`, e não exatamente `0.3`. Para a maioria das contas, essa diferença é irrelevante; ela só importa quando se compara valores reais com `==`, como você verá na Seção 3.5.

3.3 Operadores aritméticos

Os operadores aritméticos fazem as contas. Quase todos funcionam como na matemática, com duas atenções especiais: a divisão e o resto.

Operação	C++	Portugol	Exemplo em C++
soma	`+`	`+`	`17 + 5` vale 22
subtração	`-`	`-`	`17 - 5` vale 12
multiplicação	`*`	`*`	`17 * 5` vale 85
divisão	`/`	`/` ou `div`	`17 / 5` vale 3
resto	`%`	`mod` ou `%`	`17 % 5` vale 2
potência	`pow(a, b)`	`^`	`pow(2, 10)` vale 1024

Em C++, o operador `/` faz **divisão inteira** quando os dois lados são inteiros: `17 / 5` vale 3, e a parte decimal é descartada (não há arredondamento: `19 / 5` também vale 3). Se pelo menos um dos lados for real, a divisão é real: `17.0 / 5` vale 3.4. No Portugol do VisuAlg, `/` é sempre a divisão real, e a inteira se escreve `div`.

O operador `%` dá o **resto** da divisão inteira, e é muito mais útil do que parece: `n % 2` é 0 quando `n` é par; `n % 10` é o último dígito de `n`; `n % 7` diz em que dia da semana cai o dia `n` de uma contagem. Ele só funciona com inteiros.

Quando uma expressão tem vários operadores, a **precedência** decide a ordem: multiplicação, divisão e resto vêm antes de soma e subtração, e operadores do mesmo nível são calculados da esquerda para a direita. Parênteses mudam a ordem, e vale a pena usá-los sempre que deixarem a intenção mais clara.

```
#include <iostream>
using namespace std;

int main() {
    int a = 17, b = 5;
    cout << a + b << " " << a - b << " " << a * b << endl;
    // 22 12 85
```

```
    cout << a / b << endl;              // 3: divisão inteira
    cout << a % b << endl;              // 2: resto
    cout << 17.0 / 5 << endl;           // 3.4
    cout << static_cast<double>(a) / b << endl;   // 3.4
    cout << 2 + 3 * 4 << " " << (2 + 3) * 4 << endl;  // 14 20
    cout << 1234 % 10 << " " << 1234 / 10 << endl;    // 4 123

    int pontos = 10;
    pontos += 5;       // o mesmo que pontos = pontos + 5: 15
    pontos++;          // soma 1: 16
    pontos *= 2;       // o mesmo que pontos = pontos * 2: 32
    pontos--;          // subtrai 1: 31
    cout << pontos << endl;             // 31
    return 0;
}
```

O `static_cast<double>(a)` converte o valor de `a` para `double` só naquela conta, sem mudar a variável; com isso, a divisão passa a ser real. Os operadores `+=`, `-=`, `*=` e `/=` são atalhos para alterar uma variável usando o próprio valor dela, e `++` e `-` somam e subtraem 1. Eles aparecem o tempo todo nas repetições do Capítulo 5.

3.4 Conversões de tipo

Quando valores de tipos diferentes se encontram em uma conta ou atribuição, o C++ converte um deles. As conversões que não perdem informação, como de `int` para `double`, são feitas sem problema. As que perdem informação exigem cuidado:

```
#include <iostream>
#include <cmath>       // round
using namespace std;

int main() {
    double preco = 9.99;
    int reais = static_cast<int>(preco);   // 9: corta os decimais
    cout << reais << " " << round(preco) << endl;   // 9 10

    int quantidade = 7;
    double metade = quantidade / 2.0;      // 3.5
    cout << metade << endl;

    char letra = 'A';
    cout << static_cast<int>(letra) << endl;        // 65
    cout << static_cast<char>(letra + 2) << endl;   // C
    return 0;
}
```

Converter um `double` para `int` **corta** a parte decimal, sem arredondar: 9,99 vira 9. Para arredondar, use `round`, da biblioteca `<cmath>`. A conversão entre `char` e `int` mostra que, para o computador, cada caractere é um número: o código do `'A'` na tabela ASCII é 65, o do `'B'` é 66, e assim por diante. Por isso `'A' + 2` dá 67, o código do `'C'`.

3.5 Operadores relacionais

Os operadores relacionais comparam dois valores e dão como resultado um `bool`: `true` ou `false`. São eles que formam as condições das decisões e repetições dos próximos capítulos.

Pergunta	C++	Portugol	Com x = 7
igual a	`==`	`=`	`x == 7` é `true`
diferente de	`!=`	`<>`	`x != 7` é `false`
maior que	`>`	`>`	`x > 7` é `false`
menor que	`<`	`<`	`x < 10` é `true`
maior ou igual a	`>=`	`>=`	`x >= 7` é `true`
menor ou igual a	`<=`	`<=`	`x <= 6` é `false`

Atenção à igualdade: em C++, `=` é **atribuição** e `==` é **comparação**. No Portugol é o contrário, porque a atribuição é a seta `<-`, e `=` compara. Escrever `x = 7` onde se queria `x == 7` é um dos erros mais comuns em C++, e o Capítulo 4 mostra por que ele é tão traiçoeiro.

Também por causa da representação em binário, não compare valores `double` com `==`: `0.1 + 0.2 == 0.3` dá `false`. Em vez disso, verifique se a diferença entre eles é muito pequena, por exemplo menor que `0.000001`.

3.6 Operadores lógicos

Os operadores lógicos combinam condições, exatamente como os conectivos E, OU e NÃO do Capítulo 1, com a mesma tabela-verdade:

Conectivo	C++	Portugol	Verdadeiro quando
E	`&&`	`e`	as duas condições são verdadeiras
OU	`||`	`ou`	pelo menos uma é verdadeira
NÃO	`!`	`nao`	a condição é falsa

```
#include <iostream>
using namespace std;

int main() {
    double nota = 7.5;
    int frequencia = 80;              // em %
    int idade = 17;
    bool estudante = true;

    bool aprovado = nota >= 6 && frequencia >= 75;
    bool meiaEntrada = estudante || idade >= 60;
    bool menor = !(idade >= 18);
    bool notaValida = nota >= 0 && nota <= 10;

    cout << boolalpha;
    cout << aprovado << " " << meiaEntrada << endl;   // true true
```

```
    cout << menor << " " << notaValida << endl;       // true true
    cout << (0.1 + 0.2 == 0.3) << endl;               // false
    return 0;
}
```

Repare em `notaValida`: para dizer "a nota está entre 0 e 10", é preciso escrever as duas comparações ligadas por `&&`. A forma matemática `0 <= nota <= 10` compila, mas não faz o que parece: o C++ calcula `0 <= nota`, que dá `true` ou `false` (1 ou 0), e depois compara esse 1 ou 0 com 10, o que é sempre verdadeiro. O g++ avisa: `comparisons like 'X<=Y<=Z' do not have their mathematical meaning`.

Os operadores `&&` e `||` também fazem **avaliação em curto-circuito**: se a resposta já está decidida pelo lado esquerdo, o lado direito nem é calculado. Em `pessoas > 0 && total / pessoas > 50`, se `pessoas` for 0, a divisão por zero nunca acontece, porque `pessoas > 0` já é falso e o `&&` só pode dar falso.

3.7 Precedência geral

Com operadores de tipos diferentes na mesma expressão, a ordem de cálculo segue esta tabela, do primeiro para o último:

Ordem	Operadores	Tipo
1	`( )`	parênteses
2	`!`, `-` (negativo), `++`, `-`	unários
3	`*`, `/`, `%`	multiplicação, divisão e resto
4	`+`, `-`	soma e subtração
5	`<`, `<=`, `>`, `>=`	relacionais
6	`==`, `!=`	igualdade
7	`&&`	e lógico
8	`||`	ou lógico
9	`=`, `+=`, `-=`, ...	atribuição

A figura mostra a avaliação de uma expressão que mistura quase todos eles:

com nota = 7, faltas = 3 e bonus = true

```
nota + 2 * faltas >= 12 && !bonus
```

`nota + 6 >= 12 && !bonus`	1. multiplicação: 2 * 3
`13 >= 12 && !bonus`	2. soma: 7 + 6
`true && !bonus`	3. comparação: 13 >= 12
`true && false`	4. negação: !true
`false`	5. e lógico: true && false

Note que `&&` vem antes de `||`: `a || b && c` significa `a || (b && c)`. Ninguém precisa decorar a tabela inteira: na dúvida, use parênteses. Eles nunca atrapalham e deixam claro para quem lê o que se pretendia.

3.8 Funções matemáticas

A biblioteca `<cmath>` traz as funções matemáticas mais usadas:

Função	Calcula	Exemplo
`sqrt(x)`	raiz quadrada	`sqrt(16)` é 4
`pow(x, y)`	x elevado a y	`pow(2, 10)` é 1024
`abs(x)`	valor absoluto	`abs(-7)` é 7
`round(x)`	arredonda para o inteiro mais próximo	`round(2.5)` é 3
`floor(x)`	arredonda para baixo	`floor(2.7)` é 2
`ceil(x)`	arredonda para cima	`ceil(2.1)` é 3

O `ceil` resolve com facilidade problemas do tipo "quantos ônibus de 40 lugares são necessários para 130 alunos?": `ceil(130 / 40.0)` dá 4. Com a divisão inteira `130 / 40`, o resultado seria 3, e 10 alunos ficariam para trás.

3.9 Um programa completo

Um sistema de vídeos registra a duração em segundos, mas precisa mostrá-la em horas, minutos e segundos. A divisão inteira e o resto resolvem o problema: cada hora tem 3600 segundos, então `total / 3600` dá as horas completas; o que sobra, `total % 3600`, dividido por 60, dá os minutos; e `total % 60` dá os segundos restantes.

```
algoritmo "ConverterSegundos"
var
   total, horas, minutos, segundos: inteiro
inicio
   escreva("Duração em segundos: ")
   leia(total)
   horas <- total div 3600
   minutos <- (total mod 3600) div 60
   segundos <- total mod 60
   escreval(horas, "h ", minutos, "min ", segundos, "s")
fimalgoritmo
```

```
#include <iostream>
using namespace std;

int main() {
    int total;
    cout << "Duração em segundos: ";
    cin >> total;
    int horas = total / 3600;
    int minutos = (total % 3600) / 60;
    int segundos = total % 60;
    cout << horas << "h " << minutos << "min " << segundos << "s"
         << endl;
    return 0;
}
```

```
// Exemplo de execução:
// Duração em segundos: 3725
// 1h 2min 5s
```

Um teste de mesa confirma: $3725/3600 = 1$ hora; $3725 \bmod 3600 = 125$, e $125/60 = 2$ minutos; $3725 \bmod 60 = 5$ segundos. Conferindo: $3600 + 2 \times 60 + 5 = 3725$.

Erros Comuns

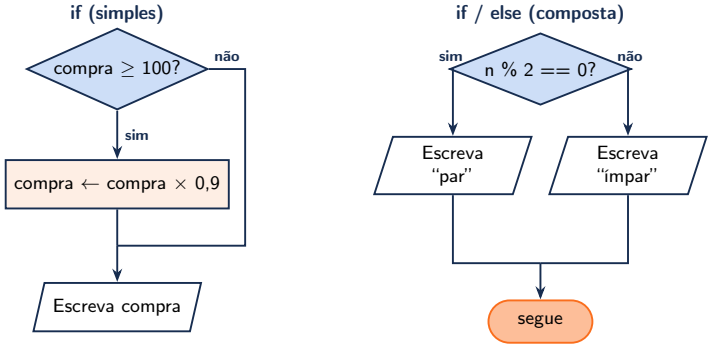
- **Usar uma variável sem inicializá-la.** Ela contém lixo, e o resultado muda a cada execução. Dê sempre um valor inicial.
- **Dividir inteiros esperando casas decimais.** `7 / 2` vale 3. Use `7.0 / 2` ou `static_cast<double>`.
- **Confundir `=` com `==`.** `=` atribui; `==` compara.
- **Escrever `0 <= x <= 10`.** Compila, mas é sempre verdadeiro. Use `x >= 0 && x <= 10`.
- **Comparar `double` com `==`.** `0.1 + 0.2 == 0.3` é falso. Compare a diferença com uma tolerância pequena.
- **Confundir `'A'` com `"A"`.** Aspas simples são um `char`; aspas duplas, um texto.
- **Esperar que a conversão para `int` arredonde.** Ela corta: 9,99 vira 9. Use `round`.
- **Estourar o `int`.** Contas que passam de 2,1 bilhões dão resultados errados. Use `long long` para números grandes.
- **Usar `%` com números reais.** O resto só existe para inteiros; com `double`, é erro de compilação.

Capítulo 4 – Estruturas de decisão

Até aqui, os programas executavam sempre os mesmos comandos, de cima para baixo, qualquer que fosse a entrada. Programas úteis precisam reagir aos dados: dar desconto só para compras acima de um valor, aprovar só quem atingiu a média, recusar uma senha errada. As **estruturas de decisão** (ou condicionais) escolhem quais comandos executar de acordo com uma condição. Neste capítulo você vai conhecer todas as formas de decisão do C++ e do Portugol, aprender a organizar decisões com várias alternativas e a evitar armadilhas que fazem o programa compilar, rodar e tomar a decisão errada.

4.1 Decisão simples: if

A **decisão simples** executa um bloco de comandos somente se uma condição for verdadeira; se for falsa, o bloco é pulado e o programa segue. A condição é qualquer expressão que resulte em verdadeiro ou falso, normalmente construída com os operadores relacionais e lógicos do Capítulo 3.



O fluxograma da esquerda corresponde a uma loja que dá 10% de desconto em compras a partir de R$ 100:

```
algoritmo "Desconto"
var
   compra: real
inicio
   escreva("Valor da compra: ")
   leia(compra)
   se compra >= 100 entao
      compra <- compra * 0.9
      escreval("Você ganhou 10% de desconto!")
   fimse
   escreval("Total: R$ ", compra:8:2)
fimalgoritmo
```

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double compra;
    cout << "Valor da compra: ";
    cin >> compra;
    if (compra >= 100) {
        compra = compra * 0.9;          // 10% de desconto
        cout << "Você ganhou 10% de desconto!" << endl;
    }
    cout << fixed << setprecision(2) << "Total: R$ " << compra
         << endl;
    return 0;
}
```

Em C++, a condição fica **entre parênteses**, e os comandos controlados pelo `if` ficam entre chaves, formando um **bloco**. Se o bloco tiver um único comando, as chaves podem ser omitidas, mas é uma boa prática usá-las sempre: elas evitam que um comando acrescentado depois fique fora do `if` sem que ninguém perceba (Seção 4.7). No Portugol, o bloco termina com `fimse`.

4.2 Decisão composta: if e else

A **decisão composta** escolhe entre dois caminhos: um para quando a condição é verdadeira, outro para quando é falsa. Exatamente um deles é executado, nunca os dois e nunca nenhum. É o fluxograma da direita na figura anterior.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Digite um número inteiro: ";
    cin >> n;
    if (n % 2 == 0) {
        cout << n << " é par" << endl;
    } else {
        cout << n << " é ímpar" << endl;
    }
    return 0;
}
```

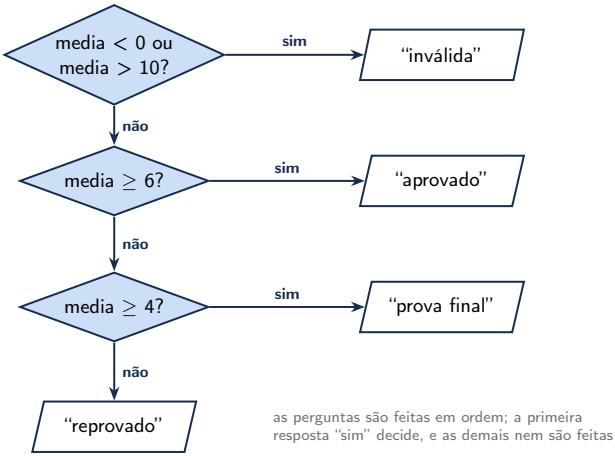
Em Portugol, a parte do "senão" é escrita com `senao`, antes do `fimse`:

```
algoritmo "ParOuImpar"
var
   n: inteiro
inicio
```

```
   escreva("Digite um número inteiro: ")
   leia(n)
   se n mod 2 = 0 entao
      escreval(n, " é par")
   senao
      escreval(n, " é ímpar")
   fimse
fimalgoritmo
```

4.3 Decisões encadeadas: else if

Muitos problemas têm mais de dois caminhos. A situação de um aluno, por exemplo, depende da média: inválida se estiver fora de 0 a 10, aprovado com 6 ou mais, prova final entre 4 e 6, reprovado abaixo de 4. Para isso, encadeiam-se várias decisões: cada `else` traz um novo `if`.




```
#include <iostream>
using namespace std;

int main() {
    double media;
    cout << "Média final: ";
    cin >> media;
    if (media < 0 || media > 10) {
        cout << "Média inválida" << endl;
    } else if (media >= 6) {
        cout << "Aprovado" << endl;
    } else if (media >= 4) {
        cout << "Prova final" << endl;
    } else {
        cout << "Reprovado" << endl;
```

```
    }
    return 0;
}
```

As condições são testadas **em ordem**, e a primeira verdadeira decide: as seguintes nem são avaliadas. Por isso, a ordem importa. Quando o programa chega em `media >= 4`, já se sabe que a média não é maior ou igual a 6, senão o caminho anterior teria sido seguido; não é preciso escrever `media >= 4 && media < 6`. Mas, se as duas condições fossem trocadas de lugar, uma média 9 cairia em "prova final", porque 9 também é maior ou igual a 4.

O Portugol do VisuAlg não tem uma palavra para "senão se": os `se` ficam um dentro do outro, cada um com o seu `fimse`.

```
algoritmo "Situacao"
var
   media: real
inicio
   escreva("Média final: ")
   leia(media)
   se (media < 0) ou (media > 10) entao
      escreval("Média inválida")
   senao
      se media >= 6 entao
         escreval("Aprovado")
      senao
         se media >= 4 entao
            escreval("Prova final")
         senao
            escreval("Reprovado")
         fimse
      fimse
   fimse
fimalgoritmo
```

Para testar uma decisão encadeada, use pelo menos um valor de cada faixa e os valores exatamente nas fronteiras: -1, 0, 3.9, 4, 5.9, 6, 10 e 11. A fronteira é onde mora o erro de usar `>` no lugar de `>=`.

4.4 Decisões aninhadas

Uma decisão pode estar **dentro** de outra. Isso é útil quando uma pergunta só faz sentido depois que outra foi respondida. Para classificar um triângulo pelos lados, por exemplo, primeiro é preciso saber se os três lados formam um triângulo: cada lado tem de ser menor que a soma dos outros dois. Só então faz sentido perguntar se ele é equilátero, isósceles ou escaleno.

```
#include <iostream>
using namespace std;
```

```
int main() {
    int a, b, c;
    cout << "Lados do triângulo: ";
    cin >> a >> b >> c;
    if (a < b + c && b < a + c && c < a + b) {
        if (a == b && b == c) {
            cout << "Equilátero" << endl;   // três iguais
        } else if (a == b || b == c || a == c) {
            cout << "Isósceles" << endl;    // dois iguais
        } else {
            cout << "Escaleno" << endl;     // todos diferentes
        }
    } else {
        cout << "Esses lados não formam um triângulo" << endl;
    }
    return 0;
}
// 3 3 3 -> Equilátero    5 5 8 -> Isósceles
// 3 4 5 -> Escaleno      1 2 10 -> não formam um triângulo
```

O recuo deixa visível o que está dentro de quê. Com muitos níveis de aninhamento, porém, o código fica difícil de ler. Muitas vezes é possível trocar decisões aninhadas por uma única condição composta: "se tem nota e se tem frequência" é o mesmo que "se tem nota **e** frequência".

```
// trecho
if (media >= 6) {                       // aninhado
    if (frequencia >= 75) {
        cout << "Aprovado" << endl;
    }
}
if (media >= 6 && frequencia >= 75) {  // o mesmo, em um só if
    cout << "Aprovado" << endl;
}
```

4.5 Escolha múltipla: switch

Quando a decisão compara **uma única variável** com vários valores fixos, como a opção de um menu, o `switch` é mais claro que uma sequência de `else if`. Ele avalia a variável uma vez e salta direto para o `case` correspondente.

```
#include <iostream>
using namespace std;

int main() {
    double a, b;
    char operacao;
    cout << "Conta (ex.: 8 * 3): ";
    cin >> a >> operacao >> b;
    switch (operacao) {
```

```
        case '+':
            cout << a + b << endl;
            break;
        case '-':
            cout << a - b << endl;
            break;
        case '*':
        case 'x':                   // dois casos, mesmo código
            cout << a * b << endl;
            break;
        case '/':
            if (b != 0) {
                cout << a / b << endl;
            } else {
                cout << "Divisão por zero" << endl;
            }
            break;
        default:                        // nenhum dos casos acima
            cout << "Operação desconhecida" << endl;
    }
    return 0;
}
// 8 * 3 -> 24        8 x 3 -> 24
// 7 / 2 -> 3.5       5 / 0 -> Divisão por zero
```

- Cada `case` indica um valor possível. Ele precisa ser uma constante, de um tipo inteiro ou `char`: o `switch` não funciona com `double`, `string` nem com faixas como "de 4 a 6".
- O `break` encerra o `switch`. Sem ele, a execução **continua no caso seguinte**, mesmo que o valor seja outro. Às vezes isso é útil, como em `'*'` e `'x'`, que compartilham o mesmo código; na maioria das vezes, esquecer o `break` é um erro.
- O `default` é executado quando nenhum caso corresponde, e funciona como o `else` final.

No Portugol do VisuAlg, a mesma estrutura se chama `escolha`, e não precisa de `break`, porque cada caso termina sozinho:

```
algoritmo "DiaDaSemana"
var
   dia: inteiro
inicio
   escreva("Dia (1 a 7): ")
   leia(dia)
   escolha dia
   caso 1, 7
      escreval("Fim de semana")
   caso 2, 3, 4, 5, 6
      escreval("Dia útil")
   outrocaso
```

```
      escreval("Dia inválido")
   fimescolha
fimalgoritmo
```

4.6 O operador ternário

Quando a decisão só escolhe entre **dois valores**, o **operador ternário `?:`** faz isso em uma linha: `condição ? valor_se_verdadeiro : valor_se_falso`. Ele não substitui o `if` em geral; serve para escolher um valor, e não para executar blocos de comandos.

O programa completo deste capítulo usa o ternário para verificar se um ano é **bissexto**. Pelo calendário gregoriano, um ano é bissexto se for divisível por 4, exceto os divisíveis por 100, que só são bissextos se também forem divisíveis por 400. Por isso 2024 e 2000 foram bissextos, mas 1900 não foi.

```
#include <iostream>
using namespace std;

int main() {
    int ano;
    cout << "Ano: ";
    cin >> ano;
    bool bissexto = (ano % 4 == 0 && ano % 100 != 0)
                    || ano % 400 == 0;
    int dias = bissexto ? 366 : 365;
    cout << ano << (bissexto ? " é" : " não é")
         << " bissexto e tem " << dias << " dias" << endl;
    return 0;
}
// 2024 é bissexto e tem 366 dias
// 1900 não é bissexto e tem 365 dias
```

Repare que a regra inteira cabe em uma única expressão lógica, guardada na variável `bissexto`. Dar um nome à condição deixa o resto do programa muito mais legível do que repetir a expressão em cada `if`. Um teste de mesa com os casos que exercitam cada parte da regra:

Ano	% 4	% 100	% 400	Bissexto?
2023	3	23	23	não (não divide por 4)
2024	0	24	24	sim
1900	0	0	300	não (divide por 100)
2000	0	0	0	sim (divide por 400)

4.7 Armadilhas das decisões

Alguns erros com decisões passam pela compilação e só aparecem no resultado. O programa abaixo tem dois deles:

```
// erro: dois enganos que o compilador aceita
int main() {
    int saldo = 50;
    if (saldo = 0) {                  // = no lugar de ==
        cout << "Sem saldo" << endl;
    }
    if (saldo > 100);                 // ; logo depois do if
    {
        cout << "Saldo alto" << endl;
    }
    cout << saldo << endl;
    return 0;
}
// Saída: Saldo alto
//        0
```

- Em `if (saldo = 0)`, o `=` **atribui** 0 ao saldo, em vez de compará-lo. O valor da atribuição, 0, é tratado como falso, então "Sem saldo" não aparece, mas o saldo foi zerado.
- Em `if (saldo > 100);`, o ponto e vírgula encerra o `if` com um comando vazio. O bloco entre chaves logo abaixo deixa de pertencer a ele e é executado sempre. Resultado: "Saldo alto" com saldo 0.

Os dois erros são avisados pelo g++ quando se compila com `-Wall -Wextra`: "`suggest parentheses around assignment used as truth value`" para o primeiro e "`suggest braces around empty body in an 'if' statement`" para o segundo. É mais um motivo para nunca ignorar os avisos.

Um terceiro cuidado é o `else` sem chaves. Sem elas, cada `if` controla apenas o comando seguinte, e um `else` sempre pertence ao `if` mais próximo que ainda não tem `else`, não importa o recuo. Com chaves em todos os blocos, essa dúvida simplesmente não existe.

Erros Comuns

- **Usar `=` na condição.** `if (x = 5)` atribui e é quase sempre verdadeiro. Para comparar, use `==`.
- **Colocar ponto e vírgula depois do `if`.** `if (cond);` encerra a decisão ali, e o bloco seguinte é executado sempre.
- **Omitir as chaves.** Um segundo comando acrescentado depois fica fora do `if`, mesmo com o recuo certo. Use chaves sempre.
- **Ordenar mal as condições encadeadas.** Testar `media >= 4` antes de `media >= 6` faz toda média alta cair no caso errado. Do mais restritivo para o menos restritivo.
- **Esquecer o `break` no `switch`.** A execução continua nos casos seguintes.
- **Usar `switch` com faixas ou textos.** Cada `case` é um valor exato, inteiro ou `char`. Para faixas, use `else if`.
- **Esquecer um caso.** Pergunte sempre o que acontece com os valores fora do esperado: média negativa, divisão por zero, opção inexistente do menu. Use `else` e `default` para tratá-los.

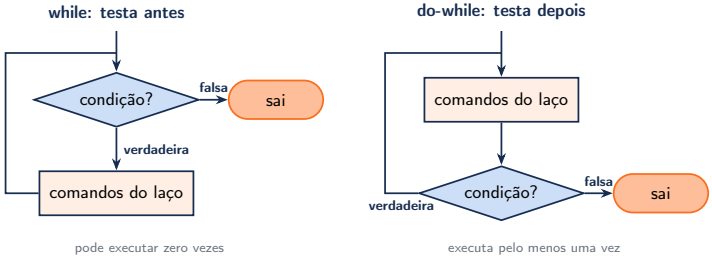
- **Testar só um caminho.** Cada ramo da decisão precisa de pelo menos um teste, e cada fronteira, de um valor exatamente nela.

Capítulo 5 – Estruturas de repetição

Imagine escrever um programa que mostra a tabuada do 7. Com o que vimos até aqui, seriam dez comandos `cout` quase iguais. E se fossem as notas de uma turma de 40 alunos? Ou de uma escola com 2 mil? Computadores são extraordinários justamente em repetir uma tarefa milhares ou bilhões de vezes, sem cansar e sem errar. As **estruturas de repetição** (ou **laços**, do inglês *loops*) executam um bloco de comandos várias vezes, enquanto uma condição for verdadeira. Neste capítulo você vai conhecer os três laços do C++ e do Portugol, os padrões de uso que aparecem em quase todo programa (contadores, acumuladores, maior e menor valor) e como evitar o erro mais típico das repetições: o laço que nunca termina.

5.1 Repetição com teste no início: while

O `while` ("enquanto") testa uma condição; se ela for verdadeira, executa o bloco e volta a testar. Quando a condição fica falsa, o laço termina e o programa segue. Cada execução do bloco é chamada de **iteração** (ou volta).



Como o teste vem antes, se a condição já começar falsa o bloco não é executado nenhuma vez. E algo dentro do bloco precisa, em algum momento, tornar a condição falsa; senão o laço nunca termina.

```
#include <iostream>
using namespace std;

int main() {
    int contador = 5;
    while (contador > 0) {
        cout << contador << " ";
        contador--;                 // aproxima o laço do fim
    }
    cout << "Fogo!" << endl;        // 5 4 3 2 1 Fogo!
    return 0;
}
```

Em Portugol, o mesmo laço é escrito com `enquanto ... faca ... fimenquanto`:

```
algoritmo "ContagemRegressiva"
var
```

```
   contador: inteiro
inicio
   contador <- 5
   enquanto contador > 0 faca
      escreva(contador, " ")
      contador <- contador - 1
   fimenquanto
   escreval("Fogo!")
fimalgoritmo
```

Laço controlado por sentinela

O `while` é o laço certo quando **não se sabe de antemão** quantas vezes repetir. Um caso muito comum é ler dados até que o usuário digite um valor especial, chamado de **sentinela**, que indica o fim. O valor da sentinela não pode ser um dado válido: para somar valores positivos, o 0 funciona bem.

```
algoritmo "SomarCompras"
var
   valor, soma, quantos: inteiro
inicio
   soma <- 0
   quantos <- 0
   escreva("Valores das compras (0 para terminar): ")
   leia(valor)                  // primeira leitura, antes do laço
   enquanto valor <> 0 faca
      soma <- soma + valor
      quantos <- quantos + 1
      leia(valor)               // próxima, no fim do bloco
   fimenquanto
   escreval(quantos, " compras, total ", soma)
fimalgoritmo
```

Repare no padrão: uma leitura antes do laço e outra no fim do bloco. Assim, a condição sempre testa o valor recém-lido, e a sentinela nunca entra na soma.

Em C++, o mesmo padrão costuma ser escrito de forma mais curta, com a leitura dentro da própria condição. A expressão `cin >> valor` lê o valor e, além disso, resulta em verdadeiro se a leitura deu certo e em falso se falhou, por exemplo quando a entrada acabou ou o usuário digitou uma letra. Assim, o laço para tanto na sentinela quanto em uma entrada que termina sem ela, o que evita um laço infinito:

```
#include <iostream>
using namespace std;

int main() {
    int valor, soma = 0, quantos = 0;
    cout << "Valores das compras (0 para terminar): ";
    while (cin >> valor && valor != 0) {   // lê e testa
        soma += valor;
```

```
        quantos++;
    }
    cout << quantos << " compras, total " << soma << endl;
    return 0;
}
// Entrada: 30 45 25 0  ->  3 compras, total 100
```

5.2 Repetição com teste no fim: do-while

O `do-while` ("faça enquanto") executa o bloco **primeiro** e só depois testa a condição. Por isso, o bloco é executado **pelo menos uma vez**. Ele é ideal quando a primeira execução é obrigatória, como pedir um dado e repetir o pedido enquanto ele for inválido.

```
#include <iostream>
using namespace std;

int main() {
    double nota;
    do {
        cout << "Nota (0 a 10): ";
        cin >> nota;
        if (nota < 0 || nota > 10) {
            cout << "Nota inválida, tente de novo." << endl;
        }
    } while (nota < 0 || nota > 10);   // repete se inválida
    cout << "Nota registrada: " << nota << endl;
    return 0;
}
// Entrada: 30, depois 7.5  ->  Nota inválida...
//                              Nota registrada: 7.5
```

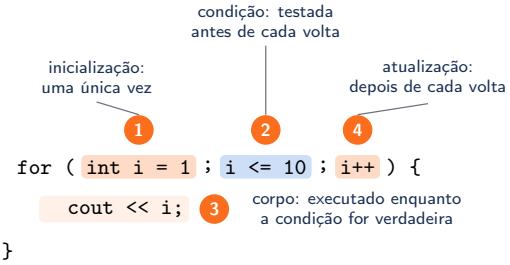
Atenção ao ponto e vírgula depois do `while (...)` do `do-while`: ele é obrigatório, ao contrário do `while` comum.

No Portugol, a estrutura equivalente é `repita ... ate`, com uma diferença importante: a condição do `ate` diz quando **parar**, e não quando continuar. O laço repete **até** a condição ficar verdadeira. Por isso, ela é o contrário da condição do `do-while`:

```
algoritmo "LerNota"
var
   nota: real
inicio
   repita
      escreva("Nota (0 a 10): ")
      leia(nota)
   ate (nota >= 0) e (nota <= 10)    // para quando for válida
   escreval("Nota registrada: ", nota)
fimalgoritmo
```

5.3 Repetição com contador: for

Quando se sabe **quantas vezes** repetir, o `for` ("para") é o laço mais claro, porque reúne em uma única linha as três partes de um laço com contador: a inicialização, a condição e a atualização.



ordem: 1, depois 2 → 3 → 4 repetidos até a condição 2 ficar falsa

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Tabuada do: ";
    cin >> n;
    for (int i = 1; i <= 10; i++) {
        cout << n << " x " << i << " = " << n * i << endl;
    }

    for (int i = 10; i >= 0; i -= 2) {   // de 10 até 0, de 2 em 2
        cout << i << " ";
    }
    cout << endl;                        // 10 8 6 4 2 0
    return 0;
}
```

A variável `i`, declarada dentro do `for`, só existe dentro do laço. O contador pode começar e terminar em qualquer valor, crescer ou diminuir, e andar de um em um ou em passos maiores: tudo depende das três partes.

No Portugol, o laço com contador é o `para ... de ... ate ... faca`, com um `passo` opcional (que vale 1 quando omitido):

```
algoritmo "Tabuada"
var
   n, i: inteiro
inicio
   escreva("Tabuada do: ")
   leia(n)
```

```
   para i de 1 ate 10 faca
      escreval(n, " x ", i, " = ", n * i)
   fimpara
   para i de 10 ate 0 passo -2 faca
      escreva(i, " ")
   fimpara
fimalgoritmo
```

Os três laços são equivalentes, no sentido de que qualquer repetição pode ser escrita com qualquer um deles. A escolha é sobre clareza:

Laço	Portugol	Use quando
`for`	`para ... fimpara`	o número de repetições é conhecido
`while`	`enquanto ... fimenquanto`	depende de algo que só se sabe durante a execução
`do-while`	`repita ... ate`	a primeira execução é obrigatória

5.4 Contadores, acumuladores, maior e menor

Quase todo laço que processa dados usa um ou mais destes quatro padrões:

- **Contador**: uma variável que começa em 0 e soma 1 a cada vez que algo acontece (`quantos++`). Com uma condição, conta só alguns casos: quantos aprovados, quantos negativos.
- **Acumulador**: uma variável que começa em 0 e soma um valor a cada volta (`soma += nota`). No fim, guarda o total. Para multiplicar, o acumulador começa em 1.
- **Maior valor**: compara cada dado com o maior visto até agora e o substitui quando encontra um maior.
- **Menor valor**: o mesmo, com a comparação invertida.

O programa abaixo usa os quatro para analisar as notas de uma turma:

```
#include <iostream>
using namespace std;

int main() {
    int alunos;
    cout << "Quantos alunos? ";
    cin >> alunos;
    double soma = 0;               // acumulador
    int aprovados = 0;             // contador
    double maior = 0, menor = 10;  // os extremos da faixa
    for (int i = 1; i <= alunos; i++) {
        double nota = 0;
        cout << "Nota do aluno " << i << ": ";
        cin >> nota;
        soma += nota;
        if (nota >= 6) aprovados++;
```

```
        if (nota > maior) maior = nota;
        if (nota < menor) menor = nota;
    }
    if (alunos > 0) {
        cout << "Média: " << soma / alunos << endl;
        cout << "Aprovados: " << aprovados << endl;
        cout << "Maior: " << maior << ", menor: " << menor
             << endl;
    }
    return 0;
}
// 4 alunos com 7, 4.5, 9 e 6 -> Média: 6.625
// Aprovados: 3    Maior: 9, menor: 4.5
```

O maior começa em 0 e o menor em 10, os extremos da faixa de notas: assim, a primeira nota lida sempre substitui os dois. Quando não há uma faixa conhecida, a forma mais segura é usar o **primeiro valor lido** como maior e menor iniciais. O `if (alunos > 0)` evita dividir por zero quando a turma está vazia.

Um teste de mesa mostra o laço trabalhando, volta a volta, com as notas 7, 4.5, 9 e 6:

i	nota	soma	aprovados	maior	menor
–	–	0	0	0	10
1	7	7	1	7	7
2	4.5	11.5	1	7	4.5
3	9	20.5	2	9	4.5
4	6	26.5	3	9	4.5

5.5 Interrompendo um laço: break e continue

Dois comandos alteram o andamento normal de um laço:

- `break` encerra o laço imediatamente, mesmo que a condição ainda seja verdadeira. É útil quando a resposta já foi encontrada e não faz sentido continuar procurando.
- `continue` pula o resto da iteração atual e passa direto para a próxima.

```
#include <iostream>
using namespace std;

int main() {
    // break: o primeiro múltiplo de 7 maior que 50
    for (int n = 51; n <= 100; n++) {
        if (n % 7 == 0) {
            cout << "Encontrado: " << n << endl;   // 56
            break;                 // não precisa olhar o resto
        }
    }

    // continue: soma só os ímpares de 1 a 10
```

```
    int soma = 0;
    for (int i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            continue;                  // pula os pares
        }
        soma += i;
    }
    cout << "Soma dos ímpares: " << soma << endl;   // 25
    return 0;
}
```

O Portugol do VisuAlg tem o comando `interrompa`, equivalente ao `break`, mas não tem equivalente ao `continue`. Use os dois com moderação: um laço com muitas saídas no meio é difícil de acompanhar. Muitas vezes, uma condição melhor no próprio laço ou um `if` resolvem o problema de forma mais clara.

5.6 Laços aninhados

Um laço pode estar dentro de outro. Para cada volta do laço de fora, o laço de dentro é executado **inteiro**. Se o de fora dá 4 voltas e o de dentro, 5, o bloco mais interno é executado $4 \times 5 = 20$ vezes. Laços aninhados aparecem sempre que os dados têm linhas e colunas: tabelas, matrizes (Capítulo 7), imagens.

```
#include <iostream>
#include <iomanip>     // setw
using namespace std;

int main() {
    // um triângulo: a linha i tem i asteriscos
    for (int linha = 1; linha <= 4; linha++) {
        for (int j = 1; j <= linha; j++) {
            cout << "*";
        }
        cout << endl;
    }

    // a tabela de multiplicação de 1 a 5
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= 5; j++) {
            cout << setw(4) << i * j;  // 4 posições por número
        }
        cout << endl;
    }
    return 0;
}
```

A saída mostra o triângulo e a tabela, com as colunas alinhadas pelo `setw(4)`, que reserva 4 posições para cada número:

```
// saída
*
**
***
****
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
   5  10  15  20  25
```

Repare que o limite do laço de dentro, no triângulo, depende da variável do laço de fora (`j <= linha`): é isso que faz cada linha ter um asterisco a mais.

5.7 O laço que não termina

Um **laço infinito** acontece quando a condição nunca fica falsa. O programa trava, ou fica mostrando a mesma coisa sem parar, e precisa ser interrompido à força (no terminal, com Ctrl+C). As causas mais comuns são:

- esquecer de atualizar a variável da condição, como o `contador-` do `while`;
- atualizar na direção errada: `for (int i = 10; i >= 0; i++)` se afasta da condição de parada em vez de se aproximar;
- usar uma condição que nunca pode ser satisfeita, como `while (x != 10)` com `x` andando de 3 em 3 a partir de 0: ele passa por 9 e 12, mas nunca vale 10.

O erro oposto também é comum: o laço que executa **uma vez a mais ou a menos** do que deveria, por usar `<` no lugar de `<=`, ou começar em 0 no lugar de 1. Ele é tão frequente que tem nome em inglês, *off-by-one*. Antes de confiar em um laço, verifique a primeira e a última volta: com que valor do contador elas acontecem?

5.8 Um programa completo: números primos

Um número é **primo** quando é maior que 1 e só é divisível por 1 e por ele mesmo: 2, 3, 5, 7, 11, 13... Para saber se `n` é primo, basta procurar um divisor entre 2 e `n - 1`; se algum dividir, ele não é primo.

Dá para procurar bem menos. Se `n` tiver um divisor maior que sua raiz quadrada, terá também um menor que ela: $36 = 4 \times 9$, e o 4 é menor que $\sqrt{36} = 6$. Então basta testar os divisores `d` enquanto `d * d <= n`. Para um número perto de um milhão, isso reduz a busca de cerca de um milhão de divisões para mil. E, assim que um divisor aparece, o `break` encerra a busca.

```
algoritmo "Primo"
var
   n, d: inteiro
   primo: logico
inicio
   escreva("Número: ")
   leia(n)
   primo <- n >= 2
```

```
   d <- 2
   enquanto (d * d <= n) e primo faca
      se n mod d = 0 entao
         primo <- falso
      fimse
      d <- d + 1
   fimenquanto
   se primo entao
      escreval(n, " é primo")
   senao
      escreval(n, " não é primo")
   fimse
fimalgoritmo
```

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Número: ";
    cin >> n;
    bool primo = n >= 2;              // 0 e 1 não são primos
    for (int d = 2; d * d <= n; d++) {
        if (n % d == 0) {
            primo = false;            // achou um divisor
            break;
        }
    }
    cout << n << (primo ? " é primo" : " não é primo") << endl;
    return 0;
}
// 2 -> é primo     9 -> não é primo    97 -> é primo
// 1 -> não é primo     1000003 -> é primo
```

O Portugol usa a própria variável `primo` na condição do `enquanto` para parar a busca, em vez de um comando de interrupção: as duas formas são corretas. Para testar, inclua os casos especiais: 0, 1 e 2 (o único primo par), um quadrado perfeito como 9 ou 49 (que só tem o divisor exatamente na raiz e testa o `<=`) e um primo grande.

Erros Comuns

- **Esquecer de atualizar a variável da condição.** O laço nunca termina.
- **Errar o limite por um.** `i < 10` dá 10 voltas começando em 0, mas 9 começando em 1. Confira a primeira e a última volta.
- **Colocar ponto e vírgula depois do `for` ou do `while`.** `for (...);` repete um comando vazio, e o bloco abaixo executa uma única vez.
- **Esquecer o ponto e vírgula do `do-while`.** Ele é obrigatório depois de `while (condição)`.
- **Inverter a condição do `repita ... ate`.** No Portugol ela diz quando parar, ao contrário do `do-while`.

- **Não inicializar contadores e acumuladores.** Uma soma que começa com lixo dá lixo. Comece em 0 (ou em 1, para produtos).
- **Inicializar o maior ou o menor com um valor possível.** Começar o maior em 0 falha se todos os dados forem negativos. Use um valor fora da faixa ou o primeiro dado lido.
- **Somar a sentinela.** No laço com sentinela, leia antes do laço e no fim do bloco (ou, em C++, dentro da condição), para que o valor de parada nunca seja processado.
- **Dividir pela quantidade sem verificar se ela é zero.** Uma média de nenhum valor não existe.

Capítulo 6 – Funções e procedimentos

Os programas dos capítulos anteriores cabiam inteiros no `main`. Programas reais têm milhares ou milhões de linhas, e ninguém consegue entender, testar ou corrigir um bloco desse tamanho de uma vez. A saída é a mesma que usamos para qualquer problema grande: dividir em partes menores, cada uma com uma tarefa clara. Em programação, essas partes são os **subprogramas**: as **funções**, que calculam e devolvem um valor, e os **procedimentos**, que executam uma tarefa sem devolver nada. Neste capítulo você vai aprender a criar os seus, a passar dados para eles e a entender o que acontece na memória a cada chamada.

6.1 Dividir para conquistar

Você já usa funções desde o Capítulo 3, mesmo sem ter escrito nenhuma. A `sqrt`, que calcula a raiz quadrada, e a `pow`, que calcula potências, são funções prontas da biblioteca `<cmath>`:

```
#include <iostream>
#include <cmath>       // sqrt, pow
using namespace std;

int main() {
    double r = sqrt(49.0);       // 7
    double p = pow(2.0, 10);     // 1024
    cout << r << " " << p << endl;
    return 0;
}
```

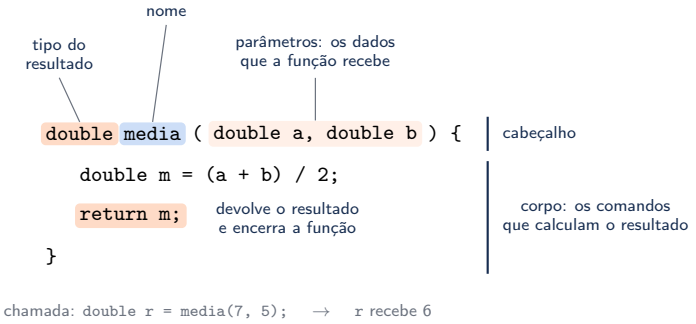
Para usar a `sqrt`, você não precisa saber como ela calcula a raiz: basta saber o que ela recebe (um número) e o que devolve (a raiz). Essa ideia, usar algo pelo que ele faz sem se preocupar com como faz, chama-se **abstração**, e é o principal ganho de dividir um programa em subprogramas. Há outros:

- **Reaproveitamento**: o código escrito uma vez é chamado quantas vezes for preciso, em vez de ser copiado e colado.
- **Manutenção**: um erro corrigido na função fica corrigido em todos os lugares que a usam.
- **Teste**: cada função pode ser testada sozinha, com poucos casos, antes de ser combinada com as outras.
- **Legibilidade**: um `main` que só chama `lerNotas`, depois `calcularMedia` e por fim `mostrarResultado` se lê quase como um texto.

Dividir um programa em partes independentes chama-se **modularização**. A regra prática é simples: cada subprograma deve fazer **uma única tarefa**, e seu nome deve dizer qual é.

6.2 Funções: recebem dados e devolvem um resultado

Uma função tem um **cabeçalho**, que diz o tipo do resultado, o nome e os dados que ela recebe, e um **corpo**, entre chaves, com os comandos que calculam o resultado.



Os **parâmetros** são variáveis declaradas no cabeçalho, que recebem os valores passados na chamada. Esses valores são os **argumentos**. Na chamada `media(n1, n2)`, o parâmetro `a` recebe o valor de `n1` e o `b`, o de `n2`: a correspondência é pela **posição**, não pelo nome. O comando `return` encerra a função e entrega o resultado a quem a chamou.

```
#include <iostream>
using namespace std;

double media(double a, double b) {
    double m = (a + b) / 2;
    return m;
}

int main() {
    double n1, n2;
    cout << "Duas notas: ";
    cin >> n1 >> n2;
    double resultado = media(n1, n2);
    cout << "Média: " << resultado << endl;
    cout << "Média de 8 e 5: " << media(8, 5) << endl;   // 6.5
    return 0;
}
```

A chamada `media(n1, n2)` é uma expressão, como `n1 + n2`: o resultado pode ser guardado em uma variável, mostrado direto no `cout` ou usado dentro de outra conta, como em `media(n1, n2) >= 6`. Os argumentos também podem ser expressões ou valores literais, como em `media(8, 5)`.

Em Portugol, a função é declarada entre a seção `var` do algoritmo e o `inicio`, com `funcao ... fimfuncao`. O tipo do resultado vem depois dos parâmetros, e o resultado é devolvido com `retorne`:

```
algoritmo "MediaComFuncao"
var
   n1, n2, resultado: real
```

```
funcao media(a, b: real): real
var
   m: real
inicio
   m <- (a + b) / 2
   retorne m
fimfuncao

inicio
   escreva("Duas notas: ")
   leia(n1, n2)
   resultado <- media(n1, n2)
   escreval("Média: ", resultado)
fimalgoritmo
```

Mais de um return

O `return` encerra a função **imediatamente**, mesmo no meio de um laço. Por isso, uma função pode ter vários, um para cada caso. O teste de número primo do Capítulo 5 fica mais curto como função: assim que um divisor aparece, a resposta já é conhecida.

```
#include <iostream>
using namespace std;

bool ehPrimo(int n) {
    if (n < 2) return false;           // 0 e 1 não são primos
    for (int d = 2; d * d <= n; d++) {
        if (n % d == 0) return false;  // achou divisor: sai já
    }
    return true;                       // nenhum divisor
}

int main() {
    cout << "Primos até 50:";
    for (int n = 1; n <= 50; n++) {
        if (ehPrimo(n)) cout << " " << n;
    }
    cout << endl;   // 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
    return 0;
}
```

Compare com o Capítulo 5: lá, o teste usava uma variável `primo` e um `break`. Aqui, o `main` só pergunta "`n` é primo?" e não precisa saber como a resposta é obtida. Funções que respondem sim ou não costumam ter nomes como `ehPrimo`, `ehPar` ou `estaAprovado`, e devolvem `bool`.

Toda função que não é `void` precisa devolver um valor em **todos** os caminhos. Se algum caminho chegar ao fim do corpo sem um `return`, o resultado é indefinido, e o g++ avisa com `-Wall`.

6.3 Procedimentos: executam uma tarefa

Nem todo subprograma calcula um valor. Mostrar um cabeçalho, desenhar uma linha ou imprimir um relatório são tarefas que não têm resultado a devolver. Esses subprogramas são os **procedimentos**. Em C++, um procedimento é uma função com tipo de retorno `void` ("vazio"):

```
#include <iostream>
#include <string>
using namespace std;

void linha(int tamanho, char simbolo) {
    for (int i = 0; i < tamanho; i++) {
        cout << simbolo;
    }
    cout << endl;
}

void cabecalho(string titulo) {
    linha(30, '=');
    cout << titulo << endl;
    linha(30, '=');
}

int main() {
    cabecalho("Relatório de notas");
    cout << "Ana     8.5" << endl;
    cout << "Bruno   6.0" << endl;
    linha(30, '-');
    return 0;
}
```

Repare que `cabecalho` chama `linha`: subprogramas podem (e devem) usar outros subprogramas. Um procedimento não tem `return` com valor, mas pode usar `return;` sozinho para terminar mais cedo, por exemplo quando recebe um dado inválido.

Em Portugol, o procedimento é escrito com `procedimento ...  fimprocedimento`, sem tipo de retorno:

```
algoritmo "Relatorio"

procedimento linha(tamanho: inteiro)
var
   i: inteiro
inicio
   para i de 1 ate tamanho faca
      escreva("-")
   fimpara
   escreval("")
fimprocedimento

inicio
```

```
    linha(30)
    escreval("Relatório de notas")
    linha(30)
fimalgoritmo
```

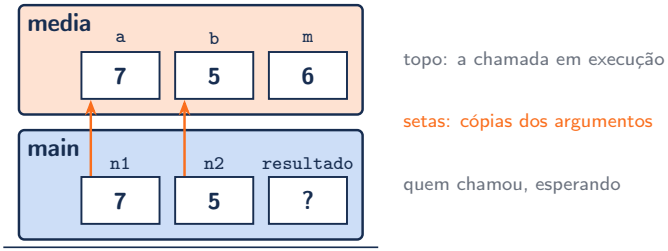
A diferença entre os dois tipos de subprograma aparece também na chamada. Uma função é usada **como um valor**, dentro de uma expressão: `x <- media(a, b)`. Um procedimento é usado **como um comando**, sozinho na linha: `linha(30)`.

6.4 O que acontece em uma chamada

Quando o programa chama uma função, quatro coisas acontecem, nesta ordem:

1. os argumentos são avaliados e seus valores são **copiados** para os parâmetros;
2. o programa salta para o início do corpo da função, e anota para onde deve voltar;
3. o corpo é executado, com seus próprios parâmetros e variáveis;
4. no `return` (ou no fim do corpo), o resultado é entregue, as variáveis da função deixam de existir e o programa continua do ponto onde parou.

Para organizar isso, o computador usa uma região da memória chamada **pilha de chamadas**. Cada chamada ganha um **registro de ativação**, um espaço com seus parâmetros e variáveis locais, empilhado sobre o de quem a chamou. Quando a função termina, seu registro é desempilhado. A figura mostra a pilha durante a chamada `media(n1, n2)`, com as notas 7 e 5:



Duas consequências importantes vêm daí. Primeiro, `a` e `b` são variáveis novas, que só recebem cópias de `n1` e `n2`: mudar `a` dentro da função não muda `n1`. Segundo, cada chamada tem seu próprio registro: chamar `media` dez vezes cria e destrói dez registros, e nenhuma chamada "lembra" os valores da anterior. A pilha também explica a recursão, assunto do Capítulo 9, em que uma função chama a si mesma.

6.5 Escopo: onde cada variável existe

O **escopo** de uma variável é a parte do programa em que ela pode ser usada. Em C++, a regra é: uma variável existe do ponto em que é declarada até o fim do bloco (`{ ... }`) que a contém.

- **Variáveis locais** são declaradas dentro de uma função (ou de um bloco, como o corpo de um `for`). Só existem ali, e somem quando o bloco termina. Os parâmetros também são locais.
- **Variáveis globais** são declaradas fora de qualquer função, e podem ser usadas por todas as funções do arquivo.

```
#include <iostream>
using namespace std;

int chamadas = 0;               // global: visível em todo lugar

int dobro(int x) {
    chamadas++;
    int resultado = 2 * x;      // local: só existe em dobro
    return resultado;
}

int main() {
    int x = 5;                  // outro x, local de main
    cout << dobro(x) << endl;   // 10
    cout << dobro(7) << endl;   // 14
    cout << "Chamadas: " << chamadas << endl;   // 2
    // cout << resultado;   erro: resultado não existe aqui
    return 0;
}
```

O `x` do `main` e o parâmetro `x` de `dobro` são variáveis **diferentes**, que só têm o mesmo nome. Isso é normal e até desejável: quem escreve `dobro` não precisa saber quais nomes o `main` usa.

Variáveis globais parecem práticas, porque evitam passar parâmetros, mas cobram um preço alto: qualquer função pode alterá-las, e quando o valor está errado é preciso investigar o programa inteiro para descobrir quem o mudou. Prefira passar os dados por parâmetro e devolvê-los com `return`. A exceção aceitável são as **constantes** globais, como `const double PI = 3.14159;`, que ninguém pode alterar.

No Portugol, as variáveis declaradas na seção `var` do algoritmo são globais, e as declaradas na seção `var` de uma função ou procedimento são locais.

6.6 Passagem por valor e por referência

Até aqui, todos os parâmetros receberam **cópias** dos argumentos. Essa é a **passagem por valor**, a forma padrão, e ela protege as variáveis de quem chama: a função pode fazer o que quiser com suas cópias. Mas às vezes queremos justamente que a função altere a variável original. O exemplo clássico é trocar o conteúdo de duas variáveis:

```
#include <iostream>
using namespace std;

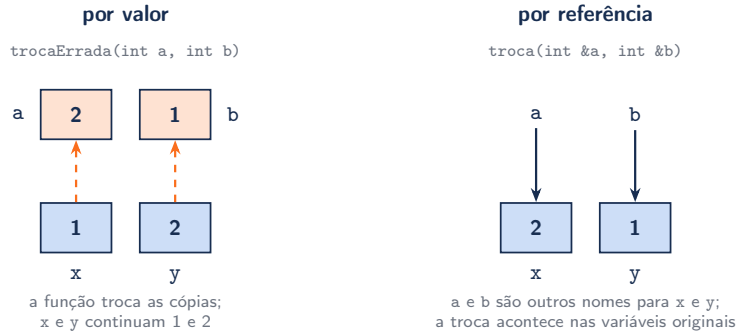
void trocaErrada(int a, int b) {     // por valor: cópias
    int aux = a;
```

```
    a = b;
    b = aux;
}

void troca(int &a, int &b) {          // por referência
    int aux = a;
    a = b;
    b = aux;
}

int main() {
    int x = 1, y = 2;
    trocaErrada(x, y);
    cout << x << " " << y << endl;    // 1 2: nada mudou
    troca(x, y);
    cout << x << " " << y << endl;    // 2 1
    return 0;
}
```

A única diferença entre as duas versões é o `&` antes do nome dos parâmetros. Com ele, o parâmetro deixa de ser uma cópia e passa a ser uma **referência**: outro nome para a mesma variável da chamada. Tudo o que a função faz com `a`, na verdade, faz com `x`.



No Portugol, a passagem por referência é indicada pela palavra `var` antes dos parâmetros:

```
algoritmo "Troca"
var
   x, y: inteiro

procedimento troca(var a, b: inteiro)
var
   aux: inteiro
inicio
   aux <- a
   a <- b
```

```
   b <- aux
fimprocedimento

inicio
   x <- 1
   y <- 2
   troca(x, y)
   escreval(x, " ", y)        // 2 1
fimalgoritmo
```

Devolvendo mais de um resultado

O `return` devolve um único valor. Quando uma tarefa produz dois ou mais resultados, a passagem por referência resolve: a função recebe variáveis de quem chama e as preenche.

```
#include <iostream>
using namespace std;

void dividir(int a, int b, int &quociente, int &resto) {
    quociente = a / b;
    resto = a % b;
}

int main() {
    int q, r;
    dividir(17, 5, q, r);
    cout << "17 = 5 x " << q << " + " << r << endl;
    return 0;                // 17 = 5 x 3 + 2
}
```

Como um parâmetro por referência precisa de uma variável à qual se ligar, a chamada `troca(1, 2)` não compila: não dá para trocar o conteúdo do número 1.

Referência constante

Copiar um `int` é barato, mas copiar um texto longo ou, mais adiante, um vetor com milhares de elementos, não é. Para evitar a cópia sem permitir alterações, usa-se a **referência constante**: `void saudar(const string &nome)`. A função lê o original, sem copiá-lo, e o compilador recusa qualquer tentativa de alterá-lo.

Passagem	Como se escreve	Use quando
por valor	`int a`	a função só precisa do valor (o caso mais comum)
por referência	`int &a` Portugol: `var a`	a função precisa alterar a variável de quem chama
referência constante	`const string &a`	o dado é grande e a função só vai lê-lo

6.7 Protótipos, valores padrão e sobrecarga

Protótipos

O compilador lê o arquivo de cima para baixo, e uma função só pode ser chamada depois de ser declarada. Por isso, até aqui, as funções vieram antes do `main`. Quando se prefere o `main` no topo, para quem lê o programa ver primeiro o panorama, declara-se antes só o cabeçalho da função, terminado em ponto e vírgula. Esse cabeçalho é o **protótipo**, e a definição completa pode vir depois:

```
#include <iostream>
using namespace std;

double areaCirculo(double raio);       // protótipo

int main() {
    cout << areaCirculo(2) << endl;    // 12.5664
    return 0;
}

double areaCirculo(double raio) {      // definição
    const double PI = 3.14159265358979;
    return PI * raio * raio;
}
```

Valores padrão e sobrecarga

O C++ tem dois recursos que o Portugol não tem. Um parâmetro pode ter um **valor padrão**, usado quando o argumento é omitido na chamada. E duas funções podem ter o **mesmo nome**, desde que os parâmetros sejam diferentes em quantidade ou tipo: é a **sobrecarga**. O compilador escolhe qual chamar pelos argumentos.

```
#include <iostream>
using namespace std;

void linha(int tamanho = 20, char simbolo = '-') {
    for (int i = 0; i < tamanho; i++) cout << simbolo;
    cout << endl;
}

int maior(int a, int b) { return a > b ? a : b; }
double maior(double a, double b) { return a > b ? a : b; }

int main() {
    linha();             // 20 traços
    linha(5);            // -----
    linha(5, '*');       // *****
    cout << maior(3, 8) << " " << maior(2.5, 1.5) << endl;
    return 0;            // 8 2.5
}
```

Os parâmetros com valor padrão precisam ser os últimos da lista: não dá para omitir

o primeiro argumento e informar o segundo.

6.8 Um programa completo: notas da turma

O programa abaixo refaz a análise de notas do Capítulo 5, agora dividida em subprogramas. Cada um tem uma única tarefa: `lerNota` pede uma nota até ela ser válida, `situacao` classifica uma nota e `mostrarResumo` imprime o resultado final.

```
#include <iostream>
#include <string>
using namespace std;

double lerNota(int aluno) {
    double nota = 0;
    do {
        cout << "Nota do aluno " << aluno << " (0 a 10): ";
        cin >> nota;
    } while (nota < 0 || nota > 10);
    return nota;
}

string situacao(double nota) {
    if (nota >= 6) return "aprovado";
    if (nota >= 4) return "recuperação";
    return "reprovado";
}

void mostrarResumo(double soma, int alunos, int aprovados) {
    cout << "Média da turma: " << soma / alunos << endl;
    cout << "Aprovados: " << aprovados << " de " << alunos
         << endl;
}

int main() {
    int alunos;
    cout << "Quantos alunos? ";
    cin >> alunos;
    double soma = 0;
    int aprovados = 0;
    for (int i = 1; i <= alunos; i++) {
        double nota = lerNota(i);
        string s = situacao(nota);
        cout << "  " << s << endl;
        soma += nota;
        if (s == "aprovado") aprovados++;
    }
    if (alunos > 0) mostrarResumo(soma, alunos, aprovados);
    return 0;
}
// 3 alunos com 7, 4.5 e 3 -> aprovado, recuperação,
// reprovado; Média da turma: 4.83333; Aprovados: 1 de 3
```

Repare em uma escolha de projeto: `situacao` não mostra nada na tela, só devolve o

texto. Quem decide o que fazer com ele é o `main`, que o imprime e também o usa para contar os aprovados. Separar o **cálculo** da **entrada e saída** deixa as funções mais fáceis de reaproveitar e de testar: para conferir `situacao`, basta chamá-la com 6, 5.9, 4 e 3.9, os valores nas fronteiras de cada faixa.

Erros Comuns

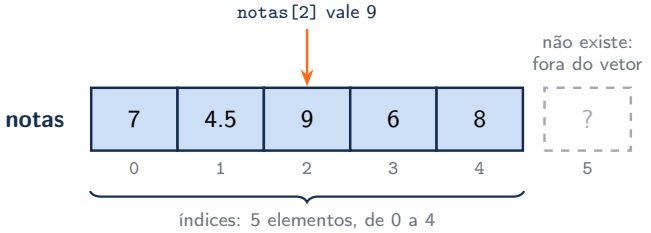
- **Esquecer o `return` em algum caminho.** Uma função que não é `void` precisa devolver um valor em todos os casos. Compile com `-Wall` para ser avisado.
- **Chamar uma função e ignorar o resultado.** `media(n1, n2);` sozinho na linha calcula a média e a joga fora. Guarde o resultado ou use-o em uma expressão.
- **Esperar que a passagem por valor altere a variável.** O parâmetro é uma cópia. Para alterar o original, use `&` em C++ ou `var` em Portugol.
- **Trocar a ordem dos argumentos.** A correspondência é pela posição: `dividir(5, 17, q, r)` divide 5 por 17, não 17 por 5.
- **Usar uma variável local fora da sua função.** Ela só existe dentro do bloco em que foi declarada.
- **Abusar de variáveis globais.** Qualquer função pode alterá-las, e os erros ficam difíceis de rastrear. Use parâmetros e `return`.
- **Chamar a função antes de declará-la.** Coloque a definição antes do `main` ou declare o protótipo no início do arquivo.
- **Fazer funções que fazem tudo.** Se o nome precisa de um "e" (`lerECalcularEMostrar`), provavelmente são três funções.

Capítulo 7 – Vetores e matrizes

Calcular a média de uma turma é fácil com um acumulador, como no Capítulo 5: cada nota é lida, somada e esquecida. Mas e se, depois da média, for preciso listar quem ficou acima dela? Agora as notas não podem ser esquecidas, porque cada uma vai ser comparada com um valor que só se conhece no fim. Declarar `nota1`, `nota2`, . . . , `nota40` não resolve: o programa ficaria enorme e não serviria para uma turma de 41 alunos. A solução são as **estruturas de dados homogêneas**: o **vetor**, uma sequência de valores do mesmo tipo sob um único nome, e a **matriz**, uma tabela com linhas e colunas. Neste capítulo você vai aprender a declará-los, percorrê-los com laços, passá-los para funções e evitar o erro mais perigoso que eles trazem: acessar uma posição que não existe.

7.1 O que é um vetor

Um **vetor** (ou *array*) é um conjunto de variáveis do mesmo tipo, guardadas lado a lado na memória e identificadas por um único nome. Cada variável do conjunto é um **elemento**, e é acessada pelo seu **índice**, a posição que ocupa no vetor, escrito entre colchetes: `notas[2]` é o elemento de índice 2 do vetor `notas`.



Em C++, o primeiro índice é sempre **0**. Por isso, um vetor de 5 elementos tem os índices de 0 a 4, e o último elemento de um vetor de tamanho `n` é o de índice `n - 1`. Na declaração, o número entre colchetes é o **tamanho**, que precisa ser uma constante:

```
#include <iostream>
using namespace std;

int main() {
    double notas[5];                  // 5 elementos, de 0 a 4
    notas[0] = 7;
    notas[1] = 4.5;
    notas[4] = notas[0] + 1;          // 8

    int primos[] = {2, 3, 5, 7, 11};  // tamanho deduzido: 5
    int contagem[10] = {};            // todos começam em 0
    int i = 3;
    cout << primos[i] << " " << primos[i + 1] << endl;   // 7 11
    cout << notas[4] << " " << contagem[9] << endl;      // 8 0
    return 0;
}
```

Como uma variável comum, um vetor declarado sem valores iniciais contém lixo. A lista entre chaves preenche os elementos na ordem, e os que sobram ficam com 0: por isso `= {}` é a forma mais curta de zerar um vetor inteiro. O índice pode ser qualquer expressão inteira, como `i` ou `i + 1`, e é isso que torna o vetor poderoso: um laço pode visitar todos os elementos mudando apenas o índice.

7.2 Percorrendo um vetor com laços

O `for` e o vetor foram feitos um para o outro: o contador do laço serve de índice e passa por todas as posições, da primeira à última. O programa abaixo resolve o problema do início do capítulo. O primeiro laço lê as notas e as soma; o segundo, que só pode ser feito depois que a média é conhecida, mostra as que ficaram acima dela.

```
#include <iostream>
using namespace std;

int main() {
    const int MAX = 50;
    double notas[MAX] = {};
    int n;
    cout << "Quantos alunos (até 50)? ";
    cin >> n;
    if (n < 1 || n > MAX) {
        cout << "Quantidade inválida." << endl;
        return 0;
    }
    double soma = 0;
    for (int i = 0; i < n; i++) {
        cout << "Nota " << i + 1 << ": ";
        cin >> notas[i];
        soma += notas[i];
    }
    double media = soma / n;
    cout << "Média: " << media << endl;
    cout << "Acima da média:";
    for (int i = 0; i < n; i++) {
        if (notas[i] > media) cout << " " << notas[i];
    }
    cout << endl;
    return 0;
}
// 4 alunos com 7, 4.5, 9 e 6 -> Média: 6.625
// Acima da média: 7 9
```

Repare em três detalhes. O vetor é declarado com uma capacidade máxima (`MAX`), porque seu tamanho precisa ser conhecido na compilação, e o programa usa só as `n` primeiras posições. O `if` logo depois da leitura garante que `n` cabe no vetor. E o laço vai de `0` enquanto `i < n`, a forma padrão de percorrer `n` elementos em C++; para o usuário, que conta a partir de 1, o programa mostra `i + 1`.

Em Portugol, o vetor é declarado com o tipo `vetor`, informando o primeiro e o último índice. Diferente do C++, quem escolhe os limites é o programador, e o mais comum é começar em 1:

```
algoritmo "AcimaDaMedia"
var
   notas: vetor[1..50] de real
   n, i: inteiro
   soma, media: real
inicio
   escreva("Quantos alunos (até 50)? ")
   leia(n)
   soma <- 0
   para i de 1 ate n faca
      escreva("Nota ", i, ": ")
      leia(notas[i])
      soma <- soma + notas[i]
   fimpara
   media <- soma / n
   escreval("Média: ", media)
   escreval("Acima da média:")
   para i de 1 ate n faca
      se notas[i] > media entao
         escreval(notas[i])
      fimse
   fimpara
fimalgoritmo
```

7.3 Cuidado com os limites

O que acontece com `notas[5]` em um vetor de 5 elementos? Em C++, a resposta é: **qualquer coisa**. A linguagem não confere se o índice existe. Ela simplesmente calcula onde estaria o elemento 5, logo depois do último, e lê ou escreve naquela posição da memória, que pertence a outra variável ou nem pertence ao programa.

```
// erro: índices fora do vetor
int v[5] = {1, 2, 3, 4, 5};
for (int i = 0; i <= 5; i++) {   // <= faz uma volta a mais
    cout << v[i] << " ";         // v[5] não existe
}
v[-1] = 0;                       // nem v[-1]
```

O resultado é chamado de **comportamento indefinido**: o programa pode mostrar um valor sem sentido, alterar silenciosamente outra variável, travar ou, o pior caso, funcionar normalmente nos testes e falhar mais tarde. O compilador às vezes avisa quando o índice é uma constante, mas nunca quando ele vem de um cálculo. Esse tipo de erro, conhecido como **estouro de vetor** (*buffer overflow*), já foi a porta de entrada de muitos ataques a sistemas, porque permite escrever onde o programa não deveria.

A defesa é disciplina: percorra com `i < n`, e não `i <= n`; confira, antes de usar, todo índice que vem do usuário ou de um cálculo; e nunca deixe o número de elementos passar da capacidade declarada. O Portugol do VisuAlg, ao contrário do C++, interrompe a execução com uma mensagem de erro quando o índice sai dos limites.

7.4 Algoritmos clássicos com vetores

Contagem por categoria

Quando os valores possíveis são poucos números inteiros, o próprio valor pode servir de índice. O programa abaixo apura uma pesquisa de satisfação com notas de 1 a 5: em vez de cinco contadores separados, um vetor `votos`, em que `votos[k]` conta quantas vezes a nota `k` apareceu.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int votos[6] = {};        // usa 1 a 5; a posição 0 sobra
    int voto;
    cout << "Notas de 1 a 5 (0 para terminar): ";
    while (cin >> voto && voto != 0) {
        if (voto >= 1 && voto <= 5) {
            votos[voto]++;    // o voto é o índice
        }
    }
    for (int k = 1; k <= 5; k++) {
        cout << k << ": " << string(votos[k], '*')
             << " (" << votos[k] << ")" << endl;
    }
    return 0;
}
```

Com a entrada `5 4 5 3 5 4 1 0`, a saída é um pequeno gráfico de barras, desenhado com `string(n, '*')`, que cria um texto com `n` asteriscos:

```
// saída
1: * (1)
2:  (0)
3: * (1)
4: ** (2)
5: *** (3)
```

O `if` que confere a faixa é essencial: sem ele, um voto 9 escreveria em `votos[9]`, fora do vetor.

Maior elemento e sua posição

O padrão do maior valor, do Capítulo 5, ganha uma informação extra com vetores: **onde** o maior está. Basta guardar o índice em vez do valor, começando pelo primeiro elemento:

```
#include <iostream>
using namespace std;
```

```
int main() {
    double temp[7] = {22.5, 25.1, 27.8, 24.0, 28.3, 26.6, 21.9};
    int iMaior = 0;                    // começa pelo primeiro
    for (int i = 1; i < 7; i++) {
        if (temp[i] > temp[iMaior]) iMaior = i;
    }
    cout << "Dia mais quente: " << iMaior + 1
         << " (" << temp[iMaior] << " graus)" << endl;
    return 0;                          // dia 5 (28.3 graus)
}
```

Invertendo um vetor

Para inverter um vetor no próprio lugar, troca-se o primeiro com o último, o segundo com o penúltimo, e assim por diante, com dois índices que caminham um em direção ao outro até se encontrarem no meio:

```
#include <iostream>
using namespace std;

int main() {
    int v[6] = {1, 2, 3, 4, 5, 6};
    for (int i = 0, j = 5; i < j; i++, j--) {
        int aux = v[i];                // a troca do Capítulo 6
        v[i] = v[j];
        v[j] = aux;
    }
    for (int i = 0; i < 6; i++) cout << v[i] << " ";
    cout << endl;                      // 6 5 4 3 2 1
    return 0;
}
```

A condição `i < j` é o ponto delicado: com `i < 6`, o laço inverteria o vetor e depois o desinverteria, voltando ao original. A biblioteca do C++ oferece a função `swap(v[i], v[j])`, que faz a mesma troca das três linhas. Outros algoritmos clássicos com vetores, a busca de um valor e a ordenação, têm um capítulo inteiro, o 10.

7.5 Vetores e funções

Um vetor pode ser passado para uma função, com uma diferença importante em relação às variáveis comuns: ele **não é copiado**. A função recebe a localização do vetor original na memória, e por isso tudo o que ela altera nos elementos fica alterado para quem chamou, como na passagem por referência. Além disso, a função não tem como descobrir o tamanho do vetor: ele precisa ser passado em um parâmetro separado.

```
#include <iostream>
using namespace std;

double media(const double v[], int n) {   // const: só lê
    double soma = 0;
```

```
    for (int i = 0; i < n; i++) soma += v[i];
    return soma / n;
}

void aumentar(double v[], int n, double bonus) {
    for (int i = 0; i < n; i++) {
        v[i] += bonus;
        if (v[i] > 10) v[i] = 10;        // nota máxima: 10
    }
}

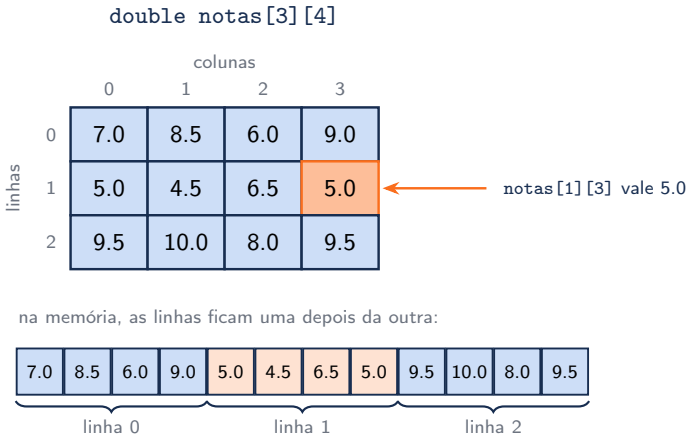
int main() {
    double notas[4] = {7, 4.5, 9.8, 6};
    cout << "Antes: " << media(notas, 4) << endl;   // 6.825
    aumentar(notas, 4, 0.5);             // altera o original
    cout << "Depois: " << media(notas, 4) << endl;  // 7.25
    return 0;
}
```

O parâmetro é escrito com colchetes vazios, `double v[]`. Quando a função só lê o vetor, como `media`, o `const` documenta isso e faz o compilador recusar qualquer alteração por engano. Uma função também não pode devolver um vetor com `return`: para produzir um vetor, ela recebe um vetor de quem chama e o preenche.

No VisuAlg, vetores não podem ser passados como parâmetro. Os subprogramas trabalham com o vetor declarado na seção `var` do algoritmo, que é global, e recebem por parâmetro apenas os valores simples, como a quantidade de elementos.

7.6 Matrizes

Muitos dados têm duas dimensões: as notas de cada aluno em cada prova, as vendas de cada loja em cada mês, os pixels de uma imagem, o tabuleiro de um jogo. Para eles existe a **matriz**, um vetor de duas dimensões, em que cada elemento é identificado por dois índices: a **linha** e a **coluna**.



Em C++, a matriz é declarada com dois tamanhos, `double notas[3][4]` (3 linhas e 4 colunas), e cada elemento é acessado com dois pares de colchetes: `notas[1][2]` é o elemento da linha 1, coluna 2. Na memória, as linhas ficam uma depois da outra, como mostra a parte de baixo da figura.

Percorrer uma matriz pede dois laços aninhados (Capítulo 5): o de fora passa pelas linhas e, para cada linha, o de dentro passa pelas colunas. O programa abaixo calcula a média de cada aluno (somando uma linha) e a média de cada prova (somando uma coluna):

```
#include <iostream>
using namespace std;

const int ALUNOS = 3, PROVAS = 4;

int main() {
    double notas[ALUNOS][PROVAS] = {
        {7.0, 8.5, 6.0, 9.0},       // aluno 0
        {5.0, 4.5, 6.5, 5.0},       // aluno 1
        {9.5, 10.0, 8.0, 9.5}       // aluno 2
    };
    for (int i = 0; i < ALUNOS; i++) {          // cada linha
        double soma = 0;
        for (int j = 0; j < PROVAS; j++) soma += notas[i][j];
        cout << "Aluno " << i + 1 << ": " << soma / PROVAS
             << endl;
    }
    for (int j = 0; j < PROVAS; j++) {          // cada coluna
        double soma = 0;
        for (int i = 0; i < ALUNOS; i++) soma += notas[i][j];
        cout << "Prova " << j + 1 << ": " << soma / ALUNOS
             << endl;
    }
```

```
    return 0;
}
// Aluno 1: 7.625   Aluno 2: 5.25   Aluno 3: 9.25
// Prova 1: 7.16667   Prova 2: 7.66667 ...
```

Compare os dois pares de laços: para somar uma **linha**, a linha `i` fica fixa e a coluna `j` varia; para somar uma **coluna**, é o contrário. Trocar os papéis de `i` e `j` é um dos erros mais comuns com matrizes.

Em Portugol, a matriz também é um `vetor`, declarado com os limites das duas dimensões separados por vírgula, e os dois índices também vão separados por vírgula:

```
algoritmo "NotasDaTurma"
var
   notas: vetor[1..3, 1..4] de real
   i, j: inteiro
   soma: real
inicio
   para i de 1 ate 3 faca
      soma <- 0
      para j de 1 ate 4 faca
         escreva("Aluno ", i, ", prova ", j, ": ")
         leia(notas[i, j])
         soma <- soma + notas[i, j]
      fimpara
      escreval("Média do aluno ", i, ": ", soma / 4)
   fimpara
fimalgoritmo
```

Para passar uma matriz a uma função em C++, o número de **colunas** precisa aparecer no parâmetro, como em `void mostrar(const double m[][PROVAS], int linhas)`. Só o número de linhas pode ficar vazio, porque o compilador precisa saber o tamanho de cada linha para calcular onde começa a linha seguinte.

7.7 Um programa completo: jogo da velha

O tabuleiro do jogo da velha é uma matriz 3×3 de caracteres. Para saber se alguém venceu, é preciso verificar as 3 linhas, as 3 colunas e as 2 diagonais. Na **diagonal principal**, a linha e a coluna são iguais (`t[0][0]`, `t[1][1]`, `t[2][2]`); na **diagonal secundária**, elas somam 2 (`t[0][2]`, `t[1][1]`, `t[2][0]`).

```
#include <iostream>
using namespace std;

bool iguais(char a, char b, char c) {
    return a != ' ' && a == b && b == c;   // casa vazia não vale
}

char vencedor(const char t[3][3]) {
    for (int i = 0; i < 3; i++) {
```

```
        if (iguais(t[i][0], t[i][1], t[i][2])) return t[i][0];
        if (iguais(t[0][i], t[1][i], t[2][i])) return t[0][i];
    }
    if (iguais(t[0][0], t[1][1], t[2][2])) return t[1][1];
    if (iguais(t[0][2], t[1][1], t[2][0])) return t[1][1];
    return ' ';                              // ninguém venceu
}

void mostrar(const char t[3][3]) {
    for (int i = 0; i < 3; i++) {
        cout << " " << t[i][0] << " | " << t[i][1]
             << " | " << t[i][2] << endl;
        if (i < 2) cout << "---+---+---" << endl;
    }
}

int main() {
    char t[3][3] = {
        {'X', 'O', 'X'},
        {'O', 'X', 'O'},
        {' ', 'O', 'X'}
    };
    mostrar(t);
    char v = vencedor(t);
    if (v == ' ') cout << "Ninguém venceu ainda." << endl;
    else cout << "Venceu: " << v << endl;   // Venceu: X
    return 0;
}
```

O laço de `vencedor` aproveita a mesma variável `i` para as linhas (`t[i][...]`) e para as colunas (`t[...][i]`). A função `iguais` evita repetir a mesma comparação oito vezes e, ao exigir `a != ' '`, impede que uma linha vazia seja contada como vitória. Para testar, monte tabuleiros que vençam por linha, por coluna, por cada diagonal, e um sem vencedor.

7.8 Vetores com tamanho variável: vector

O vetor da linguagem C++, herdado da linguagem C, tem tamanho fixo e não confere índices. A biblioteca padrão oferece uma alternativa mais segura e flexível, o `vector`, que cresce conforme os elementos são acrescentados e sabe o próprio tamanho:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<double> notas;            // começa vazio
    double nota;
    cout << "Notas (negativo para terminar): ";
    while (cin >> nota && nota >= 0) {
        notas.push_back(nota);       // acrescenta no fim
```

```
    }
    double soma = 0;
    for (size_t i = 0; i < notas.size(); i++) soma += notas[i];
    if (!notas.empty()) {
        cout << notas.size() << " notas, média "
             << soma / notas.size() << endl;
    }
    return 0;
}
// Entrada: 7 4.5 9 6 -1  ->  4 notas, média 6.625
```

Com `vector`, não é preciso definir uma capacidade máxima nem guardar a quantidade em outra variável: `push_back` acrescenta um elemento e `size()` informa quantos há. O tipo `size_t`, um inteiro sem sinal, é o tipo de `size()`, e usá-lo no contador evita um aviso do compilador. O acesso com `notas[i]` continua sem conferência, mas `notas.at(i)` confere o índice e interrompe o programa com uma mensagem de erro se ele não existir. Nos capítulos seguintes, sempre que o tamanho não for conhecido de antemão, o `vector` será a escolha natural.

Erros Comuns

- **Começar ou terminar no índice errado.** Em C++, os índices vão de 0 a `n - 1`. Percorra com `for (int i = 0; i < n; i++)`.
- **Acessar fora dos limites.** `v[n]`, `v[-1]` ou um índice vindo do usuário sem conferência levam a comportamento indefinido.
- **Esquecer de inicializar.** Um vetor de contadores precisa começar zerado: `int votos[6] = {};`.
- **Usar mais elementos do que a capacidade.** Confira se a quantidade lida cabe no vetor antes do laço de leitura.
- **Achar que a função recebe uma cópia do vetor.** Ela trabalha no original. Use `const` quando a função só deve ler.
- **Esquecer de passar o tamanho para a função.** Dentro dela, não há como descobri-lo.
- **Trocar linha e coluna.** Em `m[i][j]`, o primeiro índice é a linha. Para somar uma linha, fixe `i`; para somar uma coluna, fixe `j`.
- **Escrever `m[i, j]` em C++.** Essa é a sintaxe do Portugol. Em C++, são dois pares de colchetes: `m[i][j]`.

Capítulo 8 – Caracteres e strings

Boa parte dos dados que um programa manipula é texto: nomes, endereços, e-mails, senhas, mensagens, o conteúdo de um documento. Até aqui, os textos só apareceram como mensagens fixas no `cout`. Neste capítulo, eles passam a ser dados de verdade: você vai ver como o computador representa cada caractere como um número, como guardar e ler textos com o tipo `string`, como percorrê-los letra por letra, como procurar, recortar e substituir pedaços, e por que as letras acentuadas do português exigem um cuidado especial. No fim, tudo se junta em um programa que cifra mensagens.

8.1 Caracteres são números

O tipo `char` guarda **um** caractere, escrito entre **aspas simples**: `'A'`, `'7'`, `' '`. Por dentro, o computador não guarda letras, só números: cada caractere tem um código, definido por uma tabela. A tabela básica, usada por praticamente todos os computadores, é a **ASCII**, que numera de 0 a 127 as letras sem acento, os dígitos, a pontuação e alguns caracteres de controle, como a quebra de linha.

Caracteres	Códigos	Exemplo
espaço	32	`' '` = 32
dígitos `'0'` a `'9'`	48 a 57	`'7'` = 55
maiúsculas `'A'` a `'Z'`	65 a 90	`'C'` = 67
minúsculas `'a'` a `'z'`	97 a 122	`'c'` = 99

Não é preciso decorar os números, mas vale entender a organização: os dígitos, as maiúsculas e as minúsculas vêm **em sequência**, e cada minúscula fica exatamente 32 posições depois da maiúscula correspondente. Como um `char` é um número pequeno, dá para fazer contas com ele:

```
#include <iostream>
using namespace std;

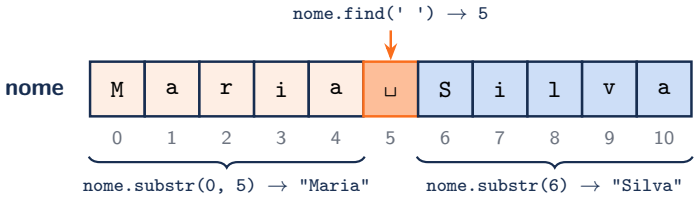
int main() {
    char letra = 'A';
    cout << letra << " tem código " << int(letra) << endl;  // 65
    char proxima = letra + 1;
    cout << proxima << endl;                // B
    char digito = '7';
    int valor = digito - '0';               // 55 - 48 = 7
    cout << valor * 2 << endl;              // 14
    cout << char('g' - 'a' + 'A') << endl;  // G
    return 0;
}
```

Dois truques aparecem aí e vão ser usados no capítulo inteiro. `digito - '0'` transforma o caractere de um dígito no número que ele representa, porque os dígitos são consecutivos a partir do `'0'`. E `c - 'a'` dá a posição de uma minúscula no alfabeto (`'a'` é 0, `'b'` é 1...), que somada a `'A'` dá a maiúscula correspondente.

No Portugol do VisuAlg não existe um tipo para um único caractere: o tipo `caractere` guarda textos de qualquer tamanho. As funções `asc("A")`, que devolve 65, e `carac(65)`, que devolve `"A"`, fazem a conversão entre caracteres e códigos.

8.2 O tipo string

Uma **string** é uma sequência de caracteres. Em C++, ela é o tipo `string`, da biblioteca `<string>`, e seus valores literais são escritos entre **aspas duplas**. Uma string se parece muito com o `vector` do Capítulo 7: cada caractere tem um índice, a partir de 0, e o tamanho é dado por `size()` (ou pelo sinônimo `length()`).



`nome.size()` vale 11: os índices vão de 0 a 10

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nome = "Ada";
    string sobrenome = "Lovelace";
    string completo = nome + " " + sobrenome;   // concatenação
    cout << completo << endl;                   // Ada Lovelace
    cout << completo.size() << " caracteres" << endl;   // 12
    cout << completo[0] << completo[4] << endl; // AL
    completo[0] = 'a';                          // pode alterar
    cout << completo << endl;                   // ada Lovelace
    completo += "!";                            // acrescenta
    cout << completo << endl;                   // ada Lovelace!
    return 0;
}
```

O operador `+` junta (**concatena**) duas strings, e `+=` acrescenta no fim. As strings também podem ser comparadas com `==`, `!=`, `<` e os demais operadores relacionais. A comparação é feita caractere a caractere, pelos códigos, como na ordem de um dicionário: `"casa" < "caso"`, porque na primeira diferença `'a'` vem antes de `'o'`. Mas, como a ordem é a dos códigos, toda maiúscula vem antes de qualquer minúscula, e `"Zebra" < "abacate"` é verdadeiro. Para comparar sem diferenciar maiúsculas de minúsculas, converta as duas strings para a mesma forma antes (Seção 8.4).

No Portugol, o tipo é `caractere`, a concatenação também é feita com `+`, e as operações são funções:

```
algoritmo "Textos"
var
   nome: caractere
inicio
```

```
    escreva("Nome: ")
    leia(nome)
    escreval("Tamanho: ", compr(nome))
    escreval("Primeira letra: ", copia(nome, 1, 1))
    escreval("Em maiúsculas: ", maiusc(nome))
    escreval("Olá, " + nome + "!")
fimalgoritmo
```

No Portugol, as posições começam em 1: `copia(nome, 1, 1)` copia, a partir da posição 1, um caractere.

8.3 Lendo textos

O `cin >>` lê uma **palavra**: ele para no primeiro espaço. Com a entrada `Ada Lovelace`, `cin >> nome` guarda só `"Ada"`, e o `"Lovelace"` fica esperando a próxima leitura. Para ler uma linha inteira, com espaços, use `getline(cin, nome)`.

Misturar os dois exige um cuidado. Quando o usuário digita a idade e tecla Enter, o `cin >> idade` lê o número e deixa o Enter para trás. Um `getline` logo depois encontra esse Enter e devolve uma linha vazia, sem esperar o usuário. A solução é descartar os espaços e quebras de linha pendentes com `ws` antes do `getline`:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int idade;
    string nome;
    cout << "Idade: ";
    cin >> idade;
    cout << "Nome completo: ";
    getline(cin >> ws, nome);    // ws descarta o Enter que sobrou
    cout << nome << ", " << idade << " anos" << endl;
    return 0;
}
// Entrada: 36, depois Ada Lovelace  ->  Ada Lovelace, 36 anos
```

8.4 Percorrendo uma string

Como uma string é uma sequência, um laço pode visitar cada caractere. Além do `for` com índice, o C++ tem uma forma mais curta para percorrer uma sequência inteira, o **for de intervalo**: `for (char c : texto)` executa o bloco uma vez para cada caractere, guardado em `c`. Se a variável for uma referência, `for (char &c : texto)`, alterar `c` altera a própria string, exatamente como na passagem por referência do Capítulo 6.

A biblioteca `<cctype>` tem funções para classificar e converter caracteres: `isalpha` (é letra?), `isdigit` (é dígito?), `isspace` (é espaço?), `isalnum` (é letra ou dígito?), `toupper` e `tolower` (convertem para maiúscula e minúscula). O programa abaixo usa várias delas:

```
#include <iostream>
#include <string>
#include <cctype>      // isdigit, isspace, toupper, tolower...
using namespace std;

bool ehVogal(unsigned char c) {
    c = tolower(c);
    return c == 'a' || c == 'e' || c == 'i' || c == 'o'
        || c == 'u';
}

int main() {
    string frase;
    cout << "Frase: ";
    getline(cin, frase);
    int vogais = 0, digitos = 0, espacos = 0;
    for (unsigned char c : frase) {         // cada caractere
        if (ehVogal(c)) vogais++;
        else if (isdigit(c)) digitos++;
        else if (isspace(c)) espacos++;
    }
    cout << vogais << " vogais, " << digitos << " dígitos, "
         << espacos << " espaços" << endl;
    for (char &c : frase) {                 // & altera a frase
        c = toupper((unsigned char) c);
    }
    cout << frase << endl;
    return 0;
}
// Entrada: Turma 2026 da noite
// Saída:   6 vogais, 4 dígitos, 3 espaços
//          TURMA 2026 DA NOITE
```

O `unsigned char` parece um detalhe estranho, mas tem motivo: as funções de `<cctype>` exigem um código entre 0 e 255, e um `char` comum pode ser negativo para os caracteres fora da tabela ASCII, como os acentuados (Seção 8.7). Converter para `unsigned char` antes de chamá-las evita o problema.

8.5 Procurando, recortando e convertendo

A classe `string` oferece operações para trabalhar com pedaços do texto. As duas mais usadas andam juntas: `find` procura um caractere ou um trecho e devolve a posição da primeira ocorrência, e `substr(p, n)` copia `n` caracteres a partir da posição `p` (sem o `n`, copia até o fim). A figura da Seção 8.2 mostra as duas em ação sobre `"Maria Silva"`.

Quando o `find` não encontra nada, ele devolve o valor especial `string::npos`. Esse valor precisa **sempre** ser testado antes de usar a posição, porque ele é um número enorme, e usá-lo como índice leva a erro.

```
#include <iostream>
#include <string>
```

```
using namespace std;

int main() {
    string nome = "Maria Silva";
    size_t espaco = nome.find(' ');               // 5
    if (espaco != string::npos) {
        cout << nome.substr(0, espaco) << endl;   // Maria
        cout << nome.substr(espaco + 1) << endl;  // Silva
    }

    string data = "25/12/2026";
    int dia = stoi(data.substr(0, 2));            // texto -> int
    int mes = stoi(data.substr(3, 2));
    int ano = stoi(data.substr(6));
    cout << dia + mes + ano << endl;              // 2063

    string aviso = "Nota final: X";
    aviso.replace(aviso.find('X'), 1, to_string(9));
    cout << aviso << endl;                        // Nota final: 9
    return 0;
}
```

As funções `stoi` (*string to int*) e `stod` (*string to double*) convertem um texto em número, e `to_string` faz o caminho contrário. A soma `dia + mes + ano` só está ali para mostrar que, depois do `stoi`, os pedaços da data são números de verdade. A tabela resume as operações mais úteis, com os equivalentes do Portugol:

C++	O que faz	Portugol
`s.size()`	tamanho	`compr(s)`
`s + t`	concatena	`s + t`
`s.substr(p, n)`	`n` caracteres a partir de `p`	`copia(s, p, n)`
`s.find(t)`	posição de `t` em `s`	`pos(t, s)`
`s.insert(p, t)`	insere `t` na posição `p`	–
`s.erase(p, n)`	apaga `n` caracteres	–
`s.replace(p, n, t)`	troca `n` caracteres por `t`	–
`toupper(c)`	maiúscula (um caractere)	`maiusc(s)`
`stoi(s)`, `stod(s)`	texto para número	`caracpnum(s)`
`to_string(x)`	número para texto	`numpcarac(x)`

Lembre que no Portugol as posições começam em 1, e que `pos` devolve 0 quando não encontra o trecho. O `maiusc` do Portugol converte a string inteira, enquanto o `toupper` do C++ converte um caractere por vez.

8.6 Algoritmos com strings: palíndromos

Um **palíndromo** é um texto que se lê igual nos dois sentidos, como “radar” ou “Roma me tem amor”. Para frases, a convenção é ignorar espaços, pontuação e a diferença entre maiúsculas e minúsculas. O algoritmo tem duas etapas: primeiro, monta uma versão “limpa” do texto, só com letras e dígitos em minúsculas; depois, compara o primeiro caractere com o último, o segundo com o penúltimo, e assim por diante, com os dois índices que caminham um em direção ao outro, como na inversão de vetores do Capítulo 7.

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

bool ehPalindromo(const string &texto) {
    string limpo;
    for (unsigned char c : texto) {
        if (isalnum(c)) limpo += char(tolower(c));
    }
    int i = 0, j = int(limpo.size()) - 1;
    while (i < j) {
        if (limpo[i] != limpo[j]) return false;  // diferença
        i++;
        j--;
    }
    return true;
}

int main() {
    string frases[] = {"radar", "Roma me tem amor",
                       "A base do teto desaba", "algoritmo"};
    for (const string &f : frases) {
        cout << f << ": " << (ehPalindromo(f) ? "sim" : "não")
             << endl;
    }
    return 0;
}
// radar: sim   Roma me tem amor: sim
// A base do teto desaba: sim   algoritmo: não
```

O `return false` encerra a busca na primeira diferença: não adianta continuar comparando. A função recebe o texto por referência constante (Capítulo 6), porque só precisa lê-lo. E o `for` de intervalo também funciona com vetores, como o `frases` do `main`.

8.7 Acentos e a codificação UTF-8

A tabela ASCII não tem letras acentuadas. Para representar os caracteres de todas as línguas, o padrão atual é o **Unicode**, e a forma mais comum de gravá-lo é a codificação **UTF-8**. Nela, os caracteres da tabela ASCII continuam ocupando um byte, mas os demais ocupam dois ou mais. Como o `string` do C++ é uma sequência de bytes, e não de letras, isso tem consequências para quem escreve em português:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string palavra = "ação";
    cout << palavra.size() << endl;   // 6, e não 4
```

```
    return 0;                          // (ç e ã têm 2 bytes)
}
```

Por isso, `palavra[1]` não é o `'ç'`, mas só o primeiro dos seus dois bytes, e `toupper` não transforma `'ã'` em `'Ã'`. Contar, inverter ou converter letra por letra um texto acentuado exige bibliotecas próprias para Unicode, que estão além desta disciplina. Os exemplos deste capítulo que processam caractere por caractere supõem textos sem acentos. Guardar, ler, mostrar, concatenar e comparar com == textos acentuados, por outro lado, funciona normalmente.

8.8 Strings no estilo C

Antes do tipo `string`, a linguagem C representava um texto como um **vetor de char** terminado pelo caractere especial `'\0'` (o caractere de código zero), que marca onde o texto acaba. Esse formato ainda aparece em código antigo, em bibliotecas e nos textos entre aspas duplas, que no C++ são, na verdade, vetores de `char`.

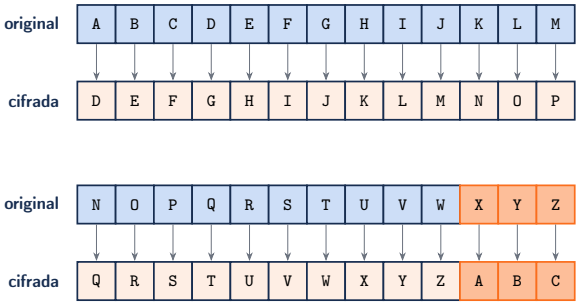
```
#include <iostream>
#include <cstring>     // strlen, strcmp, strcpy
using namespace std;

int main() {
    char nome[10] = "Ana";     // 'A' 'n' 'a' '\0' e zeros
    cout << nome << " tem " << strlen(nome) << " letras" << endl;
    cout << (strcmp(nome, "Ana") == 0) << endl;   // 1: iguais
    return 0;
}
```

As strings do estilo C herdam todos os riscos dos vetores: têm capacidade fixa, não podem ser comparadas com == (que compara endereços, e não o conteúdo; para isso existe `strcmp`) nem atribuídas com =, e é fácil escrever além do fim. Em programas novos, prefira sempre o `string`.

8.9 Um programa completo: a cifra de César

Um dos métodos de criptografia mais antigos que se conhecem é atribuído a Júlio César, que o usava em mensagens militares: cada letra é trocada pela letra que fica um número fixo de posições adiante no alfabeto, a **chave**. Com chave 3, o A vira D, o B vira E e, ao chegar ao fim, o alfabeto recomeça: o X vira A.



chave 3: cada letra vira a que está 3 posições adiante;
no fim, o alfabeto recomeça (X → A, Y → B, Z → C)

A conta usa os dois truques da Seção 8.1. A posição da letra no alfabeto é `c - 'a'`; somando a chave e tirando o resto da divisão por 26, a posição "dá a volta" no fim do alfabeto; e somar `'a'` de novo transforma a posição em letra. Para decifrar, basta cifrar de novo com a chave `26 - k`, que completa a volta.

```
#include <iostream>
#include <string>
using namespace std;

char deslocar(char c, int k) {
    k = (k % 26 + 26) % 26;          // chave entre 0 e 25
    if (c >= 'a' && c <= 'z') return 'a' + (c - 'a' + k) % 26;
    if (c >= 'A' && c <= 'Z') return 'A' + (c - 'A' + k) % 26;
    return c;                        // o resto não muda
}

string cifrar(const string &texto, int k) {
    string resultado = texto;
    for (char &c : resultado) c = deslocar(c, k);
    return resultado;
}

int main() {
    int chave;
    string mensagem;
    cout << "Chave: ";
    cin >> chave;
    cout << "Mensagem: ";
    getline(cin >> ws, mensagem);
    string secreta = cifrar(mensagem, chave);
    cout << "Cifrada:   " << secreta << endl;
    cout << "Decifrada: " << cifrar(secreta, 26 - chave) << endl;
    return 0;
}
// Chave 3, "Ataque ao amanhecer"
// Cifrada:   Dwdtxh dr dpdqkhfhu
// Decifrada: Ataque ao amanhecer
```

A primeira linha de `deslocar` traz qualquer chave, inclusive negativa ou maior que 26, para a faixa de 0 a 25. Sem ela, uma chave negativa daria um resto negativo e um caractere fora do alfabeto. Os caracteres que não são letras, como espaços e pontuação, passam sem alteração.

A cifra de César é ótima para praticar strings, mas péssima para proteger segredos: como só existem 25 chaves possíveis, basta testar todas para ler a mensagem. A criptografia usada hoje, nos sites e aplicativos, se baseia em matemática bem mais sofisticada.

Erros Comuns

- **Confundir aspas simples e duplas.** `'A'` é um `char`; `"A"` é um texto. `'Ana'` não é válido.
- **Usar `getline` logo depois de `cin >>`.** O Enter que sobrou é lido como uma linha vazia. Use `getline(cin >> ws, s)`.
- **Ler um nome com espaços usando `cin >>`.** Só a primeira palavra é lida.
- **Usar a posição do `find` sem testar `npos`.** Quando o trecho não existe, a posição é um número enorme e inválido.
- **Acessar `s[i]` fora do tamanho.** Como nos vetores, `s[s.size()]` não é um caractere válido do texto.
- **Esquecer que a comparação diferencia maiúsculas.** `"ana" == "Ana"` é falso, e `"Zebra" < "abacate"` é verdadeiro.
- **Confundir o caractere de um dígito com o número.** `'7'` vale 55; o número 7 é `'7' - '0'`.
- **Processar letra por letra um texto com acentos.** Em UTF-8, uma letra acentuada ocupa mais de um byte.
- **Comparar strings do estilo C com `==`.** Use `strcmp`, ou melhor, use `string`.

Capítulo 9 – Recursão e backtracking

No Capítulo 6, as funções chamavam outras funções. Nada impede que uma função chame **a si mesma**, e essa ideia, a **recursão**, é uma das mais poderosas da programação. Muitos problemas são naturalmente definidos em termos de versões menores deles mesmos: o fatorial de 5 é 5 vezes o fatorial de 4; uma pasta contém arquivos e outras pastas; para mover uma pilha de discos, move-se uma pilha menor. Neste capítulo você vai aprender a escrever funções recursivas, a entender o que acontece na pilha de chamadas a cada uma delas, a reconhecer quando a recursão é uma boa escolha e quando é um desperdício, e a usá-la na técnica de **backtracking**, que explora sistematicamente todas as possibilidades de um problema.

9.1 Uma função que chama a si mesma

O fatorial de um número n, escrito $n!$, é o produto de todos os inteiros de 1 até n: $4! = 4 \times 3 \times 2 \times 1 = 24$. No Capítulo 5, ele seria calculado com um laço. Mas repare que $4! = 4 \times 3!$, e em geral $n! = n \times (n-1)!$. Essa é uma **definição recursiva**: o fatorial é definido usando o próprio fatorial, de um número menor. Para que ela não continue para sempre, é preciso um ponto de parada: $1! = 1$ (e, por convenção, $0! = 1$).

Toda função recursiva tem essas duas partes:

- o **caso base**, um caso simples o bastante para ser respondido diretamente, sem recursão;
- o **caso recursivo**, que resolve o problema usando a resposta de um problema **menor** do mesmo tipo, cada vez mais perto do caso base.

```
#include <iostream>
using namespace std;

long long fatorial(int n) {
    if (n <= 1) return 1;              // caso base
    return n * fatorial(n - 1);        // caso recursivo
}

int main() {
    int n;
    cout << "n (0 a 20): ";
    cin >> n;
    if (n < 0 || n > 20) {
        cout << "Fora da faixa." << endl;   // 21! não cabe
        return 0;
    }
    cout << n << "! = " << fatorial(n) << endl;
    return 0;
}
// 4 -> 4! = 24        20 -> 20! = 2432902008176640000
```

O tipo `long long` guarda inteiros de até cerca de $9{,}2 \times 10^{18}$. Como o fatorial cresce muito depressa, 20! é o maior que cabe nele, e por isso o programa recusa valores maiores.

O Portugol também aceita funções recursivas, com a mesma estrutura:

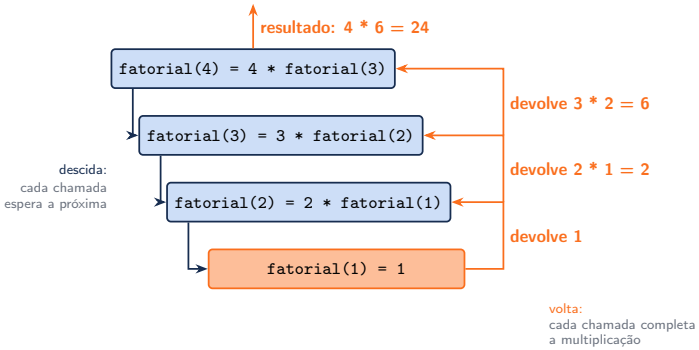
```
algoritmo "FatorialRecursivo"
var
   n: inteiro

funcao fatorial(k: inteiro): inteiro
inicio
   se k <= 1 entao
      retorne 1
   senao
      retorne k * fatorial(k - 1)
   fimse
fimfuncao

inicio
   escreva("n: ")
   leia(n)
   escreval(n, "! = ", fatorial(n))
fimalgoritmo
```

9.2 O que acontece na pilha de chamadas

Uma chamada recursiva funciona exatamente como qualquer outra chamada (Seção 6.4): ela ganha seu próprio registro de ativação na pilha, com seu próprio `n`. A chamada `fatorial(4)` não termina enquanto não recebe a resposta de `fatorial(3)`, que espera `fatorial(2)`, que espera `fatorial(1)`. Por um momento, há quatro registros empilhados, cada um com um `n` diferente. Só então as respostas voltam, na ordem inversa, e cada chamada completa sua multiplicação:



Esse "ir e voltar" tem duas consequências práticas. Primeiro, o trabalho de cada chamada se divide em duas partes: o que ela faz **antes** da chamada recursiva (na descida) e o que ela faz **depois** (na volta). No fatorial, a multiplicação acontece na volta. Segundo, a pilha tem tamanho limitado, normalmente de alguns megabytes. Uma recursão muito profunda, ou uma sem caso base, enche a pilha, e o programa é encerrado com um erro de **estouro de pilha** (*stack overflow*):

```
// erro: sem caso base, a recursão nunca termina
long long fatorial(int n) {
    return n * fatorial(n - 1);   // 4, 3, 2, 1, 0, -1, -2...
}
```

O mesmo acontece quando o caso base existe, mas a recursão nunca o alcança: se o teste fosse `if (n == 1)`, a chamada `fatorial(-3)` desceria para -4, $-5\ldots$ sem nunca passar pelo 1. É por isso que o programa usa `n <= 1`. Por isso a regra da Seção 9.1: cada chamada recursiva precisa ser com um problema **menor**, que caminhe na direção do caso base.

9.3 Pensando recursivamente

Para escrever uma função recursiva, não tente acompanhar mentalmente todas as chamadas. Faça três perguntas: qual é o caso mais simples, que eu sei responder direto? Como diminuo o problema? E, **supondo que a chamada recursiva já funciona** para o problema menor, como uso a resposta dela para resolver o problema inteiro? Esse ato de confiança é o coração do raciocínio recursivo. Veja como ele resolve quatro problemas diferentes:

```
#include <iostream>
#include <string>
using namespace std;

// b elevado a e: b^0 = 1; b^e = b * b^(e-1)
long long potencia(long long b, int e) {
    if (e == 0) return 1;
    return b * potencia(b, e - 1);
}

// soma dos n primeiros elementos: a soma dos n-1 primeiros
// mais o último
int soma(const int v[], int n) {
    if (n == 0) return 0;              // vetor vazio
    return soma(v, n - 1) + v[n - 1];
}

// inverter: o resto invertido, seguido da primeira letra
string inverter(const string &s) {
    if (s.size() <= 1) return s;
    return inverter(s.substr(1)) + s[0];
}

// máximo divisor comum, pelo algoritmo de Euclides
int mdc(int a, int b) {
    if (b == 0) return a;
    return mdc(b, a % b);
}

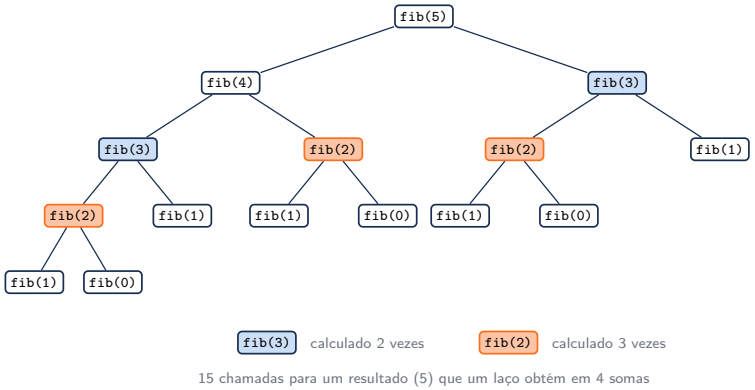
int main() {
    int v[] = {4, 8, 15, 16, 23, 42};
```

```
    cout << potencia(2, 10) << endl;       // 1024
    cout << soma(v, 6) << endl;            // 108
    cout << inverter("recursão") << endl;  // cuidado: acento!
    cout << inverter("pilha") << endl;     // ahlip
    cout << mdc(48, 18) << endl;           // 6
    return 0;
}
```

O algoritmo de Euclides, com mais de dois mil anos, usa um fato da aritmética: o MDC de `a` e `b` é igual ao MDC de `b` e do resto de `a` por `b`. A sequência de chamadas para 48 e 18 é `mdc(48, 18)`, `mdc(18, 12)`, `mdc(12, 6)` e `mdc(6, 0)`, que devolve 6. O `inverter("recursão")` foi incluído de propósito, para lembrar a Seção 8.7: ele inverte os bytes, e o `ã`, que ocupa dois, sai embaralhado.

9.4 Quando a recursão desperdiça: Fibonacci

A sequência de Fibonacci começa com 0 e 1, e cada termo seguinte é a soma dos dois anteriores: 0, 1, 1, 2, 3, 5, 8, 13, 21... A definição é recursiva, e a função sai quase igual a ela: `fib(n) = fib(n - 1) + fib(n - 2)`, com `fib(0) = 0` e `fib(1) = 1` como casos base. Mas há um problema escondido. Cada chamada faz **duas** chamadas recursivas, e elas repetem o mesmo trabalho:



15 chamadas para um resultado (5) que um laço obtém em 4 somas

Só para calcular `fib(5)`, o `fib(2)` é calculado três vezes. Para valores maiores, a repetição explode: o número de chamadas cresce quase tão depressa quanto a própria sequência. O programa abaixo conta as chamadas e compara com uma versão que guarda cada resultado já calculado em um vetor e o consulta antes de recalcular. Essa técnica se chama **memoização**.

```
#include <iostream>
#include <vector>
using namespace std;

long long fib(int n, long long &chamadas) {
    chamadas++;
```

```
    if (n <= 1) return n;
    return fib(n - 1, chamadas) + fib(n - 2, chamadas);
}

long long fibMemo(int n, vector<long long> &memo) {
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];        // já calculado
    memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
    return memo[n];
}

int main() {
    for (int n = 10; n <= 30; n += 10) {
        long long chamadas = 0;
        long long f = fib(n, chamadas);
        cout << "fib(" << n << ") = " << f << ", "
             << chamadas << " chamadas" << endl;
    }
    vector<long long> memo(91, 0);
    cout << "fib(90) = " << fibMemo(90, memo) << endl;
    return 0;
}
// fib(10) = 55, 177 chamadas
// fib(20) = 6765, 21891 chamadas
// fib(30) = 832040, 2692537 chamadas
// fib(90) = 2880067194370816120
```

A versão simples precisa de quase 2,7 milhões de chamadas para `fib(30)`, e de mais de 330 milhões para `fib(40)`. Para `fib(90)`, ela levaria séculos. A versão com memoização calcula cada termo uma única vez e chega a `fib(90)` com menos de 200 chamadas. Um simples laço, que vai somando os dois últimos termos, resolveria com a mesma rapidez. A lição: uma definição recursiva elegante não é, necessariamente, um bom algoritmo.

9.5 Recursão ou laço?

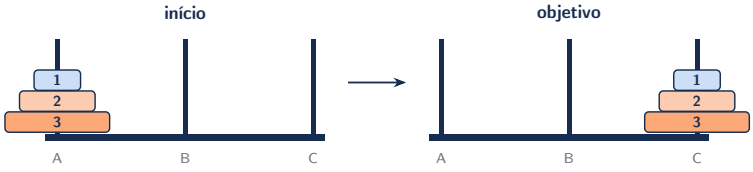
Todo laço pode ser escrito como recursão, e toda recursão pode ser escrita com laços (às vezes com a ajuda de uma pilha explícita). A escolha é sobre clareza e custo:

Prefira o laço quando	Prefira a recursão quando
o problema é uma sequência simples de passos, como somar, contar, fatorial	o problema se divide em subproblemas do mesmo tipo, como pastas, árvores, Torres de Hanói
a profundidade pode ser muito grande (milhares de níveis)	a profundidade é pequena, ou cresce devagar
a versão recursiva repete trabalho, como no Fibonacci	é preciso explorar possibilidades e voltar atrás (backtracking)

Cada chamada recursiva custa um pouco mais que uma volta de laço, porque precisa criar e destruir um registro na pilha. Nos exemplos simples da Seção 9.3, o laço seria um pouco mais eficiente. Nos problemas das próximas seções, a recursão é tão mais clara que a versão com laços quase ninguém escreveria.

9.6 As Torres de Hanói

O quebra-cabeça das Torres de Hanói tem três pinos e uma pilha de discos de tamanhos diferentes, todos no primeiro pino, do maior (embaixo) para o menor. O objetivo é levar a pilha inteira para o terceiro pino, movendo **um disco por vez** e **nunca** colocando um disco sobre outro menor.



um disco por vez; nunca um disco maior sobre um menor

Com laços, é difícil até começar. Com o raciocínio recursivo, a solução cabe em três frases. Para mover n discos de A para C, usando B como apoio:

1. mova os $n-1$ discos de cima de A para B (usando C como apoio);
2. mova o maior disco, que ficou sozinho, de A para C;
3. mova os $n-1$ discos de B para C (usando A como apoio).

Os passos 1 e 3 são o mesmo problema, com um disco a menos: é a "confiança" da Seção 9.3. O caso base é mover zero discos, que não exige nada.

```
#include <iostream>
using namespace std;

void hanoi(int n, char origem, char destino, char apoio,
           int &movimentos) {
    if (n == 0) return;                          // caso base
    hanoi(n - 1, origem, apoio, destino, movimentos);
    movimentos++;
    cout << movimentos << ": disco " << n << " de " << origem
         << " para " << destino << endl;
    hanoi(n - 1, apoio, destino, origem, movimentos);
}

int main() {
    int movimentos = 0;
    hanoi(3, 'A', 'C', 'B', movimentos);
    cout << movimentos << " movimentos" << endl;   // 7
    return 0;
}
// 1: disco 1 de A para C     5: disco 1 de B para A
// 2: disco 2 de A para B     6: disco 2 de B para C
// 3: disco 1 de C para B     7: disco 1 de A para C
// 4: disco 3 de A para C
```

Cada disco a mais dobra o trabalho e acrescenta um movimento: com n discos, são $2^n - 1$ movimentos. Com 3 discos, 7; com 10, 1.023; com 64 discos, como na lenda que

deu origem ao quebra-cabeça, mais de 18 quintilhões, o que levaria mais de 500 bilhões de anos a um movimento por segundo.

9.7 Backtracking: tentar, avançar e voltar atrás

Muitos problemas pedem que se encontre uma combinação de escolhas que satisfaça certas regras: preencher um sudoku, montar um horário sem conflitos, encontrar a saída de um labirinto. O **backtracking** ("voltar atrás") resolve esse tipo de problema fazendo uma escolha de cada vez:

1. se todas as escolhas já foram feitas, encontrou-se uma solução;
2. senão, para cada opção possível da próxima escolha: se ela não viola as regras, faça-a, continue recursivamente com as escolhas seguintes e, ao voltar, **desfaça** a escolha para tentar a próxima opção.

A recursão cuida de lembrar onde se estava: quando uma tentativa leva a um beco sem saída, a função simplesmente retorna, e a chamada anterior continua com sua próxima opção. Um exemplo simples é gerar todas as **permutações** de um texto, ou seja, todas as ordens possíveis de suas letras. A cada posição `k`, escolhe-se qual letra vai ocupá-la, trocando-a para lá; depois de permutar o resto, a troca é desfeita:

```
#include <iostream>
#include <string>
using namespace std;

void permutar(string &s, int k) {
    if (k == int(s.size())) {          // todas as posições feitas
        cout << s << " ";
        return;
    }
    for (int i = k; i < int(s.size()); i++) {
        swap(s[k], s[i]);              // põe s[i] na posição k
        permutar(s, k + 1);            // permuta o resto
        swap(s[k], s[i]);              // desfaz a escolha
    }
}

int main() {
    string s = "ABC";
    permutar(s, 0);
    cout << endl;                      // ABC ACB BAC BCA CBA CAB
    return 0;
}
```

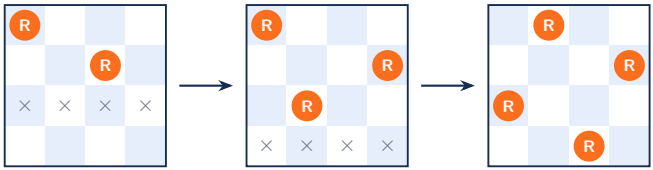
Sem o segundo `swap`, cada tentativa deixaria o texto alterado para a próxima, e o resultado sairia com repetições e faltas. Desfazer a escolha é o que permite que cada opção seja testada a partir do mesmo ponto de partida.

9.8 Um programa completo: as oito rainhas

O problema das oito rainhas é o exemplo clássico de backtracking: posicionar 8 rainhas em um tabuleiro de xadrez 8×8 de modo que nenhuma ataque outra. Como a rainha

ataca na horizontal, na vertical e nas diagonais, não pode haver duas rainhas na mesma linha, na mesma coluna ou na mesma diagonal.

Como cada linha terá exatamente uma rainha, a escolha a fazer, linha por linha, é a **coluna** da rainha daquela linha. O vetor `coluna` guarda essas escolhas: `coluna[l]` é a coluna da rainha da linha `l`. Duas rainhas estão na mesma diagonal quando a distância entre suas linhas é igual à distância entre suas colunas. A figura mostra o backtracking em ação em um tabuleiro 4×4:



linha 2 sem casa livre: volta e muda a linha 1

linha 3 sem casa livre: volta (até a linha 0)

primeira solução

```
#include <iostream>
#include <cstdlib>     // abs
using namespace std;

const int N = 8;

// a linha "linha" pode receber uma rainha na coluna "col"?
bool livre(const int coluna[], int linha, int col) {
    for (int l = 0; l < linha; l++) {
        if (coluna[l] == col) return false;     // mesma coluna
        if (abs(coluna[l] - col) == linha - l) { // mesma diagonal
            return false;
        }
    }
    return true;
}

void mostrar(const int coluna[]) {
    for (int l = 0; l < N; l++) {
        for (int c = 0; c < N; c++) {
            cout << (coluna[l] == c ? " R" : " .");
        }
        cout << endl;
    }
}

void posicionar(int coluna[], int linha, int &solucoes) {
    if (linha == N) {                   // as N rainhas estão lá
        solucoes++;
        if (solucoes == 1) mostrar(coluna); // só a primeira
        return;
    }
```

```
    for (int col = 0; col < N; col++) {
        if (livre(coluna, linha, col)) {
            coluna[linha] = col;            // escolhe e avança
            posicionar(coluna, linha + 1, solucoes);
        }       // a próxima volta do for sobrescreve a escolha
    }
}

int main() {
    int coluna[N];
    int solucoes = 0;
    posicionar(coluna, 0, solucoes);
    cout << solucoes << " soluções" << endl;   // 92 soluções
    return 0;
}
```

A primeira solução encontrada, mostrada pelo programa, é:

```
// saída
 R . . . . . . .
 . . . . R . . .
 . . . . . . . R
 . . . . . R . .
 . . R . . . . .
 . . . . . . R .
 . R . . . . . .
 . . . R . . . .
92 soluções
```

Aqui, desfazer a escolha não exige um comando explícito: a próxima volta do laço grava outra coluna em `coluna[linha]`, e a função `livre` só olha as linhas anteriores à atual. Em vez de testar as 8^8 (mais de 16 milhões) formas de pôr uma rainha por linha, o backtracking abandona cada tentativa assim que um conflito aparece, e a função `posicionar` é chamada apenas 2.057 vezes. Mudando `N`, o programa resolve tabuleiros de outros tamanhos: há 2 soluções para $N = 4$, 10 para $N = 5$ e 4 para $N = 6$.

Erros Comuns

- **Esquecer o caso base.** A recursão nunca para, e o programa termina com estouro de pilha.
- **Não diminuir o problema.** Cada chamada recursiva precisa se aproximar do caso base; `f(n)` chamando `f(n)` é um laço infinito disfarçado.
- **Esquecer o `return` da chamada recursiva.** Em uma função que devolve valor, escrever `fatorial(n - 1);` sem usar o resultado joga a resposta fora.
- **Usar recursão que repete trabalho.** Como no Fibonacci, duas chamadas que resolvem os mesmos subproblemas fazem o tempo explodir. Use memoização ou um laço.
- **Recursão profunda demais.** Uma soma recursiva de um vetor com um milhão de elementos cria um milhão de registros na pilha. Para sequências longas, use um laço.

- **Esquecer de desfazer a escolha no backtracking.** A tentativa seguinte começa de um estado errado.
- **Tentar acompanhar todas as chamadas de cabeça.** Confie que a chamada recursiva resolve o problema menor e verifique só o caso base e a combinação.

Capítulo 10 – Busca e ordenação

Procurar e ordenar estão entre as tarefas que os computadores mais executam. Toda vez que você procura um contato no celular, filtra produtos pelo menor preço ou vê a classificação de um campeonato, há um algoritmo de busca ou de ordenação trabalhando. Elas também são o lugar ideal para uma pergunta nova: não basta o algoritmo estar certo, ele precisa ser **rápido o suficiente**. Neste capítulo você vai conhecer as duas buscas fundamentais, a sequencial e a binária, três algoritmos clássicos de ordenação (seleção, bolha e inserção) e uma forma simples de comparar a eficiência de algoritmos contando quantas operações eles fazem.

10.1 Busca sequencial

A forma mais direta de procurar um valor em um vetor é olhar os elementos um por um, do primeiro ao último, até encontrá-lo ou até o vetor acabar. Essa é a **busca sequencial** (ou linear). A função devolve a **posição** em que o valor foi encontrado ou, se ele não estiver no vetor, um valor que não pode ser uma posição válida, como -1.

```
#include <iostream>
using namespace std;

int buscaSequencial(const int v[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (v[i] == x) return i;    // achou: devolve a posição
    }
    return -1;                      // olhou tudo e não achou
}

int main() {
    int v[] = {42, 7, 19, 73, 4, 56, 23};
    int x;
    cout << "Procurar: ";
    cin >> x;
    int pos = buscaSequencial(v, 7, x);
    if (pos >= 0) cout << "Achou na posição " << pos << endl;
    else cout << "Não encontrado" << endl;
    return 0;
}
// 73 -> Achou na posição 3       10 -> Não encontrado
```

Em Portugol, sem um `return` no meio do laço, a busca usa uma variável que guarda a posição encontrada e faz parte da condição do `enquanto`: o laço para quando acha o valor ou quando os elementos acabam.

```
algoritmo "BuscaSequencial"
var
   v: vetor[1..7] de inteiro
   i, x, pos: inteiro
inicio
   para i de 1 ate 7 faca
      escreva("Elemento ", i, ": ")
```

```
      leia(v[i])
   fimpara
   escreva("Procurar: ")
   leia(x)
   pos <- 0                    // 0: ainda não encontrado
   i <- 1
   enquanto (i <= 7) e (pos = 0) faca
      se v[i] = x entao
         pos <- i
      fimse
      i <- i + 1
   fimenquanto
   se pos > 0 entao
      escreval("Achou na posição ", pos)
   senao
      escreval("Não encontrado")
   fimse
fimalgoritmo
```

A busca sequencial funciona com qualquer vetor, ordenado ou não. O preço é que, no pior caso (o valor está no fim ou não está no vetor), ela olha **todos** os n elementos. Em um vetor com um milhão de elementos, isso são um milhão de comparações.

10.2 Busca binária

Ninguém procura uma palavra no dicionário lendo da primeira página em diante. Abre-se o dicionário mais ou menos no meio e, conforme a palavra vem antes ou depois da página aberta, descarta-se a metade em que ela não pode estar. A **busca binária** faz exatamente isso, e por isso só funciona em vetores **ordenados**.

O algoritmo mantém dois índices, `ini` e `fim`, que delimitam a parte do vetor onde o valor ainda pode estar. A cada passo, compara o valor procurado com o elemento do meio dessa parte: se for igual, achou; se o procurado for maior, ele só pode estar à direita do meio; se for menor, só à esquerda. Quando `ini` passa de `fim`, a parte ficou vazia, e o valor não está no vetor.



```
#include <iostream>
using namespace std;

int buscaBinaria(const int v[], int n, int x) {
    int ini = 0, fim = n - 1;
    while (ini <= fim) {
        int meio = ini + (fim - ini) / 2;
        if (v[meio] == x) return meio;
        if (v[meio] < x) ini = meio + 1;   // está à direita
        else fim = meio - 1;               // está à esquerda
    }
    return -1;                             // a faixa ficou vazia
}

int main() {
    int v[] = {3, 8, 11, 15, 19, 23, 27, 32, 36, 41, 47, 52, 58,
               63, 70};
    cout << buscaBinaria(v, 15, 41) << endl;   // 9
    cout << buscaBinaria(v, 15, 3) << endl;    // 0
    cout << buscaBinaria(v, 15, 40) << endl;   // -1
    return 0;
}
```

O meio é calculado como `ini + (fim - ini) / 2`, e não como `(ini + fim) / 2`. As duas contas dão o mesmo resultado, mas a segunda pode estourar o limite do `int` quando os índices são muito grandes, um erro que passou anos despercebido em bibliotecas usadas no mundo inteiro.

A cada passo, a busca binária descarta metade do que resta. Um vetor de 15 elementos precisa de no máximo 4 comparações; um de mil, de 10; um de um milhão, de apenas 20, porque 20 divisões ao meio reduzem um milhão a um único elemento. Em termos matemáticos, o número de passos é cerca de $\log_2 n$.

A busca binária também tem uma versão recursiva natural (Capítulo 9): procurar em uma faixa é olhar o meio e procurar em uma das metades.

```
int buscaRec(const int v[], int ini, int fim, int x) {
    if (ini > fim) return -1;              // caso base: vazia
    int meio = ini + (fim - ini) / 2;
    if (v[meio] == x) return meio;
    if (v[meio] < x) return buscaRec(v, meio + 1, fim, x);
    return buscaRec(v, ini, meio - 1, x);
}
```

10.3 Quanto custa um algoritmo

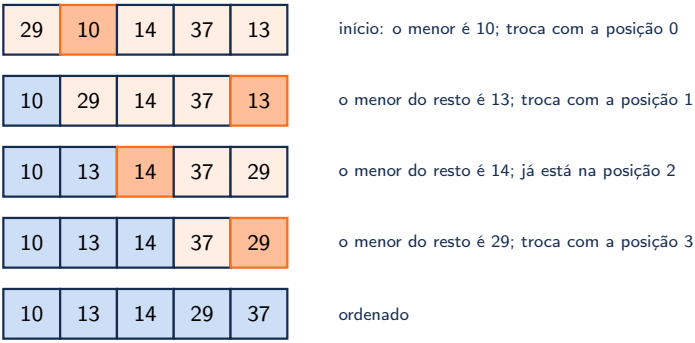
Medir o tempo de um programa com um cronômetro diz pouco, porque o resultado depende do computador, da linguagem e até do que mais estava rodando. Uma medida mais útil é **contar as operações** básicas que o algoritmo faz, como comparações, em função do tamanho n da entrada, e observar como esse número cresce quando n cresce. Essa é a ideia de **complexidade** de um algoritmo:

n	sequencial (n)	binária ($\log_2 n$)	ordenação simples (n^2)	sort ($n \log_2 n$)
10	10	4	100	33
1.000	1.000	10	1 milhão	10 mil
1 milhão	1 milhão	20	1 trilhão	20 milhões

A tabela mostra o número aproximado de operações. Os valores exatos importam menos que a forma do crescimento. Quando n fica mil vezes maior, a busca sequencial fica mil vezes mais lenta, enquanto a binária precisa de só 10 passos a mais. Já os algoritmos de ordenação simples deste capítulo, que crescem com n^2, ficam um **milhão** de vezes mais lentos. A um bilhão de operações por segundo, ordenar um milhão de números leva 20 milissegundos com um algoritmo $n \log n$, e mais de 15 minutos com um algoritmo n^2. Os cientistas da computação escrevem essas formas de crescimento com a notação O ("ó grande"): $O(n)$, $O(\log n)$, $O(n^2)$, $O(n \log n)$.

10.4 Ordenação por seleção

A **ordenação por seleção** é a forma como muita gente ordena cartas na mão: encontra a menor e a põe em primeiro lugar; entre as que sobraram, encontra a menor e a põe em segundo; e assim por diante. Cada passada pelo vetor coloca um elemento na sua posição definitiva.



azul: posições definitivas; laranja: o menor da parte que falta

```
void selecao(int v[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int iMenor = i;                         // procura o menor
        for (int j = i + 1; j < n; j++) {       // no resto
            if (v[j] < v[iMenor]) iMenor = j;
        }
        swap(v[i], v[iMenor]);                  // e o põe em i
    }
}
```

O padrão do menor elemento é o do Capítulo 7, aplicado à parte do vetor que ainda não está ordenada. Em Portugol, com o vetor de 1 a `n` e a troca feita por uma variável auxiliar:

```
algoritmo "OrdenacaoPorSelecao"
var
   v: vetor[1..5] de inteiro
   i, j, iMenor, aux: inteiro
inicio
   para i de 1 ate 5 faca
      leia(v[i])
   fimpara
   para i de 1 ate 4 faca
      iMenor <- i
      para j de i + 1 ate 5 faca
         se v[j] < v[iMenor] entao
            iMenor <- j
         fimse
      fimpara
      aux <- v[i]
      v[i] <- v[iMenor]
      v[iMenor] <- aux
   fimpara
   para i de 1 ate 5 faca
      escreva(v[i], " ")
   fimpara
fimalgoritmo
```

A primeira passada faz $n-1$ comparações, a segunda $n-2$, e assim por diante, num total de $n(n-1)/2$: para mil elementos, quase meio milhão. Esse número é sempre o mesmo, esteja o vetor bagunçado ou já ordenado. Em compensação, a seleção faz no máximo $n-1$ trocas, o que é uma vantagem quando trocar elementos é caro.

10.5 Ordenação por bolha

A **ordenação por bolha** (*bubble sort*) compara cada par de elementos vizinhos e os troca quando estão fora de ordem. Ao fim da primeira passada, o maior elemento "borbulhou" até a última posição; na segunda, o segundo maior chega à penúltima, e assim por diante. Se uma passada inteira termina sem nenhuma troca, o vetor já está ordenado e o algoritmo pode parar.

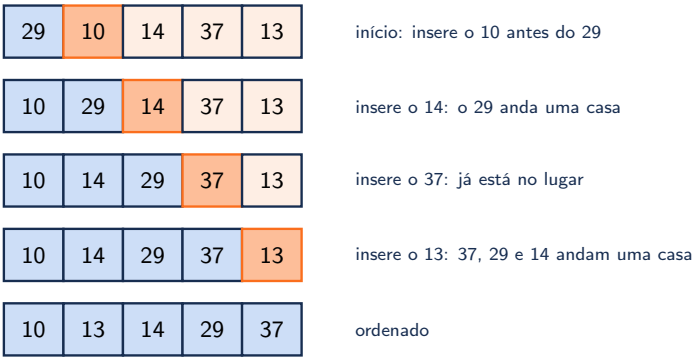
```
void bolha(int v[], int n) {
    for (int fim = n - 1; fim > 0; fim--) {
        bool trocou = false;
        for (int j = 0; j < fim; j++) {     // até o fim atual
            if (v[j] > v[j + 1]) {          // fora de ordem
                swap(v[j], v[j + 1]);
                trocou = true;
            }
        }
        if (!trocou) break;                 // já está ordenado
    }
}
```

Graças ao `trocou`, um vetor que já está ordenado é verificado com uma única passada, de $n-1$ comparações. Em um vetor bagunçado, porém, a bolha faz tantas comparações

quanto a seleção e muito mais trocas, e costuma ser o mais lento dos três algoritmos deste capítulo. Ela é estudada por ser simples, não por ser boa.

10.6 Ordenação por inserção

A **ordenação por inserção** imita outra forma de ordenar cartas: recebe-se uma carta de cada vez e insere-se cada uma no lugar certo entre as que já estão na mão, que permanecem ordenadas. No vetor, a parte da esquerda é a "mão" ordenada. A cada passo, o próximo elemento é guardado em `atual`, os maiores que ele são deslocados uma posição para a direita, abrindo espaço, e `atual` entra no buraco aberto.



azul: a parte já ordenada (a "mão"); laranja: o elemento a inserir

```
void insercao(int v[], int n) {
    for (int i = 1; i < n; i++) {
        int atual = v[i];                   // a carta da vez
        int j = i - 1;
        while (j >= 0 && v[j] > atual) {    // os maiores andam
            v[j + 1] = v[j];                // uma casa à direita
            j--;
        }
        v[j + 1] = atual;                   // entra no buraco
    }
}
```

A ordem das condições no `while` importa: `j >= 0` vem primeiro para que, quando `j` chegar a -1, o `&&` pare antes de acessar `v[-1]` (avaliação em curto-circuito, Seção 3.6).

A inserção tem uma qualidade especial: o laço de dentro para assim que encontra um elemento menor que `atual`. Em um vetor quase ordenado, cada elemento anda pouco, e o algoritmo fica muito rápido, perto de n comparações. Em um vetor bagunçado, faz em média metade das comparações da seleção. Por isso, entre os algoritmos simples, ela costuma ser a melhor escolha, e é usada até dentro de algoritmos sofisticados para ordenar trechos pequenos.

10.7 Comparando os algoritmos e usando a biblioteca

O programa abaixo põe as três ordenações para trabalhar sobre os mesmos dados e conta as comparações de cada uma. Os vetores são sorteados com o gerador de números aleatórios da biblioteca `<random>`, e cada função recebe um `vector` **por valor**, ou seja, uma cópia, para que todas ordenem os mesmos números.

```
#include <iostream>
#include <vector>
#include <random>
#include <algorithm>     // sort
#include <iomanip>       // setw
#include <string>
using namespace std;

long long selecao(vector<int> v) {
    long long comp = 0;
    int n = v.size();
    for (int i = 0; i < n - 1; i++) {
        int iMenor = i;
        for (int j = i + 1; j < n; j++) {
            comp++;
            if (v[j] < v[iMenor]) iMenor = j;
        }
        swap(v[i], v[iMenor]);
    }
    return comp;
}

long long bolha(vector<int> v) {
    long long comp = 0;
    int n = v.size();
    for (int fim = n - 1; fim > 0; fim--) {
        bool trocou = false;
        for (int j = 0; j < fim; j++) {
            comp++;
            if (v[j] > v[j + 1]) {
                swap(v[j], v[j + 1]);
                trocou = true;
            }
        }
        if (!trocou) break;
    }
    return comp;
}

long long insercao(vector<int> v) {
    long long comp = 0;
    int n = v.size();
    for (int i = 1; i < n; i++) {
        int atual = v[i];
        int j = i - 1;
        while (j >= 0) {
```

```
            comp++;
            if (v[j] <= atual) break;
            v[j + 1] = v[j];
            j--;
        }
        v[j + 1] = atual;
    }
    return comp;
}

int main() {
    mt19937 gerador(2026);                    // semente fixa
    uniform_int_distribution<int> sorteio(1, 1000000);
    cout << "    n  dados        seleção    bolha  inserção"
         << endl;
    for (int n = 1000; n <= 4000; n *= 2) {
        vector<int> v(n);
        for (int &x : v) x = sorteio(gerador);
        for (int vez = 1; vez <= 2; vez++) {
            string dados = vez == 1 ? "aleatório" : "ordenado ";
            cout << setw(5) << n << "  " << dados << "  "
                 << setw(9) << selecao(v) << setw(9) << bolha(v)
                 << setw(10) << insercao(v) << endl;
            sort(v.begin(), v.end());    // 2a vez: v ordenado
        }
    }
    return 0;
}
```

Compilado com o g++, o programa mostra a saída abaixo. Como a semente do gerador é fixa (2026), os números sorteados são sempre os mesmos; outro compilador pode sortear outros, e as contagens da bolha e da inserção mudam um pouco, mas o padrão é o mesmo.

```
// saída
    n  dados        seleção    bolha  inserção
 1000  aleatório     499500   496872    253165
 1000  ordenado      499500      999       999
 2000  aleatório    1999000  1996515   1030270
 2000  ordenado     1999000     1999      1999
 4000  aleatório    7998000  7990860   3918434
 4000  ordenado     7998000     3999      3999
```

Repare em três coisas. Quando n dobra, as comparações nos vetores aleatórios quadruplicam: é o crescimento n^2 da Seção 10.3. A inserção faz cerca de metade das comparações da seleção. E, no vetor já ordenado, a bolha e a inserção fazem só $n - 1$ comparações, enquanto a seleção continua fazendo todas.

O sort da biblioteca

Na prática, ninguém escreve a própria ordenação para usar em um programa: a biblioteca `<algorithm>` oferece a função `sort`, que usa um algoritmo $O(n \log n)$, muito mais

rápido que os deste capítulo para vetores grandes, e a `binary_search`, que faz a busca binária:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {42, 7, 19, 73, 4, 56, 23};
    sort(v.begin(), v.end());                    // crescente
    for (int x : v) cout << x << " ";
    cout << endl;                          // 4 7 19 23 42 56 73
    cout << binary_search(v.begin(), v.end(), 56) << endl;   // 1
    int w[] = {5, 2, 9, 1};
    sort(w, w + 4);                              // vetor comum
    cout << w[0] << " " << w[3] << endl;         // 1 9
    return 0;
}
```

Os algoritmos que fazem $O(n \log n)$ comparações, como a ordenação por intercalação (*merge sort*) e o *quicksort*, usam a ideia de dividir o vetor em partes, ordenar cada parte recursivamente e combinar os resultados. Eles ficam para um estudo mais avançado de estruturas de dados. Estudar os algoritmos simples continua valendo a pena: eles ensinam a raciocinar sobre laços, trocas e custo, e são a base para entender os mais sofisticados.

10.8 Um programa completo: classificação da turma

Em um programa real, os dados ordenados raramente são só números: cada nota pertence a um aluno. Quando nomes e notas estão em dois vetores **paralelos**, em que `nomes[i]` e `notas[i]` são do mesmo aluno, a ordenação precisa trocar os dois vetores juntos, senão os nomes se separam das notas. O programa abaixo monta a classificação da turma, da maior nota para a menor, e depois procura a posição de um aluno.

```
#include <iostream>
#include <string>
using namespace std;

const int N = 6;

// seleção pela maior nota, levando o nome junto
void ordenarPorNota(string nomes[], double notas[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int iMaior = i;
        for (int j = i + 1; j < n; j++) {
            if (notas[j] > notas[iMaior]) iMaior = j;
        }
        swap(notas[i], notas[iMaior]);
        swap(nomes[i], nomes[iMaior]);    // o nome vai junto
    }
```

```
}

int buscarNome(const string nomes[], int n, const string &nome) {
    for (int i = 0; i < n; i++) {
        if (nomes[i] == nome) return i;
    }
    return -1;
}

int main() {
    string nomes[N] = {"Ana", "Bruno", "Carla",
                       "Diego", "Elisa", "Fabio"};
    double notas[N] = {7.5, 9.0, 6.0, 9.5, 8.0, 5.5};
    ordenarPorNota(nomes, notas, N);
    for (int i = 0; i < N; i++) {
        cout << i + 1 << "o lugar: " << nomes[i] << " ("
             << notas[i] << ")" << endl;
    }
    string nome;
    cout << "Posição de quem? ";
    cin >> nome;
    int pos = buscarNome(nomes, N, nome);
    if (pos >= 0) {
        cout << nome << " ficou em " << pos + 1 << "o lugar"
             << endl;
    } else {
        cout << nome << " não está na turma" << endl;
    }
    return 0;
}
// 1o lugar: Diego (9.5)    4o lugar: Ana (7.5)
// 2o lugar: Bruno (9)      5o lugar: Carla (6)
// 3o lugar: Elisa (8)      6o lugar: Fabio (5.5)
// Posição de quem? Elisa  ->  Elisa ficou em 3o lugar
```

A busca pelo nome é sequencial porque, depois de ordenado por nota, o vetor de nomes não está em ordem alfabética, e a busca binária exigiria isso. Os vetores paralelos funcionam, mas é fácil esquecer de trocar um deles. No Capítulo 11, os dados de cada aluno vão ser reunidos em um único registro, e a ordenação passa a mover o registro inteiro de uma vez.

Erros Comuns

- **Usar a busca binária em um vetor desordenado.** Ela não dá erro: simplesmente responde errado.
- **Errar a atualização dos limites na busca binária.** `ini = meio` em vez de `meio + 1` pode deixar o laço preso para sempre.
- **Confundir a posição com o valor.** A busca devolve **onde** o valor está, e -1 (ou 0, no Portugol com índices a partir de 1) quando não acha. Teste esse caso antes de usar a posição.
- **Errar os limites dos laços de ordenação.** Na bolha, `v[j + 1]` exige `j < fim`; com `j <= fim`, o último acesso sai do vetor.

- **Inverter a ordem das condições na inserção.** `v[j] > atual && j >= 0` acessa `v[-1]` antes de testar o índice.
- **Ordenar um vetor paralelo e esquecer o outro.** Os nomes deixam de corresponder às notas.
- **Escrever a própria ordenação sem necessidade.** Em programas reais, use `sort`; os algoritmos simples servem para aprender e para vetores pequenos.

Capítulo 11 – Introdução às estruturas de dados

No fim do Capítulo 10, os dados de cada aluno estavam espalhados em dois vetores paralelos, e bastava esquecer de trocar um deles para os nomes se separarem das notas. O problema é de **organização**: os programas ficam mais simples e mais seguros quando os dados são guardados de uma forma que combina com o uso que se faz deles. Essa é a ideia de **estrutura de dados**. Neste capítulo você vai conhecer os **registros**, que reúnem dados diferentes sobre a mesma coisa, e três estruturas clássicas, cada uma com suas regras de entrada e saída: a **pilha**, a **fila** e a **lista encadeada**. Para construir a última, você também vai dar os primeiros passos com **ponteiros**.

11.1 Registros

Um **registro** reúne, sob um único nome, várias informações que podem ser de tipos diferentes. Cada informação é um **campo**. Um aluno tem nome (texto), matrícula (inteiro) e nota (real); em vez de três variáveis soltas, um registro `Aluno` guarda as três juntas. Em C++, um registro é declarado com `struct`, e os campos são acessados com um ponto:

```
#include <iostream>
#include <string>
using namespace std;

struct Aluno {                  // um novo tipo, com três campos
    string nome;
    int matricula;
    double nota;
};

int main() {
    Aluno a;
    a.nome = "Ana";
    a.matricula = 2026001;
    a.nota = 8.5;
    Aluno b = {"Bruno", 2026002, 7.0};   // todos de uma vez
    cout << a.nome << " (" << a.matricula << "): " << a.nota
         << endl;                        // Ana (2026001): 8.5
    Aluno c = a;                         // copia tudo
    c.nota = 10;
    cout << a.nota << " " << c.nota << " " << b.nome << endl;
    return 0;                            // 8.5 10 Bruno
}
```

A declaração `struct Aluno` cria um **tipo novo**, que se usa como `int` ou `string`: dá para declarar variáveis, copiar com `=`, passar para funções e devolver com `return`. Como um registro pode ser grande, a passagem por referência constante (Capítulo 6) é a forma usual de entregá-lo a uma função que só vai lê-lo: `void mostrar(const Aluno &a)`.

Na notação de Portugol usada em muitos livros, o registro é declarado com `tipo ... registro`. As ferramentas de Portugol variam no suporte a esse recurso, mas a ideia é

a mesma:

```
algoritmo "Registro"
tipo Aluno = registro
   nome: caractere
   matricula: inteiro
   nota: real
fimregistro
var
   a: Aluno
inicio
   a.nome <- "Ana"
   a.matricula <- 2026001
   a.nota <- 8.5
   escreval(a.nome, ": ", a.nota)
fimalgoritmo
```

11.2 Vetores de registros

Registros e vetores se combinam naturalmente: um vetor de `Aluno` é a turma inteira, e `turma[i].nota` é a nota do aluno `i`. Agora, ao ordenar, cada aluno se move **inteiro**, e o problema dos vetores paralelos desaparece. Melhor ainda: o `sort` da biblioteca aceita um terceiro argumento, uma função que diz se um elemento deve vir antes de outro, e assim ordena registros pelo critério que se quiser.

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;

struct Aluno {
    string nome;
    double nota;
};

// a deve vir antes de b? Sim, se tiver nota maior.
bool maiorNota(const Aluno &a, const Aluno &b) {
    return a.nota > b.nota;
}

int main() {
    vector<Aluno> turma = {
        {"Ana", 7.5}, {"Bruno", 9.0}, {"Carla", 6.0},
        {"Diego", 9.5}, {"Elisa", 8.0}, {"Fabio", 5.5}
    };
    turma.push_back({"Gabi", 8.7});          // mais um cadastro
    sort(turma.begin(), turma.end(), maiorNota);
    for (size_t i = 0; i < turma.size(); i++) {
        cout << i + 1 << "o " << turma[i].nome << " ("
             << turma[i].nota << ")" << endl;
```

```
    }
    return 0;
}
// 1o Diego (9.5)  2o Bruno (9)  3o Gabi (8.7)  4o Elisa (8)
// 5o Ana (7.5)    6o Carla (6)  7o Fabio (5.5)
```

Compare com o programa da Seção 10.8: não há mais dois vetores para manter em sincronia, a ordenação é uma única linha, e acrescentar um campo novo, como a matrícula, não exige mudar a ordenação.

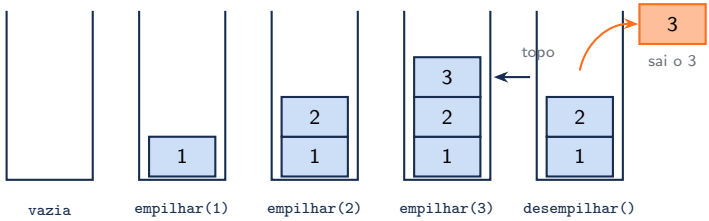
11.3 Estruturas de dados e suas operações

Uma **estrutura de dados** é uma forma de organizar dados na memória junto com as **operações** que se pode fazer sobre eles: inserir, remover, procurar, percorrer. O vetor é a mais simples: acesso direto a qualquer posição, mas inserir no meio exige deslocar todos os elementos seguintes. Cada estrutura faz algumas operações muito bem e outras mal, e escolher a estrutura certa é, muitas vezes, o que separa um programa rápido de um lento.

Ao estudar uma estrutura, vale separar duas perguntas. A primeira é **o que** ela faz: quais operações oferece e com que regras. A segunda é **como** ela faz: com vetores, com ponteiros, de que jeito. A descrição só da primeira parte é chamada de **tipo abstrato de dados**. Quem usa uma pilha precisa saber que o último a entrar é o primeiro a sair; não precisa saber se por dentro há um vetor. As próximas seções apresentam três tipos abstratos clássicos e uma forma simples de implementar cada um.

11.4 Pilha

Uma **pilha** funciona como uma pilha de pratos: um prato novo é colocado no topo, e o único que pode ser retirado é o do topo. O último a entrar é o primeiro a sair, regra conhecida pela sigla em inglês **LIFO** (*last in, first out*). As operações são **empilhar** (*push*), **desempilhar** (*pop*), consultar o **topo** e verificar se a pilha está **vazia**.



o último a entrar é o primeiro a sair (LIFO)

Pilhas estão por toda parte na computação: o botão "desfazer" de um editor guarda as ações em uma pilha, o botão "voltar" do navegador guarda as páginas visitadas, e a pilha de chamadas do Capítulo 6 é, como o nome diz, uma pilha. A implementação mais simples usa um vetor e um contador, `topo`, que indica quantos elementos há (e, portanto, a próxima posição livre):

```
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

struct Pilha {
    char dados[MAX];
    int topo = 0;              // quantos elementos há
};

bool vazia(const Pilha &p) { return p.topo == 0; }

bool empilhar(Pilha &p, char x) {
    if (p.topo == MAX) return false;   // pilha cheia
    p.dados[p.topo] = x;
    p.topo++;
    return true;
}

char desempilhar(Pilha &p) {           // só se não estiver vazia
    p.topo--;
    return p.dados[p.topo];
}

int main() {
    Pilha p;
    string palavra = "ROMA";
    for (char c : palavra) empilhar(p, c);
    while (!vazia(p)) cout << desempilhar(p);   // ao contrário
    cout << endl;                               // AMOR
    return 0;
}
```

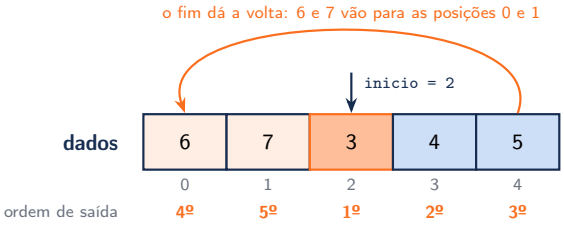
As funções recebem a pilha por referência, porque empilhar e desempilhar precisam alterá-la. O valor inicial `topo = 0`, escrito na própria declaração do registro, garante que toda pilha nova começa vazia. E quem chama `desempilhar` precisa verificar antes se a pilha não está vazia: desempilhar de uma pilha vazia levaria `topo` a -1 e acessaria uma posição fora do vetor.

11.5 Fila

Uma **fila** funciona como a fila de um caixa: quem chega entra no **fim**, e quem é atendido sai do **início**. O primeiro a entrar é o primeiro a sair, regra conhecida como **FIFO** (*first in, first out*). As operações são **enfileirar**, **desenfileirar**, consultar o primeiro e verificar se está vazia. Filas organizam tudo o que precisa ser atendido na ordem de chegada: documentos esperando a impressora, mensagens esperando envio, pedidos em um sistema de vendas.

Implementar uma fila com um vetor tem uma dificuldade. Se o início ficasse sempre na posição 0, cada saída obrigaria a deslocar todos os elementos uma posição para a frente. A solução é deixar o início andar: um índice `inicio` indica onde está o primeiro elemento, e um contador `tamanho` diz quantos há. Quando o fim chega à última posição

do vetor, ele **dá a volta** e continua na posição 0, que já foi liberada. Por isso essa implementação se chama **fila circular**, e o resto da divisão (`%`) faz a volta, como na cifra de César do Capítulo 8.



depois de enfileirar 1, 2, 3 e 4, desenfileirar duas vezes e enfileirar 5, 6 e 7:
`tamanho` = 5, e a próxima posição livre seria (2 + 5) % 5 = 2, ou seja, a fila está cheia

```
#include <iostream>
using namespace std;

const int MAX = 5;              // pequeno, para ver a volta

struct Fila {
    int dados[MAX];
    int inicio = 0;             // posição do primeiro
    int tamanho = 0;            // quantos elementos há
};

bool enfileirar(Fila &f, int x) {
    if (f.tamanho == MAX) return false;        // fila cheia
    int fim = (f.inicio + f.tamanho) % MAX;    // dá a volta
    f.dados[fim] = x;
    f.tamanho++;
    return true;
}

int desenfileirar(Fila &f) {   // só se não estiver vazia
    int x = f.dados[f.inicio];
    f.inicio = (f.inicio + 1) % MAX;
    f.tamanho--;
    return x;
}

int main() {
    Fila f;
    for (int x = 1; x <= 4; x++) enfileirar(f, x);   // 1 2 3 4
    int a = desenfileirar(f);
    int b = desenfileirar(f);
    cout << a << " " << b << endl;                   // 1 2
    for (int x = 5; x <= 7; x++) enfileirar(f, x);   // 6 e 7 dão
    while (f.tamanho > 0) {                          // a volta
        cout << desenfileirar(f) << " ";
```

```
    }
    cout << endl;                                    // 3 4 5 6 7
    return 0;
}
```

Depois das duas primeiras saídas, as posições 0 e 1 do vetor ficam livres. O 5 ocupa a última posição, a 4, e o 6 e o 7 dão a volta e ocupam as posições 0 e 1. Mesmo assim, eles saem na ordem certa, porque o `inicio` também anda em círculo.

11.6 Ponteiros e memória dinâmica

Pilhas e filas feitas com vetores têm um limite: a capacidade `MAX`, decidida antes de o programa rodar. Para estruturas que crescem e encolhem livremente, é preciso criar espaço na memória **durante a execução**, e para isso a linguagem C++ usa **ponteiros**.

Um **ponteiro** é uma variável que guarda um **endereço** de memória, ou seja, a localização de outra variável (Capítulo 3). O operador `&` dá o endereço de uma variável, e o operador `*` acessa o valor que está no endereço guardado pelo ponteiro:

```
int x = 10;
int *p = &x;           // p guarda o endereço de x
cout << *p << endl;    // 10: o valor que está nesse endereço
*p = 20;               // altera x através de p
cout << x << endl;     // 20

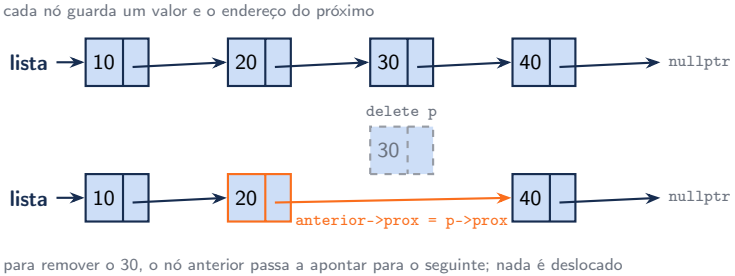
int *q = new int(5);   // cria um int durante a execução
cout << *q << endl;    // 5
delete q;              // devolve a memória ao sistema
q = nullptr;           // agora q não aponta para nada
```

O operador `new` reserva memória para um valor e devolve o endereço dela. Essa memória não some no fim do bloco, como uma variável local: ela fica reservada até o programa a devolver com `delete`. Esquecer o `delete` causa um **vazamento de memória**: a memória fica ocupada sem uso, e um programa que roda por muito tempo pode esgotá-la. Usar um ponteiro depois do `delete` é ainda pior, porque o endereço pode já pertencer a outra coisa. O valor especial `nullptr` indica um ponteiro que não aponta para nada, e é a forma segura de marcar "nenhum".

Os ponteiros são uma das partes mais delicadas do C++, e a programação moderna nessa linguagem procura usá-los o mínimo possível, preferindo `vector`, `string` e as estruturas da biblioteca, que cuidam da memória sozinhas. Mas eles são a melhor forma de entender como as estruturas encadeadas funcionam por dentro.

11.7 Lista encadeada

Em uma **lista encadeada**, cada elemento fica em um **nó**, um registro criado com `new` que guarda o valor e um ponteiro para o **próximo** nó. Os nós podem estar em qualquer lugar da memória: o que os une é a corrente de ponteiros. A lista em si é só um ponteiro para o primeiro nó, e o último nó aponta para `nullptr`.



A grande vantagem da lista é que inserir e remover não exige deslocar nada: basta **religar os ponteiros**. Para inserir no início, o novo nó aponta para o antigo primeiro e passa a ser o primeiro. Para remover um nó, o anterior passa a apontar para o seguinte, e o nó removido é devolvido com `delete`. A desvantagem é que não há acesso direto: para chegar ao quinto elemento, é preciso passar pelos quatro primeiros.

```
#include <iostream>
using namespace std;

struct No {
    int valor;
    No *prox;                   // o próximo nó (nullptr no último)
};

void inserirNoInicio(No *&lista, int x) {
    No *novo = new No;
    novo->valor = x;
    novo->prox = lista;         // aponta para o antigo primeiro
    lista = novo;               // e passa a ser o primeiro
}

void mostrar(const No *lista) {
    for (const No *p = lista; p != nullptr; p = p->prox) {
        cout << p->valor << " -> ";
    }
    cout << "fim" << endl;
}

bool remover(No *&lista, int x) {
    No *anterior = nullptr;
    No *p = lista;
    while (p != nullptr && p->valor != x) {   // procura x
        anterior = p;
        p = p->prox;
    }
    if (p == nullptr) return false;           // não achou
    if (anterior == nullptr) lista = p->prox; // era o primeiro
    else anterior->prox = p->prox;            // pula o nó
    delete p;
```

```
    return true;
}

void liberar(No *&lista) {
    while (lista != nullptr) {
        No *p = lista;
        lista = lista->prox;
        delete p;
    }
}

int main() {
    No *lista = nullptr;                        // lista vazia
    int valores[] = {40, 30, 20, 10};
    for (int x : valores) inserirNoInicio(lista, x);
    mostrar(lista);              // 10 -> 20 -> 30 -> 40 -> fim
    remover(lista, 30);
    mostrar(lista);              // 10 -> 20 -> 40 -> fim
    liberar(lista);              // devolve todos os nós
    return 0;
}
```

Três detalhes de sintaxe aparecem aqui. A seta `p->valor` é a forma curta de `(*p).valor`: o campo `valor` do nó para onde `p` aponta. O parâmetro `No *&lista` é uma **referência para um ponteiro**: `inserirNoInicio` e `remover` precisam mudar **qual** é o primeiro nó, e por isso alteram o próprio ponteiro de quem chamou. E a função `liberar` percorre a lista devolvendo cada nó, porque todo `new` precisa de um `delete`.

11.8 As estruturas da biblioteca

Implementar pilhas, filas e listas é o melhor jeito de entendê-las, mas em programas reais usam-se as versões prontas da biblioteca padrão do C++, testadas e eficientes, que cuidam da memória sozinhas: `stack` (pilha), `queue` (fila), `list` (lista duplamente encadeada) e o já conhecido `vector`.

```
#include <iostream>
#include <string>
#include <stack>
#include <queue>
using namespace std;

int main() {
    stack<int> pilha;
    pilha.push(1);
    pilha.push(2);
    pilha.push(3);
    cout << pilha.top() << endl;       // 3: o último que entrou
    pilha.pop();                       // remove o 3
    cout << pilha.top() << " " << pilha.size() << endl;   // 2 2
```

```
    queue<string> fila;
    fila.push("Ana");
    fila.push("Bruno");
    cout << fila.front() << endl;      // Ana: a primeira
    fila.pop();
    cout << fila.front() << " " << fila.empty() << endl;
    return 0;                          // Bruno 0
}
```

Repare que `pop` remove o elemento, mas não o devolve: para ler e remover, consulta-se antes com `top` (na pilha) ou `front` (na fila). A tabela resume quando usar cada estrutura:

Estrutura	Faz bem	Faz mal
`vector`	acesso por posição; inserir e remover no fim	inserir e remover no início ou no meio
`list`	inserir e remover em qualquer ponto já localizado	acesso por posição (precisa percorrer)
`stack`	último a entrar, primeiro a sair	qualquer acesso que não seja o topo
`queue`	primeiro a entrar, primeiro a sair	qualquer acesso que não seja as pontas

Árvores, tabelas hash, grafos e outras estruturas, com a análise detalhada do custo de cada operação, são o assunto de um estudo próprio de estruturas de dados.

11.9 Um programa completo: parênteses balanceados

Compiladores, editores de código e calculadoras precisam verificar se os parênteses, colchetes e chaves de uma expressão estão **balanceados**: cada um que abre precisa ser fechado, pelo símbolo correspondente e na ordem certa. `{a[i] * (b + c)}` está correto; `(a + b]` e `(a + [b) - c]` não estão.

A pilha resolve o problema com elegância, porque o último símbolo aberto é sempre o primeiro que precisa ser fechado. Percorre-se a expressão: cada símbolo que abre é empilhado; cada símbolo que fecha precisa corresponder ao que está no topo, que é desempilhado. No fim, a pilha precisa estar vazia, senão sobrou algo aberto.

```
#include <iostream>
#include <string>
#include <stack>
using namespace std;

// o símbolo que abre o par de quem fecha
char abertura(char fecha) {
    if (fecha == ')') return '(';
    if (fecha == ']') return '[';
    return '{';
}
```

```
bool balanceado(const string &expr) {
    stack<char> pilha;
    for (char c : expr) {
        if (c == '(' || c == '[' || c == '{') {
            pilha.push(c);                          // abre: guarda
        } else if (c == ')' || c == ']' || c == '}') {
            if (pilha.empty()) return false;  // fecha sem abrir
            if (pilha.top() != abertura(c)) {
                return false;                 // par trocado
            }
            pilha.pop();                            // par completo
        }
    }
    return pilha.empty();                       // sobrou aberto?
}

int main() {
    string testes[] = {"{a[i] * (b + c)}", "(a + b]",
                       "(a + [b) - c]", "((x)", "x) + (y"};
    for (const string &t : testes) {
        cout << t << ": " << (balanceado(t) ? "ok" : "erro")
             << endl;
    }
    string expr;
    cout << "Sua expressão: ";
    getline(cin, expr);
    cout << (balanceado(expr) ? "balanceada" : "não balanceada")
         << endl;
    return 0;
}
// {a[i] * (b + c)}: ok     (a + b]: erro     (a + [b) - c]: erro
// ((x): erro               x) + (y: erro
```

Cada um dos testes exercita um tipo de erro diferente: o par trocado (`(a + b]`), o cruzamento (`(a + [b) - c]`), o símbolo que sobra aberto (`((x)`) e o que fecha sem ter sido aberto (`x) + (y`), que tem o mesmo número de aberturas e fechamentos, mas na ordem errada. Contar os símbolos não bastaria para pegar os três últimos casos; a pilha pega todos.

Erros Comuns

- **Esquecer o ponto e vírgula depois do `struct`.** A declaração termina com `};`.
- **Passar registros grandes por valor sem necessidade.** Use `const Aluno &a` quando a função só lê.
- **Desempilhar de uma pilha vazia ou desenfileirar de uma fila vazia.** Verifique antes com `vazia` ou `empty()`.
- **Esquecer a volta na fila circular.** Sem o `% MAX`, os índices passam do fim do vetor.
- **Usar um ponteiro que não aponta para nada.** Acessar `p->valor` com `p` igual a `nullptr`, ou com `p` não inicializado, encerra o programa ou corrompe a

memória.

- **Esquecer o `delete`.** Cada `new` precisa de um `delete`, senão há vazamento de memória.
- **Usar um nó depois do `delete`.** Leia o `prox` antes de apagar o nó, como em `liberar`.
- **Perder o início da lista.** Percorra com um ponteiro auxiliar (`p`); se o próprio `lista` andar, o começo da lista se perde.
- **Esperar que `pop()` devolva o elemento.** Nas estruturas da biblioteca, leia com `top()` ou `front()` e depois remova com `pop()`.